CogniShield: Edge-Based Adversarial Immunization for Digital Media

BITE497J – Project I

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Information Technology

by

ROHIT KUMAR
23BIT0030

SHIVANSH RAI
23BIT0067

AVIRAL BHAN
23BIT0241

Under the guidance of
Dr. Angulakshmi M

School of Computer Science Engineering and Information Systems
VIT, Vellore



November, 2026

CogniShield: Edge-Based Adversarial Immunization for Digital Media

BITE497J – Project I

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Information Technology

by

ROHIT KUMAR
23BIT0030

SHIVANSH RAI
23BIT0067

AVIRAL BHAN
23BIT0241

Under the guidance of
Dr. Angulakshmi M

School of Computer Science Engineering and Information Systems
VIT, Vellore



November, 2026

DECLARATION

We hereby declare that the BITE497J – Project I thesis entitled "**CogniShield: Edge-Based Adversarial Immunization for Digital Media**" submitted by us, for the award of the degree of ***Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems*** to VIT is a record of bonafide work carried out by us under the supervision of **Dr. Angulakshmi M, Associate Professor, SCORE, VIT, Vellore.**

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

Rohit Kumar (23BIT0030)
Shivansh Rai (23BIT0067)
Aviral Bhan (23BIT0241)

CERTIFICATE

This is to certify that the BITE497J – Project I thesis entitled "**CogniShield: Edge-Based Adversarial Immunization for Digital Media**" submitted by **Rohit Kumar (23BIT0030), Shivansh Rai (23BIT0067) & Aviral Bhan (23BIT0241)**, SCORE, VIT, for the award of the degree of ***Bachelor of Technology in Information Technology, School of Computer Science Engineering and Information Systems***, is a record of bonafide work carried out by them under my supervision during the period, 06.07.2026 to 03.11.2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore
Date:

Signature of the Guide
Dr. Angulakshmi M

Internal Examiner **External Examiner**

Head of the Department
Department of Information Technology

ACKNOWLEDGEMENT

It is our pleasure to express with a deep sense of gratitude to our **BITE497J – Project I** guide **Dr. Angulakshmi M**, Associate Professor, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for constant guidance and continual encouragement in our endeavor. Our association with the guide is not confined to academics only, but it is a great opportunity on our part to work with an intellectual and an expert in the field of **Image Security and Adversarial Machine Learning**.

We would like to express our heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Dr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T.**

Our whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Information Technology, **Dr. Arivuselvan K**, Information Technology Project Coordinator **Dr. Suganya P**, SCORE School Project Coordinator **Dr. Karthikeyan P**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout our course of study in unlimited ways.

It is indeed a pleasure to thank our parents and friends who persuaded and encouraged us to take up and complete our project successfully. Last, but not least, we express our gratitude and appreciation to all those who have helped us directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: **Rohit Kumar, Shivansh Rai, Aviral Bhan**

Executive Summary

Generative image models can edit a downloaded photograph within seconds, yet most defences detect fakes only after the harm is done. This project presents CogniShield, a local prototype that protects an image before it is shared. The tool adds a seeded, edge-aware, high-frequency perturbation limited to at most 8 of 255 intensity levels, refines it with feedback from the real JPEG encoder, measures PSNR, SSIM, MSE and absolute differences, tests JPEG robustness at several qualities, and signs the protected PNG with Ed25519 over a SHA-256 digest so that tampering is detectable. A Flask web interface and command-line tools run entirely on the user's computer, and an optional module compares SD-Turbo image-to-image outputs for the original and protected images. On 14 test images the mean PSNR was 41.56 dB and the mean SSIM 0.9800; protection took about 0.27 s per image, results were reproducible, and all 18 recorded manifests matched their files. About half of the perturbation survived JPEG qualities 95 and 75, but little survived quality 50 or rescaling, and no visible disruption of SD-Turbo was observed for the test portrait. A second evaluation on 500 MS-COCO images gave a mean PSNR of 39.06 dB and a mean SSIM of 0.9774; 99.6 percent of the images met the quality thresholds, 26.6 percent met the JPEG criterion at qualities 95, 75 and 50, and 10.2 percent produced SD-Turbo outputs that diverged beyond the preset threshold. CogniShield is therefore a transparent, measurable baseline for stronger model-aware protection, not a guarantee of immunity.

CONTENTS

Page No.

Acknowledgement i

Executive Summary ii

Table of Contents iii

List of Figures vi

List of Tables viii

Abbreviations xi

Symbols and Notations xii

1 INTRODUCTION **1**

1.1 Objective 2

1.2 Motivation 3

1.3 Background 4

1.4 Organization of the Report 8

2 PROJECT DESCRIPTION AND GOALS **9**

2.1 Problem Statement 9

2.2 Scope of the Project 10

2.3 Threat Model 10

2.4 Goals and Objectives 12

2.5 System Overview 12

2.6 Literature Survey 13

2.7 Research Gap and Contribution 16

2.8 Deliverables 17

2.9 Use Cases and Requirement Traceability 17

3 TECHNICAL SPECIFICATION **19**

3.1 Hardware Requirements 19

3.2 Software Requirements 19

3.3 Libraries and Dependencies 20

3.4 Functional Requirements 20

3.5 Non-Functional Requirements 21
3.6 System Architecture 21
3.7 Interface Specification 23
3.8 Data Specification 25
3.9 Environment Setup 26
3.10 Output Tree of a Run 26
3.11 Assumptions and Dependencies 27

4 DESIGN APPROACH AND DETAILS 28
4.1 Design Approach / Materials & Methods 28
4.2 Codes and Standards 45
4.3 Constraints, Alternatives and Tradeoffs 46

5 SCHEDULE, TASKS AND MILESTONES 49
5.1 Work Breakdown 49
5.2 Timeline 49
5.3 Milestones 50
5.4 Risks and Mitigation 50
5.5 Quality Assurance and Review Practice 51

6 PROJECT DEMONSTRATION 52
6.1 Demonstration Setup 52
6.2 Walkthrough of the Web Interface 52
6.3 Demonstration 1: Portrait Protection 53
6.4 Demonstration 2: JPEG Variants 55
6.5 Demonstration 3: Signed Provenance 55
6.6 Demonstration 4: SD-Turbo Target Test 56
6.7 Demonstration 5: Other Images 57
6.8 Demonstration 6: Command-Line and Batch Workflow 58
6.9 Behaviour in Error Situations 59

7 COST ANALYSIS / RESULT & DISCUSSION 60
7.1 Experimental Setup 60

7.2 Imperceptibility of the Protection 62
7.3 Effect of the Perturbation Budget 64
7.4 Effect of the Compression Feedback 65
7.5 Robustness to JPEG Compression 68
7.6 Robustness to Other Transformations 70
7.7 Edge-Aware Allocation of the Perturbation 71
7.8 Reproducibility and Seed Sensitivity 72
7.9 Provenance Verification 73
7.10 Effect on the SD-Turbo Target Model 74
7.11 Running Time and Scalability 75
7.12 Evaluation on the MS-COCO Dataset 76
7.13 Discussion 80
7.14 Cost Analysis 81
7.15 Ethical and Societal Considerations 82

8 SUMMARY 84

8.1 Summary of the Work 84
8.2 Key Findings 85
8.3 Limitations 86
8.4 Recommendations and Future Work 87
8.5 Conclusion 88

9 REFERENCES 89

APPENDIX A

List of Figures

Figure No.	Title	Page No.
1.1	Reactive Detection Versus Proactive Immunization	8
2.1	Threat Model and Actors in the CogniShield Scenario	11
2.2	System Context of CogniShield	13
3.1	Layered Software Stack of CogniShield	22
3.2	Module Structure of the CogniShield Library	22
3.3	Sequence of Calls for a Protection Request	23
4.1	End-to-End Processing Pipeline of CogniShield	29
4.2	Flowchart of the Protection Algorithm with JPEG Feedback	33
4.3	Signing and Verification Protocol of the Provenance Module	37
4.4	Process Isolation of the Optional SD-Turbo Test	38
4.5	Flowchart of a Protection Request in the Web Application	39
4.6	Level-One Data Flow Diagram of CogniShield	41
4.7	Flowchart of the Dataset Evaluation Workflow of run_evaluation.py	44
5.1	Project Timeline from 06 July to 03 November 2026	50
6.1	Screenshot of the CogniShield Web Interface in Dark Mode After a Protection Run	53
6.2	Portrait Run at the Default Settings: Original, Protected, Difference ×12 and Noise Preview	54
6.3	Protected Portrait After JPEG Compression at Qualities 95, 75 and 50	55
6.4	SD-Turbo Image-to-Image Outputs for the Original and the Protected Portrait	57
6.5	Difference Between the Two SD-Turbo Outputs, Magnified Twelve Times, and Its Histogram	57
6.6	Protection of a Colour Portrait From the Sample Data	58

6.7	Protection of a Handwritten Notebook Page	58
7.1	Contact Sheet of the 14 Test Images	61
7.2	PSNR and SSIM of the Protected Images at the Default Settings	63
7.3	Distribution of Pixel Changes for the Colour Portrait (Budget 8/255)	64
7.4	Effect of the Perturbation Budget on PSNR, SSIM and Maximum Change	65
7.5	Effect of the Number of Feedback Iterations on PSNR and Perturbation Retention	66
7.6	Correlation of the Final Perturbation With the Initial Noise and With the Image’s Own JPEG Residual	67
7.7	Mean PSNR and SSIM After JPEG Compression, With and Without Protection	69
7.8	Mean Perturbation Retention Under Compression and Filtering (Error Bars Show One Standard Deviation)	71
7.9	Ratio of Mean Perturbation at Edges to Mean Perturbation in Flat Areas	72
7.10	PSNR of the Portrait Image for Ten Different Seeds	73
7.11	Protection Time Versus Image Size	75
7.12	Charts Generated by the Evaluation Runner for the 500 MS-COCO Images	77
7.13	Evaluation Dashboard of the Web Application for the coco500 Run	79

List of Tables

Table No.	Title	Page No.
1.1	Detection, Provenance and Immunization Compared	7
1.2	Organization of the Report	8
2.1	Scope of the Prototype	10
2.2	Attack Scenarios and Expected Outcomes	12
2.3	Project Goals and Acceptance Criteria	12
2.4	Related Work and Position of CogniShield	13
2.5	Literature Survey of Seventeen Recent Papers on Adversarial Image Protection	14
2.6	Use Cases	17
2.7	Traceability From Goals to Requirements and Evidence	18
3.1	Hardware Requirements	19
3.2	Software Requirements	20
3.3	Python Libraries and Their Roles	20
3.4	Functional Requirements	20
3.5	Non-Functional Requirements	21
3.6	Responsibilities of the Library Modules	23
3.7	HTTP Endpoints of the Web Application	24
3.8	Protection Controls and Their Effects	24
3.9	Command-Line Programs and Options	24
3.10	Folder Layout of the Project	25
3.11	Fields of the Provenance Manifest	26
4.1	Quality Measures Reported by CogniShield	35

4.2	Worked Example on a 4 × 4 Patch of the Colour Portrait (Red Channel, Default Settings)	43
4.3	Standards Followed by the Implementation	45
4.4	Design Constraints	46
4.5	Design Alternatives and Reasons for the Choices Made	46
5.1	Work Breakdown Structure	49
5.2	Project Milestones and Evidence of Completion	50
5.3	Risk Register	50
6.1	Steps of a Demonstration Session	53
6.2	Metrics of the Portrait Run (Budget 8/255, Four Iterations, Seed 42)	55
6.3	Functional Test Cases and Expected Behaviour	59
7.1	Test Images Used in the Experiments	60
7.2	Imperceptibility Metrics at the Default Settings (ε = 8/255, 4 Iterations, Seed 42)	62
7.3	Effect of the Perturbation Budget ε (Mean of 4 Images, 4 Iterations)	64
7.4	Effect of the Number of Compression-Feedback Iterations (Mean of 4 Images, ε = 8/255)	65
7.5	Composition of the Final Perturbation at the Default Settings	67
7.6	Mean Effect of JPEG Compression With and Without Protection (14 Images)	68
7.7	Quality of the Protected Image After JPEG Compression, Compared With the Original	68
7.8	Perturbation Retention ρ After JPEG Compression for Each Image	69
7.9	Perturbation Retention ρ Under Common Transformations (6 Images)	70
7.10	Edge-Aware Allocation of the Perturbation (Values on the 0–1 Pixel Scale)	72
7.11	Seed Sensitivity on the Portrait Image (ε = 8/255, 4 Iterations)	73

7.12	Provenance Integrity Results	74
7.13	SD-Turbo Test on the Portrait Image	74
7.14	Protection Time Versus Image Size (Three Runs, Median)	75
7.15	Configuration and Criteria of the 500-Image Evaluation	76
7.16	Aggregate Results on 500 MS-COCO Images	77
7.17	Distribution of the Per-Image Values (500 Images)	77
7.18	Assessment Against the Project Goals	80
7.19	Software and Resource Cost for the Prototype	81

List of Abbreviations

AI	Artificial Intelligence
APA	American Psychological Association (referencing style)
API	Application Programming Interface
CLI	Command-Line Interface
COCO	Common Objects in Context (MS-COCO image dataset)
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DCT	Discrete Cosine Transform
DDPM	Denoising Diffusion Probabilistic Model
EdDSA	Edwards-curve Digital Signature Algorithm
FGSM	Fast Gradient Sign Method
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
MAE	Mean Absolute Error (mean absolute delta)
MSE	Mean Squared Error
MVP	Minimum Viable Product
PEM	Privacy-Enhanced Mail (text key encoding)
PGD	Projected Gradient Descent
PNG	Portable Network Graphics
PSNR	Peak Signal-to-Noise Ratio
RGB	Red, Green, Blue
SD	Stable Diffusion
SHA	Secure Hash Algorithm
SSIM	Structural Similarity Index Measure
UI	User Interface
UTC	Coordinated Universal Time
WEBP	Web Picture Format

Symbols and Notations

ε	Perturbation budget (maximum absolute change per pixel, 0–1 scale)
E	Perturbation budget in 8-bit levels, $\varepsilon = E/255$
δ	Perturbation added to the original image
x	Original image (floating-point array in [0, 1])
x_p	Protected image
ξ	Raw seeded Gaussian noise field
n	Normalised high-frequency noise
m	Edge mask
w	Edge weight (default 0.65)
g	Gradient magnitude of the grey-level image
λ	Compression weight (default 0.16)
N	Number of feedback iterations
Q	Set of JPEG qualities used in the feedback loop
J_q	JPEG round trip at quality q
f_k	Compression residual at iteration k
ρ_T	Perturbation retention under transformation T
r	Ratio of edge to flat perturbation
μ, σ	Local mean and standard deviation (SSIM)
H, W	Image height and width in pixels
h	SHA-256 digest of the protected file
σ_{sig}	Ed25519 signature

CHAPTER 1
INTRODUCTION

Digital images have become the default way in which people record events, express identity and build trust online. A single photograph posted to a social platform can be copied, edited and re-shared within minutes, usually without the knowledge of the person who took it or the person who appears in it. For most of the history of digital photography this was a manageable risk, because convincing manipulation required skill, time and specialised software. That barrier has now collapsed. Generative image models based on diffusion can take an ordinary photograph, add controlled noise to it and denoise it again under a text prompt, producing a new image that keeps the identity and layout of the original but changes its content. A few lines of code, or a single button in a web interface, are enough to turn a harmless portrait into a fabricated scene.

The response of the research and industry community to this problem has largely been reactive. Deepfake detectors, forensic classifiers and platform moderation pipelines try to decide, after a manipulated image already exists, whether it is real; surveys of the field are given by Mirsky and Lee (2021) and Tolosana et al. (2020), and benchmarks such as FaceForensics++ (Rössler et al., 2019) have driven much of the progress. Detection is valuable, but it is an arms race: every improvement in a generator removes artefacts that a detector relied upon, and a detector can only act after the harmful content has been created and often after it has been seen. A complementary and less explored idea is to act before the image is published. If the owner of a photograph could make a small, almost invisible change to the file that disturbs the way generative models interpret it, and could attach a cryptographic proof that the published file is the one they released, then the burden would shift from catching fakes to making fakes harder and traceable.

This report describes **CogniShield**, a local prototype that implements this idea for still images. CogniShield takes an input image on the user’s own machine and produces a protected copy in which every pixel differs from the original by at most a user-selected budget, typically 8 out of 255 intensity levels. The perturbation is built from seeded high-frequency noise, concentrated on edges and textured regions where it is least visible, and refined through a feedback loop that anticipates JPEG compression, the transformation that almost every shared image undergoes. The tool then measures

how close the protected image remains to the original using PSNR, SSIM, mean squared error and absolute pixel differences, tests how the protected image behaves after JPEG compression at several quality levels, and finally signs the exact bytes of the protected file with an Ed25519 signature computed over a SHA-256 digest. An optional module sends the original and the protected image through a locally installed SD-Turbo image-to-image model so that the effect on a real generative model can be inspected side by side.

The work was carried out as the BITE497J Project I of the B.Tech. programme in Information Technology at the School of Computer Science Engineering and Information Systems, VIT, Vellore. The software is written in Python 3.12, uses Flask for a browser-based interface that runs entirely on the local machine, and is also available as command-line tools for single-image and batch processing. Chapter 1 introduces the objective, motivation and technical background. Chapter 2 defines the problem, scope, threat model and goals and reviews related work. Chapter 3 gives the technical specification, and Chapter 4 describes the design and algorithms in detail. Chapter 5 presents the schedule and milestones, Chapter 6 demonstrates the working system, and Chapter 7 reports the quantitative results, discussion and cost analysis. Chapter 8 summarises the findings, and Chapter 9 lists the references.

1.1 Objective

The overall objective of this project is to design, implement and evaluate a local, edge-based system that *immunizes* digital images against misuse by generative models and attaches verifiable provenance to the protected output. The system must work on an ordinary computer, must not upload the user's image to any external service, and must report honestly how much protection it provides and what that protection costs in visual quality.

This overall objective is divided into the following specific objectives:

- ✔ To generate a protected version of an input image by adding a bounded, seeded, edge-aware, high-frequency perturbation whose maximum per-pixel change is controlled by a single budget parameter.
- ✔ To make the perturbation aware of lossy compression by feeding the residual of JPEG round trips back into the perturbation for a configurable number of

iterations.

- ✔ To quantify the visual cost of protection with PSNR, SSIM, mean squared error, mean absolute delta and maximum absolute delta, and to compare the protected image with the original after JPEG compression at qualities 95, 85, 75, 60 and 50.
- ✔ To create cryptographic provenance for every protected image by hashing the file with SHA-256, signing the digest with an Ed25519 private key stored locally, and recording the result in a portable JSON manifest that can be verified later.
- ✔ To provide an optional evaluation module that passes the original and protected images through a local SD-Turbo image-to-image pipeline with identical prompt and seed so that the behaviour of a real generative model can be compared.
- ✔ To package the pipeline as a responsive web interface and as command-line tools so that the system can be demonstrated, reproduced and extended by other students and researchers.

The success of the project is judged against the review metrics suggested at the outset: PSNR should generally stay above 30 dB and SSIM should stay close to 1.0 for the protected image, JPEG robustness must be reported at qualities 95, 75 and 50, and the comparison of original and protected target-model outputs must be available for inspection. Chapter 7 reports how the finished prototype performs against each of these criteria.

1.2 Motivation

Three developments motivate this work. The first is the **accessibility of image-to-image generation**. Latent diffusion models released with open weights can run on a consumer graphics card, and distilled variants such as SD-Turbo produce a result in one to four denoising steps. An attacker no longer needs a training set, a cluster or expertise: a downloaded photograph and a prompt are sufficient. Because image-to-image generation starts from the victim’s actual photograph, the output inherits the victim’s face, pose and background, which is precisely what makes it convincing and harmful.

The second is the **asymmetry between creation and detection**. A manipulated image can be created in seconds, but proving that it is fake requires specialised tools

and expert interpretation, and the proof usually arrives after the image has spread. Even when a detector is accurate on the data it was trained on, its accuracy is known to drop on content produced by new generators and on images that have been recompressed or resized by social platforms. A defence that only reacts therefore leaves the victim permanently one step behind. Immunization changes the economics: a small amount of computation by the image owner, performed once before upload, can increase the effort required from every later attacker.

The third is **privacy and control**. Many existing protection services require the user to upload the photograph they wish to protect, which means that the very image a person is worried about is sent to a third party. A tool that runs at the edge, on the user's own computer, removes that contradiction. CogniShield processes the image locally, stores its outputs and keys in project folders on the same machine, and serves its interface only at the loopback address 127.0.0.1.

There is also a practical, pedagogical motivation. Adversarial machine learning, applied cryptography and web engineering are usually taught separately. This project joins them in a single, inspectable pipeline of about one thousand lines of Python in which every stage can be run, measured and explained, which makes it a suitable vehicle for studying the trade-offs between imperceptibility, robustness and verifiability that real systems must balance.

Finally, the project is motivated by honesty about what protection can and cannot achieve. Claims that an image is ”immune” to manipulation are easy to make and hard to defend. The review-oriented design of CogniShield therefore emphasises measurement: every output is accompanied by metrics, the effect on the target model is shown rather than asserted, and the limitations are reported explicitly in Chapters 7 and 8.

1.3 Background

1.3.1 Generative models and image editing

Modern image generators are dominated by diffusion models. A denoising diffusion probabilistic model learns to reverse a gradual noising process: during training, images are corrupted with Gaussian noise over many steps and a neural network learns to predict the noise, and during generation the network is applied

repeatedly to turn pure noise into an image (Ho et al., 2020). Latent diffusion moves this process from pixel space into the compact latent space of an autoencoder, which makes high-resolution synthesis affordable and underlies the Stable Diffusion family of models (Rombach et al., 2022).

Editing an existing photograph with such a model is conceptually simple. In the approach known as SDEdit, the input image is first perturbed with a controlled amount of noise and then denoised under a new prompt, so that coarse structure from the input is preserved while details are regenerated (Meng et al., 2022). The amount of noise is controlled by a *strength* parameter between 0 and 1: a low value preserves almost everything, and a high value gives the model more freedom. Open-source toolkits expose this as an image-to-image pipeline, and it is exactly the operation that CogniShield uses as its target. SD-Turbo is a distilled model that applies adversarial diffusion distillation to compress the sampling process into very few steps (Sauer et al., 2023), which makes it a practical and fast target for local experiments.

1.3.2 Adversarial examples and image immunization

Neural networks are known to be sensitive to small, deliberately chosen perturbations. Szegedy et al. (2014) showed that imperceptible changes can cause an image classifier to output a wrong label, and Goodfellow et al. (2015) explained this with the linear behaviour of networks in high dimensions and proposed the fast gradient sign method. The central formulation is a constrained optimisation: find a perturbation δ of bounded magnitude that maximises a loss, as written below.

$$\max_{\delta} \; L\big(f(x+\delta),\, y\big) \qquad \text{s.t.} \qquad \|\delta\|_{\infty} \leq \varepsilon \tag{1.1}$$

In the Eqn. 1.1 the variable δ denotes the perturbation that is added to the input image, x denotes the input image, y denotes its true label, f denotes the classifier, L denotes the loss function that the attacker wants to maximise, and ε denotes the bound that controls the largest permitted change to any pixel.

The fast gradient sign method takes a single step in the direction of the sign of the gradient, as shown below. Iterating this step with projection onto the allowed set gives projected gradient descent, which is widely used as a strong first-order attack (Madry et al., 2018), and Kurakin et al. (2017) showed that such perturbations can

survive printing and re-photographing. Carlini and Wagner (2017) further showed that optimisation-based attacks defeat many proposed defences, which is why robustness claims have to be tested rather than assumed.

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x L(f(x), y)) \tag{1.2}$$

In the Eqn. 1.2 the variable x′ denotes the adversarial image, x denotes the original image, ε denotes the step size and at the same time the bound on the perturbation, sign denotes the element-wise sign function, and $\nabla_x L$ denotes the gradient of the loss L with respect to the input image.

The same machinery can be turned into a defensive tool. If the ”model” being attacked is the generative pipeline that an adversary will use to edit a photograph, then a perturbation that disrupts the encoder or the denoiser can make the edit fail or look unrealistic. This idea is called *immunization* in PhotoGuard (Salman et al., 2023), which perturbs an image so that diffusion-based editing produces unrealistic results. Related systems protect against other misuse: Fawkes cloaks personal photographs so that face-recognition models trained on them are misled (Shan et al., 2020), Glaze protects artists against style mimicry (Shan et al., 2023), Anti-DreamBooth protects against personalised fine-tuning of text-to-image models (Van Le et al., 2023), and AdvDM generates adversarial examples for diffusion models to prevent painting imitation (Liang et al., 2023). These systems generally rely on gradient access to a particular target model and on substantial computation, often on a GPU.

1.3.3 Lossy compression and its effect on perturbations

Almost every image that is shared online is stored or transmitted in a lossy format. JPEG divides an image into 8 × 8 blocks, applies a discrete cosine transform, divides the coefficients by a quantisation table whose coarseness is set by a quality factor, and entropy-codes the result (Wallace, 1991). High-frequency coefficients are quantised most aggressively because the eye is least sensitive to them, and this is precisely where small adversarial perturbations tend to reside. Dziugaite et al. (2016) observed that JPEG compression can undo much of the effect of small adversarial perturbations, and Shin and Song (2017) proposed generating perturbations with a differentiable approximation of JPEG so that they survive compression. CogniShield addresses the same issue in a lightweight way by repeatedly encoding the candidate

image with the real JPEG encoder and feeding the residual back into the perturbation, without requiring gradients.

1.3.4 Provenance and cryptographic signatures

Protecting an image from edits is only half of the problem; the owner also needs a way to show later which file is authentic. A cryptographic hash function such as SHA-256 maps any sequence of bytes to a fixed 256-bit digest, and any change to the input changes the digest unpredictably (NIST, 2015). A digital signature scheme allows the holder of a private key to sign a digest so that anyone with the public key can verify it. Ed25519 is a modern signature scheme based on the twisted Edwards form of Curve25519 with fast, deterministic signing and compact 64-byte signatures (Bernstein et al., 2012; Josefsson & Liusvaara, 2017). Industry efforts such as the Coalition for Content Provenance and Authenticity define signed manifests that travel with media files and link to certificate-based trust. CogniShield adopts the same principle at prototype scale, with a local key pair and a sidecar JSON manifest, which proves that the bytes of a protected image match what was signed without claiming anything about the scene it depicts.

1.3.5 Measuring visual quality

Imperceptibility cannot be left to opinion, so it is measured. The mean squared error between two images and the derived peak signal-to-noise ratio are simple pixel-level measures, and values above about 30 dB are commonly regarded as indicating low visible distortion for natural images (Huynh-Thu & Ghanbari, 2008). Their weakness is that they do not model perception (Wang & Bovik, 2009). The structural similarity index compares local luminance, contrast and structure and correlates better with perceived quality (Wang et al., 2004). CogniShield reports both families of measures, together with the mean and maximum absolute pixel change, and Chapter 4 gives their definitions.

Table 1.1 Detection, Provenance and Immunization Compared

Aspect	Post-hoc detection	Signed provenance	Immunization (this project)
When it acts	After a fake exists	At publication time	Before publication
What it needs	Trained classifier	Key pair and manifest	Image and a perturbation budget
What it shows	Likelihood of manipulation	Byte-level integrity of a file	Altered behaviour of a generator
Main weakness	Arms race with generators	Does not stop editing	No guarantee against every model
Where it runs	Platform or server	Any device	User’s own device

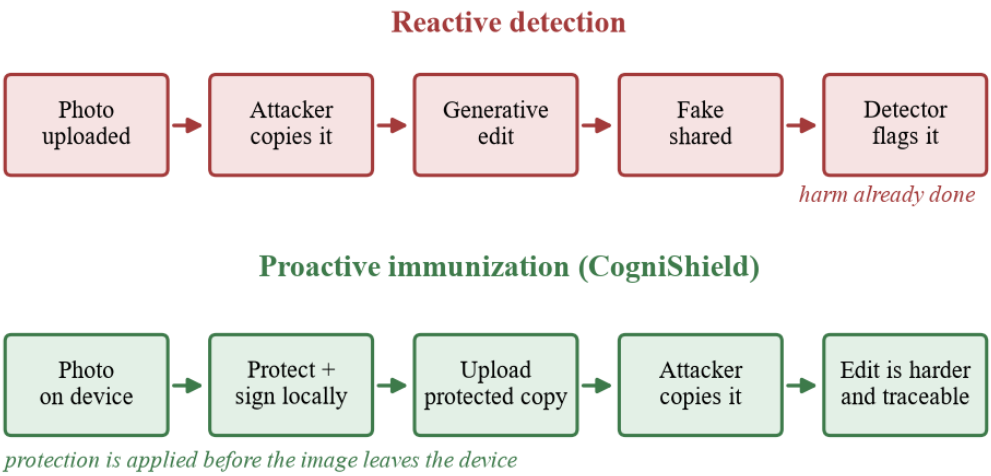


Fig. 1.1 Reactive Detection Versus Proactive Immunization

CogniShield combines the second and third columns of Table 1.1: it immunizes before publication and signs the result. Fig. 1.1 contrasts the reactive timeline of detection with the proactive timeline that this project follows.

1.4 Organization of the Report

The report follows the structure prescribed by the project template. Table 1.2 shows how the chapters relate to the work that was done and where each kind of evidence can be found.

Table 1.2 Organization of the Report

Chapter	Title	Content
1	Introduction	Objective, motivation and the technical background on generative editing, adversarial perturbations, compression, provenance and quality metrics
2	Project Description and Goals	Problem statement, scope, threat model, measurable goals, use cases, literature survey and research gap
3	Technical Specification	Hardware, software, libraries, requirements, architecture, interfaces and data formats
4	Design Approach and Details	Algorithms with equations, provenance protocol, model test, worked example, standards, constraints and trade-offs
5	Schedule, Tasks and Milestones	Work breakdown, timeline, milestones and risk register
6	Project Demonstration	Walkthrough of the interface and demonstrations on real runs
7	Cost Analysis / Result and Discussion	Experiments on 14 images and on 500 MS-COCO images, analysis, limitations, ethical considerations and cost
8	Summary	Findings, limitations, recommendations and future work
9	References	APA-style reference list
A	Appendix A	Sample code, output structure and additional figures

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

This chapter states the problem that CogniShield addresses, fixes the boundaries of what the prototype does and does not attempt, describes the adversary that the design assumes, lists the measurable goals, and places the project among related work. A precise statement of scope and threat model is important in security-oriented projects because protection claims are only meaningful relative to an attacker and a set of assumptions.

2.1 Problem Statement

Generative AI tools can manipulate images before their owners know that an attack has happened. Existing detection systems react after fake content has already been generated or shared, and the owner of the original photograph has no practical means of making the original harder to misuse or of proving later which version of a photograph is the one they released. The problem addressed by this project is therefore stated as follows.

Given a digital image held by its owner, produce a visually near-identical protected copy whose pixel values differ from the original by at most a small, user-controlled bound, that is designed to remain effective after JPEG compression, whose quality cost is measured and reported, and whose exact bytes are cryptographically signed so that later tampering is detectable. The entire process must run locally.

The problem has three coupled requirements that pull against each other. The perturbation must be **small** enough to be invisible to a human viewer, but **strong** enough to influence a generative model. It must be **robust** to the compression that platforms apply, although compression is designed to discard exactly the kind of fine detail in which small perturbations live. And the protected output must be **verifiable**, which requires that it is stored in a lossless format whose bytes do not change between signing and verification. The design in Chapter 4 resolves these tensions pragmatically and the evaluation in Chapter 7 measures how well each requirement is met.

2.2 Scope of the Project

This version of CogniShield focuses on still images only. It is designed as a review-ready minimum viable product (MVP) rather than a production service. Table 2.1 lists what is inside and outside the scope.

Table 2.1 Scope of the Prototype

In scope	Out of scope
Protecting a single RGB image at a time through the web interface, or a folder of images through the command line	Video, audio, or live camera streams
Seeded, edge-aware, high-frequency perturbation bounded by a per-pixel budget	Gradient-based optimisation against the internals of a specific generative model
JPEG-aware feedback and robustness testing at qualities 95, 85, 75, 60 and 50	Guarantees against cropping, heavy resizing, denoising, screenshots or print-and-scan
PSNR, SSIM, MSE, mean and maximum absolute delta	Perceptual studies with human participants
SHA-256 and Ed25519 signing and verification of the protected PNG	Public-key infrastructure, certificate authorities or key revocation
Optional side-by-side test with a local SD-Turbo image-to-image pipeline	Evaluation against face-swap, inpainting or text-to-video systems
Windows 10 and 11 with Python 3.12; optional NVIDIA GPU for SD-Turbo	Mobile applications, browser-only execution or cloud deployment

The input formats are PNG, JPEG and WEBP with a size limit of 200 MB, and the protected output is always written as a lossless PNG so that the signed bytes remain unchanged. The optional SD-Turbo module is disabled by default and the rest of the system works without it.

2.3 Threat Model

The threat model follows the proposal that motivated the project. The attacker has access to an image that the defender has uploaded online, and tries to transform it with a generative image-to-image model. The defender protects the image before upload. The first target model for the prototype is local SD-Turbo image-to-image generation. Fig. 2.1 summarises the actors and data flows.

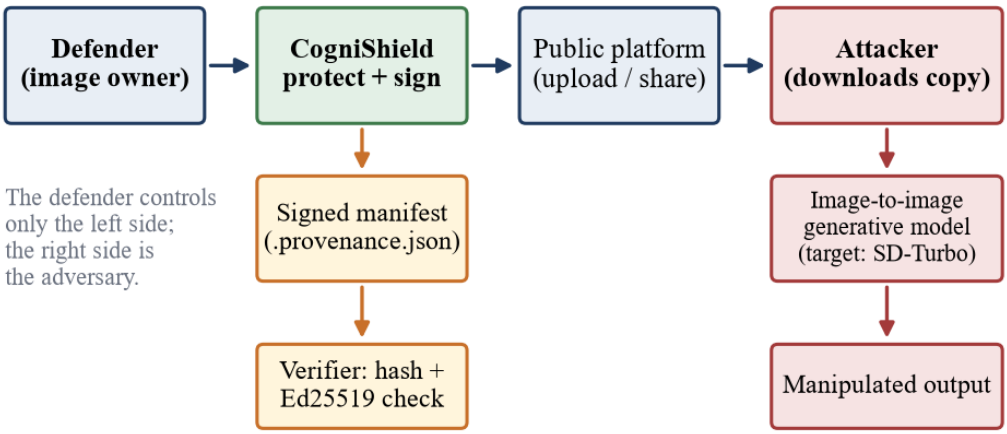


Fig. 2.1 Threat Model and Actors in the CogniShield Scenario

2.3.1 Adversary capabilities

The assumed adversary can download any image that the defender publishes, can run a pretrained image-to-image diffusion model locally with a prompt and a seed of their choosing, and can apply ordinary post-processing such as JPEG compression, which is what most platforms do automatically. The adversary does not know the defender's private signing key and cannot forge a signature over a modified file. The adversary is not assumed to know the defender's seed or budget, although the system does not rely on keeping the perturbation secret.

2.3.2 Defender capabilities and assumptions

The defender controls the original file and the computer on which protection is performed, keeps the private key in the project key folder and distributes the protected PNG and optionally the manifest. The defender is assumed to publish the lossless protected file wherever the platform allows it; when a platform recompresses the image, the JPEG-aware design aims to retain part of the perturbation, and the evaluation quantifies how much.

2.3.3 Security properties claimed and not claimed

The provenance module claims only that the bytes of a protected image match the signed hash and that the signature was produced by the holder of the local private key. It does not claim that a scene is objectively real, that the signer is who they say they are, or that a file was never copied. The perturbation module claims that the protected

image differs from the original by a bounded, measured amount and that the effect on SD-Turbo can be inspected. It does not claim immunity against every model, editor or transformation. These distinctions are repeated in the discussion in Chapter 7 because overstated security claims are a common failure of protection tools.

2.3.4 Attack scenarios and expected outcomes

The threat model can be made concrete by listing the scenarios that the design considers. Table 2.2 lists each scenario with the effect that the protection and the provenance mechanism are expected to have and the place in this report where the expectation is tested.

Table 2.2 Attack Scenarios and Expected Outcomes

Scenario	Expected effect of the perturbation	Expected provenance outcome	Section
Direct image-to-image edit of the lossless protected PNG	Perturbation fully present; its effect on the model is measured	Signature valid for the untouched file	7.9, 7.10
Platform recompresses the upload as JPEG at quality 95 or 75	About half of the perturbation survives along its own direction	The recompressed copy does not verify; the original PNG still does	7.5, 7.9
Platform recompresses at quality 50	Most of the perturbation is removed	The recompressed copy does not verify	7.5
Rescaling, blurring or median filtering before editing	Most of the perturbation is removed	The altered copy does not verify	7.6
Screenshot or print-and-scan	Not evaluated; expected to remove most of it	The copy does not verify	Not evaluated
Attacker alters the file and claims it is the original	Not relevant	Digest mismatch is detected	7.9
Attacker signs an altered file with their own key	Not relevant	Verification with the owner's public key fails	4.1.11

2.4 Goals and Objectives

The objectives listed in Section 1.1 are turned into measurable goals in Table 2.3. Each goal has an acceptance criterion that is checked in the results chapter.

Table 2.3 Project Goals and Acceptance Criteria

ID	Goal	Acceptance criterion
G1	Imperceptible protection	PSNR above 30 dB and SSIM close to 1.0 at the default settings on a varied image set
G2	Bounded change	Maximum absolute pixel change never exceeds the selected budget ε
G3	Compression awareness	Protected image evaluated at JPEG qualities 95, 75 and 50, with the effect of feedback iterations measured
G4	Reproducibility	Same image, seed and settings give a bit-identical protected image
G5	Verifiable provenance	A signature verifies for the untouched file and fails when a single byte is altered
G6	Target-model comparison	Original and protected images are passed through SD-Turbo with identical prompt and seed and shown side by side
G7	Usability	A user with no programming knowledge can protect an image in the browser and download the outputs
G8	Local operation	No image or key leaves the machine; the web server binds to the loopback address only

2.5 System Overview

CogniShield consists of a core library of six small modules, a Flask web application and three command-line programs. Fig. 2.2 shows how the user reaches

the library through either interface and where outputs, keys and the optional model are stored. The library modules and their responsibilities are detailed in Chapter 3, and the algorithms inside them are described in Chapter 4.

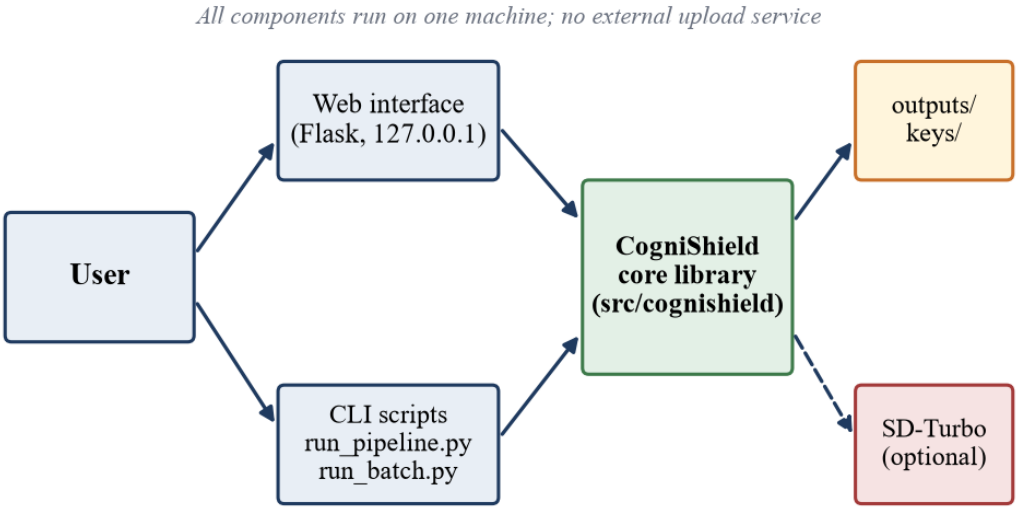


Fig. 2.2 System Context of CogniShield

The user interacts with the system in four steps. First, the user chooses an image and, if desired, adjusts the protection controls. Second, the system generates the protected image and its inspection images. Third, the system measures quality and JPEG robustness and signs the protected output. Fourth, the user downloads the protected PNG and the provenance manifest and may inspect the optional SD-Turbo comparison. All results of a run are written to a run-specific folder so that every experiment can be revisited.

2.6 Literature Survey

The idea of protecting images with small perturbations draws on three research threads: adversarial examples, protection tools aimed at specific misuse scenarios, and content provenance. Table 2.4 summarises the most relevant works and the position of CogniShield among them.

Table 2.4 Related Work and Position of CogniShield

Work	Idea	Difference from CogniShield
Szegedy et al. (2014); Goodfellow et al. (2015)	Small bounded perturbations fool classifiers; fast gradient sign method	Attack on classifiers; CogniShield applies a bounded perturbation as a protective measure
Madry et al. (2018)	Projected gradient descent as a strong first-order attack	Requires gradients of a model; CogniShield is gradient-free in this version
Dziugaite et al. (2016); Shin & Song (2017)	JPEG weakens adversarial perturbations; differentiable JPEG helps survive it	CogniShield uses the real JPEG encoder in a residual feedback loop
Salman et al. (2023) PhotoGuard	Immunizes images against diffusion-based editing by attacking the model	Requires access to the target model and substantial compute; CogniShield is lightweight and local

Work	Idea	Difference from CogniShield
Shan et al. (2020) Fawkes; Cherepanova et al. (2021) LowKey	Cloak photographs against face-recognition training or matching	Different misuse scenario: recognition rather than generative editing
Shan et al. (2023) Glaze; Liang et al. (2023) AdvDM; Van Le et al. (2023) Anti-DreamBooth	Protect artists and users from style mimicry and personalised fine-tuning	Target training or personalisation; CogniShield targets image-to-image editing and adds signing
C2PA specification	Signed provenance manifests linked to certificate trust	Needs PKI; CogniShield uses a local key pair and a sidecar manifest as a prototype

The common weakness of the protection systems in the table is that they are designed around a particular model and usually need a GPU, and none of them couples the protection step with an integrity proof that the owner can hand to others. CogniShield accepts a weaker, model-agnostic perturbation in exchange for running on an ordinary computer in a fraction of a second per image, and it couples that perturbation to verifiable provenance and a built-in measurement of cost and robustness.

Existing research shows that adversarial perturbations can disturb AI-based image manipulation models, but robustness remains a major challenge. Many perturbations are weakened by compression, resizing, cropping, denoising or social media processing, and detection-based approaches are reactive and may fail when new generative models appear. To cover the field more broadly than Table 2.4, seventeen recent papers on adversarial image protection, deepfake detection, provenance and efficient vision architectures were reviewed during the first project review. Table 2.5 lists the merits and demerits of each paper as they relate to CogniShield.

Table 2.5 Literature Survey of Seventeen Recent Papers on Adversarial Image Protection

S.No.	Title	Merits	Demerits
1	DiffusionGuard: A Robust Defense Against Malicious Diffusion-based Image Editing (Choi et al., 2025)	Targets unauthorized diffusion-based editing with adversarial noise generated at an early diffusion stage; improves robustness to masks, noise removal and transfer attacks while reducing computational cost.	Focused on diffusion-based editing threats; protection still depends on assumptions about the attack setting and target models.
2	Verifying Provenance of Digital Media: Security Analysis of C2PA and its Implementation (Golaszewski et al., 2026)	Provides a security analysis of C2PA provenance and identifies concrete implementation and specification weaknesses relevant to authenticity verification.	Analyzes provenance security rather than adversarial image protection; provenance alone cannot prevent AI manipulation.
3	Robust Universal Adversarial Perturbations (Xu & Singh, 2024)	Develops universal adversarial perturbations designed to remain effective under real-world transformations, including scaling, rotation and JPEG compression.	Evaluated primarily on classification datasets rather than diffusion-based image editing; transfer to CogniShield's target generators requires validation.
4	MobileNetV4: Universal Models for the Mobile Ecosystem (Qin et al., 2024)	Introduces efficient mobile architectures designed for strong accuracy-efficiency trade-offs across CPUs, DSPs, GPUs and edge accelerators.	It is a general mobile vision architecture paper and does not directly address adversarial perturbation generation or image protection.
5	Distraction is All You Need: Memory-Efficient Image Immunization against Diffusion-Based Image Editing (Lo et al., 2024)	Presents a memory-efficient semantic attack for image immunization against diffusion-based editing and inpainting, reducing the memory burden of earlier approaches.	Protection is targeted at diffusion editing/inpainting and requires further evaluation under broader transformations and adaptive attacks.
6	Adversarial Robust Safeguard for Evading Deep Facial Manipulation (Guan et al., 2024)	Introduces a learning-based safeguard that generates protection noise in a single forward pass and explicitly trains for robustness to common perturbations.	Targets facial manipulation rather than general image editing; robustness and performance depend on the manipulation methods represented during training.
7	Raising the Cost of Malicious AI-Powered Image Editing (Salman et al., 2023)	Establishes the proactive image-immunization approach using encoder-level and diffusion-level adversarial attacks against malicious diffusion-based editing.	Protection can be computationally expensive and later work demonstrates weaknesses under transformations such as JPEG compression.
8	Can Protective Perturbation Safeguard Personal Data from Being Exploited by Stable Diffusion? (Zhao et al., 2024)	Evaluates protective perturbations under a practical Stable Diffusion threat model and studies purification attacks that can remove protective noise.	Shows that purification can undermine existing protection methods, highlighting the need for stronger robustness mechanisms.
9	JPEG Compressed Images Can Bypass Protections Against AI Editing (Sandoval-Segura et al., 2023)	Provides direct evidence that JPEG compression can neutralize imperceptible perturbation-based protections, motivating compression-aware defense design.	Primarily diagnoses the weakness rather than providing a complete end-to-end robust protection method.
10	Watermark-embedded Adversarial Examples for Copyright Protection against Diffusion Models (Zhu et al., 2024)	Combines adversarial examples with personal watermarks to discourage unauthorized imitation by diffusion models and reports transferability across unknown models.	Focuses strongly on copyright protection and visible watermark outcomes rather than purely imperceptible protection for personal photographs.

S.No.	Title	Merits	Demerits
11	Glaze: Protecting Artists from Style Mimicry by Text-to-Image Models (Shan et al., 2023)	Uses barely perceptible style-cloaking perturbations to disrupt unauthorized style mimicry by text-to-image diffusion models and evaluates robustness with large artist studies.	Primarily protects artistic style against training/fine-tuning based mimicry rather than direct pixel-level image editing or inpainting.
12	DCT-Shield: A Robust Frequency Domain Defense against Malicious Image Editing (Bala et al., 2025)	Moves adversarial protection into the DCT/frequency domain and leverages the JPEG pipeline to improve visual quality and robustness against purification.	The method is specifically designed around frequency-domain/JPEG-aware protection and requires further validation across broader deployment conditions.
13	How Generalizable are Deepfake Image Detectors? An Empirical Study (Li et al., 2023)	Systematically evaluates deepfake-image detector generalization and shows that zero-shot performance can be poor across unseen datasets and synthesis methods.	It studies detection rather than proactive prevention, so it does not directly protect an image before manipulation.
14	UCF: Uncovering Common Features for Generalizable Deepfake Detection (Yan et al., 2023)	Improves deepfake-detection generalization by disentangling common forgery features from method-specific and irrelevant features.	Remains a reactive detection approach and does not prevent the generation or editing of manipulated images.
15	EfficientViT: Memory Efficient Vision Transformer with Cascaded Group Attention (Liu et al., 2023)	Introduces memory-efficient vision-transformer components and demonstrates strong speed/accuracy trade-offs suitable for efficient vision processing.	It is not an adversarial-protection method and requires adaptation before being used inside CogniShield's protection pipeline.
16	BlurGuard: A Simple Approach for Robustifying Image Protection Against AI-Powered Editing (Kim et al., 2025)	Improves protection robustness against noise-reversal techniques by applying adaptive per-region Gaussian blur to the protective noise spectrum.	Adds another robustness-processing stage and is primarily evaluated as an enhancement to existing protection methods.
17	Privacy Protection in Personalized Diffusion Models via Targeted Cross-Attention Adversarial Attack (Xu et al., 2024)	Introduces cross-attention-targeted adversarial protection for personalized diffusion models and aims to protect identity/content with lower perturbation levels.	Focused on personalized diffusion/fine-tuning scenarios; broader transfer to general image-editing pipelines needs additional evaluation.

The survey of the seventeen papers leads to the following findings.

- ✔ Recent protection research (DiffusionGuard, PhotoGuard, Distraction is All You Need, Adversarial Robust Safeguard, DCT-Shield, BlurGuard, Glaze and related adversarial-protection methods) demonstrates that carefully designed perturbations can proactively disrupt unauthorized generative-model use, which supports the core hypothesis behind CogniShield.
- ✔ A recurring weakness in the reviewed protection literature is sensitivity to image transformations and purification, with JPEG compression and related transformations shown to reduce protection effectiveness (Sandoval-Segura et al., 2023; Xu & Singh, 2024; Guan et al., 2024; Zhao et al., 2024; Kim et al., 2025; Bala et al., 2025).
- ✔ Detection-based approaches remain fundamentally reactive: Li et al. (2023) report weak zero-shot generalization of deepfake-image detectors, while Yan et al. (2023) address generalization through common forgery features, which reinforces the motivation for proactive protection.
- ✔ The reviewed protection methods show a continuing trade-off between protection strength, computational cost and image quality. Memory-efficient and learned approaches such as Lo et al. (2024), Guan et al. (2024), Qin et al. (2024) and Liu et al. (2023) are therefore relevant to the edge-deployment objective of the project.
- ✔ Provenance and authenticity mechanisms can complement adversarial protection, but the C2PA security analysis by Golaszewski et al. (2026) identifies

weaknesses that mean provenance should be treated as a complementary verification layer rather than a standalone guarantee.

- ✔ Lightweight-architecture research such as MobileNetV4 (Qin et al., 2024) and EfficientViT (Liu et al., 2023) demonstrates practical routes towards lower memory and computation, which supports the feasibility of evaluating CogniShield on resource-constrained hardware.
- ✔ The reviewed literature spans diffusion-image immunization, facial-manipulation protection, style protection, compression-aware defenses, deepfake detection, provenance and efficient vision architectures. However, these components are generally evaluated separately and not as one on-device pipeline that combines proactive protection, robustness testing, visual-quality control and provenance.

2.7 Research Gap and Contribution

Three gaps motivate the project. First, most immunization methods are evaluated by the authors against specific models, and a student or researcher cannot easily reproduce the pipeline end to end on a laptop. Second, quality metrics, compression robustness and target-model effects are rarely reported together in one tool, so a user cannot judge the trade-off for their own image. Third, protection and provenance are normally treated as separate topics.

The review of the seventeen papers in Table 2.5 leads to the same conclusion. Adversarial image-protection methods, robustness mechanisms, efficient vision architectures and provenance mechanisms are largely studied as separate components. The reviewed papers do not establish a single, lightweight pipeline that combines (a) specified target generative-manipulation models, (b) measurable visual-quality constraints, (c) compression- and purification-aware robustness evaluation, (d) practical edge-oriented computation and (e) complementary cryptographic provenance. CogniShield is designed to investigate this combined gap.

The contributions of this project are therefore: a complete, local, reproducible pipeline that links perturbation, measurement and signing; a gradient-free, JPEG-aware feedback mechanism that uses the real encoder; a reporting design that exposes PSNR, SSIM, MSE, absolute deltas and compression results for every run; an optional target-model test isolated in its own process so that failures cannot break the main

workflow; and an honest empirical evaluation, presented in Chapter 7, that identifies where the prototype works and where it does not.

2.8 Deliverables

The following deliverables result from the project:

- ✔ The CogniShield source code, comprising the core library, the Flask web application, the command-line programs and the PowerShell helper scripts for setup and launch.
- ✔ A responsive web interface with dark mode by default and a light-mode toggle, including a *How it works* page that explains the processing flow.
- ✔ Output artefacts for each run: the protected PNG, a noise preview, a difference image magnified twelve times, JPEG variants, the signed provenance manifest and, for command-line runs, a machine-readable report.
- ✔ Batch experiment support that writes a summary table in CSV form for any folder of images.
- ✔ The optional SD-Turbo comparison module and documentation for downloading the model locally.
- ✔ This project report, together with the experimental data and figures generated from the project code.

2.9 Use Cases and Requirement Traceability

Five use cases describe how people use the system. They are listed in Table 2.6. The first two are performed by an ordinary user of the web interface, the third by anyone who holds the protected file, the manifest and the public key, and the last two by a researcher who evaluates the tool.

Table 2.6 Use Cases

ID	Use case	Actor	Main flow	Outcome
UC1	Protect a single image	User	Choose image, set controls, press Protect, download protected PNG and manifest	Signed protected image
UC2	Inspect quality and robustness	User	Read metrics, compression table, difference image, noise preview and JPEG previews	Informed choice of settings
UC3	Verify provenance	User or third party	Recompute the digest of the file, compare with the manifest and verify the signature with the public key	Valid or invalid result with a reason
UC4	Batch evaluation	Researcher	Place images in the input folder and run the batch program	summary.csv and per-image folders
UC5	Target-model comparison	Researcher	Enable the SD-Turbo test, enter a prompt, protect an image	Side-by-side model outputs

In the web interface, verification is performed automatically when a protected image is created and its result is displayed in the provenance panel; verifying a file at a later time uses the library function *verify_image* on the image, the manifest and the public key.

Table 2.7 shows how the goals of Section 2.4 are covered by the functional requirements of Chapter 3 and by the experiments of Chapter 7, so that every goal can be traced to an implementation and to evidence.

Table 2.7 Traceability From Goals to Requirements and Evidence

Goal	Short description	Functional requirements	Evidence
G1	Imperceptible protection	FR2, FR5, FR6	Sections 7.2, 7.3
G2	Bounded change	FR2, FR5	Section 7.2
G3	Compression awareness	FR3, FR7, FR8	Sections 7.4, 7.5, 7.6
G4	Reproducibility	FR4	Section 7.8
G5	Verifiable provenance	FR9, FR10	Section 7.9
G6	Target-model comparison	FR11	Section 7.10
G7	Usability	FR1, FR12, FR14	Chapter 6
G8	Local operation	FR13 and the non-functional requirements	Sections 3.5, 4.3.4

CHAPTER 3
TECHNICAL SPECIFICATION

This chapter specifies the environment in which CogniShield runs, the software on which it depends, the functional and non-functional requirements it satisfies, the architecture of its components and the interfaces through which a user or another program can drive it. The specification is derived from the project's source code, its requirement files and its README, so that the statements below can be checked against the repository.

3.1 Hardware Requirements

The protection pipeline itself uses only CPU operations on arrays and the JPEG encoder from the Pillow library, so it runs on any reasonably modern computer. The optional SD-Turbo comparison is the only component that benefits from a graphics processor. Table 3.1 lists the requirements for the two modes of operation.

Table 3.1 Hardware Requirements

Item	Core protection (required)	SD-Turbo target test (optional)
Processor	Any 64-bit CPU; no vector extensions needed	Same; the GPU performs the model work
Graphics	Not required	NVIDIA GPU with a working CUDA driver
Memory	4 GB RAM is sufficient for images of several megapixels	Enough GPU memory to load SD-Turbo in half precision
Disk space	About 2 GB for Python packages; about 5 MB per protected run	Several additional GB for the local model weights
Display	Any modern browser at 1280 × 720 or larger	Same
Network	Needed only for the first package installation	Needed once to download the model, then offline

Storage use per run is small. In the 18 web-interface runs that were recorded during development, a run folder holding the original, the protected image, the noise preview, the difference image, three JPEG variants and the manifest occupied about 4.2 MB for an 801 × 775 image and about 10.5 MB for a 941 × 1672 image; a run that also stored two SD-Turbo outputs occupied about 4.9 MB.

3.2 Software Requirements

Table 3.2 summarises the software environment. The project targets Windows 10 and Windows 11 because the helper scripts are written in PowerShell, but the Python code is portable.

Table 3.2 Software Requirements

Component	Specification	Remarks
Operating system	Windows 10 or Windows 11	PowerShell scripts for setup and launch
Language runtime	Python 3.12	Installed with winget or from python.org
Environment	Virtual environment in .venv	Created by the setup script
Web browser	Any current browser	Interface served at http://127.0.0.1:5000
GPU stack	PyTorch with CUDA 12.8 wheels	Only for the SD-Turbo test
Model	SD-Turbo in models/sd-turbo/	Downloaded with the huggingface_hub package

3.3 Libraries and Dependencies

The dependencies are split into three requirement files so that a user who only needs protection and provenance does not have to install the large machine-learning stack. Table 3.3 lists each library and the role it plays.

Table 3.3 Python Libraries and Their Roles

Library	Group	Role in CogniShield
NumPy	Core	Float arrays, random noise generation, vectorised pixel arithmetic and metrics
Pillow	Core	Image loading, PNG and JPEG encoding, Gaussian blur, resizing
scikit-image	Core	Structural similarity index (SSIM) computation
cryptography	Core	Ed25519 key generation, signing and verification; PEM serialisation
Flask	Core	Local web server, routing, upload handling and JSON responses
OpenCV and pandas	Core	Listed for image analysis and tabulation of batch results
torch and torchvision	AI	Tensor runtime and CUDA support for SD-Turbo
diffusers and transformers	AI	Image-to-image pipeline and text encoder for SD-Turbo
accelerate, safetensors	AI	Efficient model loading and weight format
huggingface_hub and hf_xet	AI	Local download of the SD-Turbo model snapshot
matplotlib	AI	Plotting support for analysis

Python’s standard library supplies hashlib for SHA-256, json for manifests, base64 for signature encoding, multiprocessing for the isolated SD-Turbo worker, and dataclasses for the configuration objects. Beyond the standard library, the numerical arrays are handled with NumPy (Harris et al., 2020), the structural similarity measure is computed with scikit-image (van der Walt et al., 2014), the web layer is built with the Flask framework (Grinberg, 2018), and the optional model test runs on PyTorch (Paszke et al., 2019) through the Diffusers library (von Platen et al., 2022).

3.4 Functional Requirements

Table 3.4 lists the functional requirements. Each requirement is traceable to a module in the source code, which is shown in the last column.

Table 3.4 Functional Requirements

ID	Requirement	Module
FR1	Accept PNG, JPG, JPEG and WEBP images up to 200 MB and convert them to RGB	app.py, image_io
FR2	Allow the user to set the perturbation budget from 1 to 16 out of 255	app.py, protector

ID	Requirement	Module
FR3	Allow the user to set the number of compression-feedback iterations from 0 to 8	app.py, protector
FR4	Allow the user to set an integer seed from 0 to 999999 for reproducible noise	app.py, protector
FR5	Generate a protected image, a noise preview and a ×12 difference image	protector, metrics
FR6	Compute PSNR, SSIM, MSE, mean absolute delta and maximum absolute delta	metrics
FR7	Evaluate the protected image after JPEG compression at selected qualities from 95, 85, 75, 60, 50	compression
FR8	Save the protected image as PNG and JPEG variants for inspection	compression, app.py
FR9	Sign the protected PNG with Ed25519 over its SHA-256 digest and write a JSON manifest	provenance
FR10	Verify the signature and report whether the image changed after signing	provenance
FR11	Optionally run original and protected images through SD-Turbo with identical prompt and seed	sd_turbo, app.py
FR12	Provide downloads of the protected image and the manifest	app.py
FR13	Process a single image or a folder from the command line and write a report or CSV summary	run_pipeline.py, run_batch.py
FR14	Present a dark-mode default interface with a light-mode toggle and a How it works page	templates, static

3.5 Non-Functional Requirements

Table 3.5 Non-Functional Requirements

Attribute	Requirement	How it is met
Privacy	No image or key is sent to an external service	Server bound to 127.0.0.1; processing and storage are local
Reproducibility	Identical inputs and settings give identical outputs	Seeded generator; deterministic array operations
Performance	A typical image is protected in well under a second on a CPU	Vectorised NumPy operations and a few JPEG round trips
Robustness of the service	An optional-model failure must not lose the protection result	SD-Turbo runs in a separate process with a 300 s timeout
Input safety	Reject unreadable or unsupported files with a clear message	Extension check, image decoding in a try block, secure_filename
Parameter safety	Out-of-range parameters must not crash the server	Integer parsing with clamping to documented ranges
Maintainability	Small modules with a single responsibility	Six library modules, each below about 125 lines
Usability	Single-page workflow with drag and drop and clear status messages	Responsive HTML, CSS and JavaScript interface
Portability	Core features run without a GPU	Optional AI dependencies kept in a separate requirements file

3.6 System Architecture

3.6.1 Layered view

CogniShield follows a layered structure, shown in Fig. 3.1. The presentation layer consists of the HTML templates, the stylesheet, the JavaScript and the command line. The application layer holds the Flask application and the three command-line programs, which contain no image-processing logic of their own. The domain layer is the *cognishield* package, which implements protection, compression testing, metrics, provenance and the optional model runner. Below it are the scientific and cryptographic libraries, the optional machine-learning runtime and the operating platform.

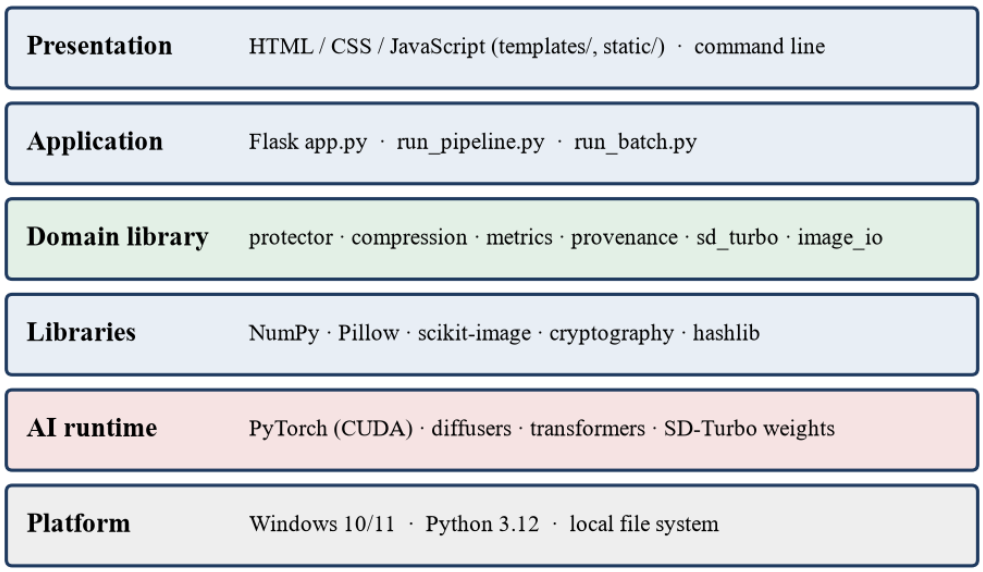


Fig. 3.1 Layered Software Stack of CogniShield

3.6.2 Module structure

The domain layer contains six modules and a configuration file, shown in Fig. 3.2 and described in Table 3.6. The modules are deliberately small and communicate through plain function calls and immutable dataclasses, which keeps the data flow easy to inspect and test.

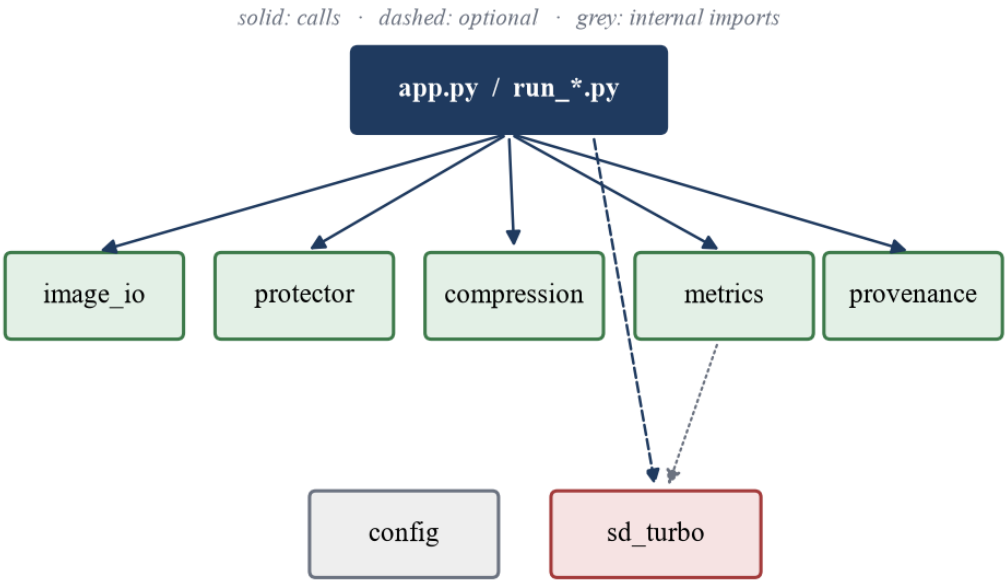


Fig. 3.2 Module Structure of the CogniShield Library

Table 3.6 Responsibilities of the Library Modules

Module	Lines	Responsibility
config.py	14	Project root, model folder, output folder, key folder and data folder paths; creation of required folders
image_io.py	30	Loading as RGB, conversion between PIL images and float arrays in [0, 1], resizing to multiples of eight for the model
protector.py	84	The protection configuration and the algorithm: high-frequency noise, edge mask, bounded perturbation and JPEG feedback
compression.py	36	JPEG round trip, saving JPEG variants and evaluating the protected image after compression
metrics.py	60	PSNR, SSIM, MSE, mean and maximum absolute delta, and the amplified difference image
provenance.py	121	SHA-256 hashing, Ed25519 key management, signing, manifest writing and verification
sd_turbo.py	84	Lazy loading of the SD-Turbo image-to-image pipeline and a seeded run method

3.6.3 Request flow in the web application

Fig. 3.3 shows the sequence of calls that the web application performs for one request. The browser posts the image and settings to the protection endpoint. The server validates them, calls the protector, then the metric and compression functions, then the provenance functions, and returns a single JSON document that the page renders into the result panels.

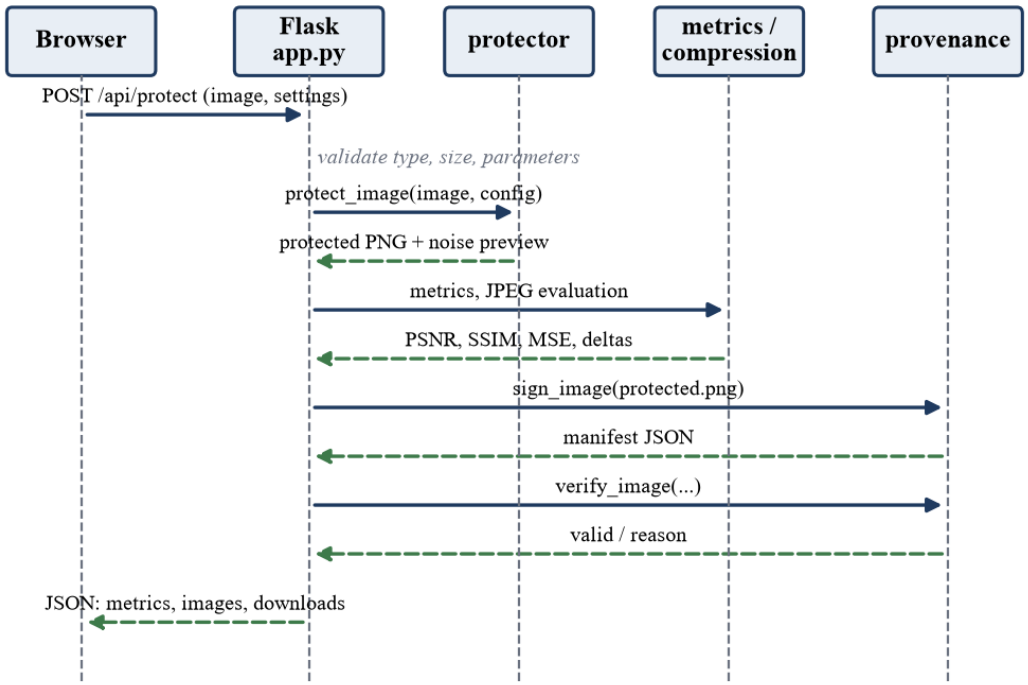


Fig. 3.3 Sequence of Calls for a Protection Request

3.7 Interface Specification

3.7.1 Web endpoints

Table 3.7 lists the HTTP endpoints exposed by the Flask application.

Table 3.7 HTTP Endpoints of the Web Application

Method	Path	Purpose
GET	/	Main workspace page
GET	/how-it-works	Same-tab guide that explains the processing flow
POST	/api/protect	Accepts the image and form fields; returns JSON with images, downloads, metrics, compression table, JPEG images, provenance result and optional model result or error
GET	/results/<run_id>/<file>	Serves a result image from the run folder for display
GET	/download/<run_id>/<file>	Serves the protected PNG or the manifest as an attachment
GET	/evaluation	Evaluation dashboard that shows the newest completed dataset-evaluation run with its headline rates, protocol settings, charts and downloads
GET	/evaluation-files/<run_name>/<file>	Serves the chart, the results.csv file and the report.md file of an evaluation run

The form fields accepted by the protection endpoint are *image*, *epsilon*, *iterations*, *seed*, *qualities* (repeated), *run_model* and *prompt*. Errors are returned as JSON with an explanatory message: HTTP 400 for a missing, unsupported or unreadable file, HTTP 413 when the 200 MB limit is exceeded and HTTP 500 for an unexpected failure during protection.

3.7.2 Control reference

Table 3.8 Protection Controls and Their Effects

Control	What it changes	Default	Practical guidance
Perturbation budget	Maximum allowed pixel change, from 1 to 16 out of 255	8	Higher is stronger but can be more visible
Compression feedback	Number of JPEG-aware refinement steps, 0 to 8	4	Higher may improve JPEG resilience but takes longer and adds energy
Seed	Random starting point of the noise field	42	Keep fixed for repeatable results
JPEG tests	Qualities to evaluate: 95, 85, 75, 60, 50	95, 75, 50	Choose qualities that match the sharing path
Run SD-Turbo target test	Runs both versions through the local model	Off	Enable only after model setup
Target prompt	Prompt used by the optional test	Portrait prompt	Use the same prompt for a fair comparison

3.7.3 Command-line programs

The three command-line programs share the library with the web application. Table 3.9 lists their options.

Table 3.9 Command-Line Programs and Options

Program	Option	Default	Meaning
run_pipeline.py	image (positional)	none	Path of the input image
run_pipeline.py	--epsilon	8/255 ≈ 0.03137	Maximum perturbation on a 0–1 scale
run_pipeline.py	--seed, --iterations	42, 4	Seed and number of feedback iterations
run_pipeline.py	--qualities	95 75 50	JPEG qualities for testing; only values of 50 and above feed back into the perturbation
run_batch.py	--input-dir	data/input	Folder searched recursively for PNG, JPG, JPEG and WEBP files
run_batch.py	--epsilon, --seed, --iterations, --qualities	as above	Same meaning as for a single image
run_evaluation.py	--input-dir	data/input	Folder searched recursively for the images to be evaluated
run_evaluation.py	--run-name	timestamped name	Name of the result folder under outputs/evaluations; an existing folder is never overwritten
run_evaluation.py	--epsilon, --seed, --iterations, --qualities	8/255, 42, 4, 95 75 50	Same meaning as for a single image
run_evaluation.py	--prompt	a realistic photograph, natural lighting	Prompt used for the SD-Turbo comparison of the original and the protected image
run_evaluation.py	--quality-psnr-threshold, --quality-ssim-threshold	30 dB, 0.95	Thresholds of the quality pass
run_evaluation.py	--jpeg-ssim-threshold, --target-ssim-threshold	0.90, 0.80	Threshold of the JPEG pass and maximum output SSIM for target divergence

Program	Option	Default	Meaning
run_evaluation.py	--skip-model, --max-images, --example-count	off, all, 12	Quality and JPEG evaluation without SD-Turbo; limit on the number of images for a quick test; number of example images saved

3.8 Data Specification

3.8.1 Folder layout

The repository separates source code, generated outputs, keys, models and data, as shown in Table 3.10. Generated outputs, keys and models are excluded from version control by the project's ignore file, which prevents private keys from being committed accidentally.

Table 3.10 Folder Layout of the Project

Path	Contents
app.py, run_pipeline.py, run_batch.py, run_evaluation.py	Entry points
src/cognishield/	The six library modules and the configuration
templates/, static/	HTML pages, stylesheet and JavaScript for the web interface
scripts/	PowerShell helpers: setup, run_app, run_cli_example, run_batch
docs/	Report outline and documentation
data/input/, data/samples/	Images to be processed; placeholders are kept in version control
outputs/web/<run id>/	Result files of each web run, named by a 12-character hexadecimal identifier
outputs/<image name>/, outputs/batch/	Result files of command-line and batch runs, and summary.csv
keys/	Ed25519 private and public keys in PEM format
models/sd-turbo/	Local SD-Turbo model snapshot
outputs/evaluations/<run name>/	results.csv, summary.json, report.md, evaluation_charts.png and an examples/ folder of one dataset-evaluation run

3.8.2 Output files of a run

Every run writes the original image, the protected PNG, the noise preview, the difference image magnified twelve times and one JPEG file for each selected quality. The signed manifest is stored next to the protected image with the suffix *.provenance.json*. When the optional model test runs, two further images are written, one for the original and one for the protected input. A dataset-evaluation run does not write these per-image files; it writes one row per image to results.csv together with summary.json, report.md, evaluation_charts.png and a folder of representative four-panel examples under outputs/evaluations.

3.8.3 Provenance manifest

The manifest is a JSON document with the fields in Table 3.11.

Table 3.11 Fields of the Provenance Manifest

Field	Meaning
project	Constant string ”CogniShield”
algorithm	”Ed25519 over SHA-256(image_bytes)”
image_file	File name of the signed image
image_hash_sha256	Hexadecimal SHA-256 digest of the image file bytes
signature_base64	Ed25519 signature of the 32-byte digest, base64 encoded (64 bytes before encoding)
public_key_file	Name of the public key file used for verification
signed_at_utc	ISO 8601 timestamp with time zone
metadata	Source file name, epsilon (fraction and pixels), iterations, seed and JPEG qualities

3.9 Environment Setup

Setup is automated for Windows. The *setup.ps1* script creates a virtual environment in *.venv*, upgrades pip and installs the core, CUDA PyTorch and AI requirement files in that order. Users who only need protection, metrics, JPEG testing and provenance can install the core requirements alone. The server is started with the *run_app.ps1* script or directly with the virtual environment’s Python interpreter running *app.py*, and it is stopped with Ctrl + C in the terminal. The PowerShell execution policy can be relaxed for the current window only, so no permanent system setting is changed. The dataset evaluation of Section 4.1.20 is started with run_evaluation.py after the server has been stopped, so that the GPU is free for the model. Appendix A reproduces the main commands.

3.10 Output Tree of a Run

The listing below shows the files in the run folder of the web run that included the optional model test. The same names are used in every run; the two *sd_turbo* files are only present when the model test was requested, and a command-line run stores its files in a folder named after the input image instead.

```
outputs/web/0684920cc8de/
    original.png                    uploaded image converted to RGB
    protected.png                   protected image (the signed file)
    protected.png.provenance.json   signed manifest
    protected_jpeg_q95.jpg          protected image re-encoded at quality 95
    protected_jpeg_q75.jpg          protected image re-encoded at quality 75
    protected_jpeg_q50.jpg          protected image re-encoded at quality 50
    noise_preview.png               perturbation magnitude as a fraction of
    the budget
    difference_x12.png              absolute difference multiplied by twelve
    sd_turbo_original.png           model output for the original (optional)
    sd_turbo_protected.png          model output for the protected image
    (optional)
```

3.11 Assumptions and Dependencies

The specification rests on a small number of assumptions that should be checked before the system is used or extended.

- ✔ Python 3.12 and the packages listed in the requirement files are available; the code uses type-hint syntax that requires a recent Python version.
- ✔ The user owns the images or is authorised to process them, and publishes the lossless protected file wherever the platform allows it.
- ✔ The loopback address and port 5000 are free on the machine; if another process uses the port, it must be stopped first.
- ✔ The SD-Turbo model, if used, has been downloaded to the model folder, the driver and the CUDA build of PyTorch are compatible, and the GPU has enough free memory.
- ✔ The key folder is writable on the first run so that the key pair can be created, and the private key is kept private.
- ✔ Results are interpreted as measurements of similarity and robustness, and not as proof that an image is authentic or safe.

CHAPTER 4
DESIGN APPROACH AND DETAILS

This chapter explains how CogniShield is designed and how each stage of the pipeline works. It begins with the design approach and the algorithms, continues with the codes and standards that the implementation follows, and ends with the constraints, alternatives and trade-offs that shaped the design. The mathematical definitions given here correspond line by line to the implementation in the *cognishield* package.

4.1 Design Approach / Materials & Methods

4.1.1 Design principles

Five principles guided the design. The first is **locality**: all computation, storage and key handling happen on the user's own computer. The second is **transparency**: every stage produces an artefact or a number that can be inspected, such as the noise preview, the magnified difference image and the metric table. The third is **boundedness**: the maximum change to any pixel is limited by a single, user-visible parameter that is enforced by clipping, not merely hoped for. The fourth is **isolation of risk**: the heavy and failure-prone SD-Turbo test runs in a separate process so that it can never destroy a protection result. The fifth is **modesty of claims**: the system reports measurements and shows model outputs instead of declaring an image immune.

4.1.2 Overall pipeline

The complete processing sequence is shown in Fig. 4.1. An input image is converted to an RGB floating-point array, a seeded high-frequency noise field and an edge mask are created, their product is limited to the perturbation budget, and the result is refined by a JPEG feedback loop. The protected PNG, a noise preview and a magnified difference image are then produced, quality and robustness are measured, and the protected file is hashed, signed and verified.

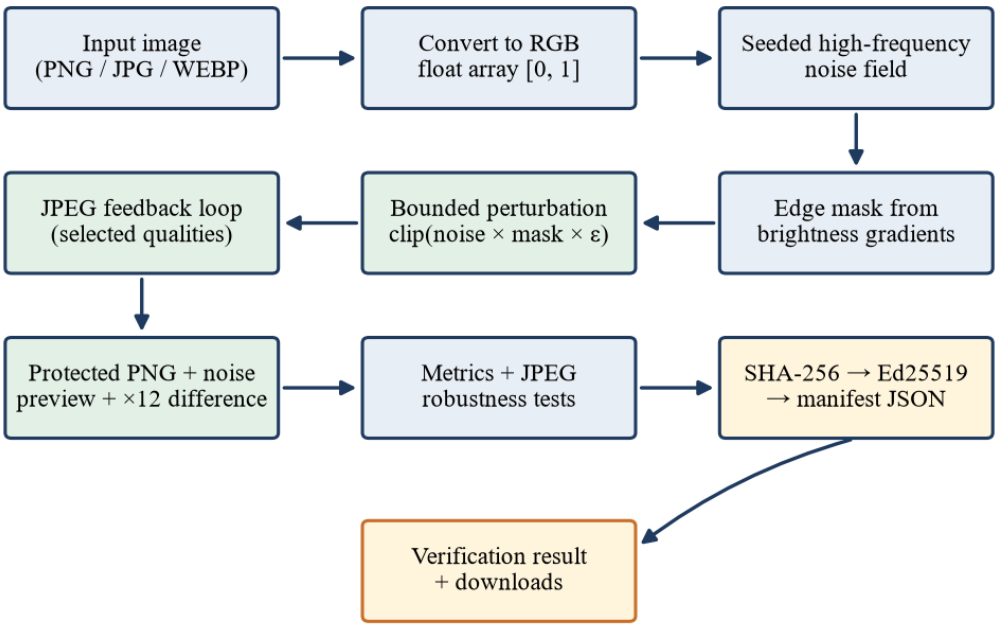


Fig. 4.1 End-to-End Processing Pipeline of CogniShield

4.1.3 Image representation

All computation uses single-precision floating point arrays with values in the range 0 to 1. The function *load_rgb_image* opens a file with Pillow and converts it to three-channel RGB, so grayscale images, palette images and images with an alpha channel all enter the pipeline in the same form. *pil_to_float_array* divides the 8-bit values by 255, and *float_array_to_pil* clips to the valid range, multiplies by 255, rounds and converts back to 8 bits. The rounding to 8 bits at the very end means that the final integer perturbation can differ from the floating-point perturbation by up to half a level; the measured maximum absolute difference in Chapter 7 is exactly the budget in 8-bit levels.

4.1.4 High-frequency noise generation

The protective signal is built from random noise because noise has no semantic content that an editing model could latch onto, and it is made high-frequency because the human eye is least sensitive to fine, rapidly varying patterns. The generator *_high_frequency_noise* draws a standard normal array of the same shape as the image from a NumPy generator initialised with the user's seed. It then forms a smoothed

copy by rescaling the array to 8 bits, applying a Gaussian blur of radius 1.2 pixels with Pillow and rescaling the result to the range minus one to one. Subtracting the smoothed copy from the raw noise removes the low-frequency part, and dividing by the largest absolute value normalises the result to the range minus one to one. The operation can be written as follows.

$$n = \frac{h}{\max |h|}, \qquad h = \xi - G_{\sigma}(\xi) \tag{4.1}$$

In the Eqn. 4.1 the variable ξ denotes the raw seeded Gaussian noise, G_{σ} denotes Gaussian smoothing, h denotes the high-frequency part that remains after the smoothed copy is subtracted, and n denotes the normalised noise whose values lie between minus one and one.

Because the generator is seeded, the same seed always yields the same noise for a given image shape, which makes every run reproducible. Different seeds give different noise patterns, and Section 7.8 measures how much the visual quality varies between seeds.

4.1.5 Edge mask

A uniform noise field would be most visible in smooth regions such as sky or skin, where the eye can detect small irregularities, and least visible near edges and textures, where natural variation already masks it. The function *_edge_mask* therefore estimates local detail from brightness gradients. It converts the image to a single channel by averaging the colour channels, computes the absolute difference between each pixel and its upper neighbour and between each pixel and its left neighbour, and adds the two. The sum is divided by its maximum so that it lies between zero and one. The mask is then compressed into the range 1 − w to 1 with the edge weight w, whose default is 0.65, so that flat regions still receive at least 35 percent of the permitted budget, as written below.

$$g(i,j) = |I(i,j) - I(i-1,j)| + |I(i,j) - I(i,j-1)|, \quad m = (1-w) + w \cdot \frac{g}{\max g} \tag{4.2}$$

In the Eqn. 4.2 the variable g(i, j) denotes the gradient magnitude of the grey-level image I at the pixel in row i and column j, max g denotes the largest gradient

value in the image, w denotes the edge weight, and m denotes the resulting edge mask.

The mask is expanded to three channels and multiplied with the noise. Choosing a floor of 35 percent rather than zero keeps the perturbation present in smooth areas as well, which matters because a model that is sensitive to the protected image may be influenced by the global pattern and not only by edges.

4.1.6 Bounded initial perturbation

The initial perturbation is the product of the noise and the mask scaled by the budget, followed by a clip that enforces the bound, as given below. Since the normalised noise lies between minus one and one and the mask between 0.35 and 1, the unclipped product already lies within the budget, and the clip is a guarantee rather than a correction. The budget ε is specified in the interface as an integer E between 1 and 16 out of 255 intensity levels, so that ε is E/255, as given below. The default value is 8, that is, about 3.1 percent of the full intensity range.

$$\delta_0 = \text{clip}\big(n \odot m \cdot \varepsilon,\ -\varepsilon,\ +\varepsilon\big) \tag{4.3}$$

In the Eqn. 4.3 the variable δ_0 denotes the initial perturbation, n denotes the normalised noise, m denotes the edge mask, $\odot$ denotes the element-wise product, ε denotes the budget, and clip denotes the operation that limits every value to the interval from −ε to +ε.

$$\delta_i \in [-\varepsilon,\ +\varepsilon], \qquad \varepsilon = \frac{E}{255} \tag{4.4}$$

In the Eqn. 4.4 the variable δ_i denotes the perturbation at any pixel, which must lie within the budget, E denotes the integer budget in intensity levels between 1 and 16, and ε denotes the same budget on the scale from 0 to 1.

4.1.7 JPEG-aware feedback loop

The perturbation described so far lives in the high-frequency part of the image, which is the part that JPEG discards first. To make the protected image keep as much of the perturbation as possible after compression, CogniShield uses a feedback loop based on the real JPEG encoder. At iteration k the candidate image is formed by adding the current perturbation to the original and clipping, as given by the first equation below. The candidate is compressed and decoded at every quality in a set Q, and the residual

that compression removed is averaged over the qualities, as given by the second equation below.

$$x_k = \text{clip}(x + \delta_k,\, 0,\, 1) \tag{4.5}$$

In the Eqn. 4.5 the variable x_k denotes the candidate image at iteration k, x denotes the original image, and δ_k denotes the current perturbation.

$$f_k = \frac{1}{|Q|} \sum_{q \in Q} (x_k - J_q(x_k)) \tag{4.6}$$

In the Eqn. 4.6 the variable f_k denotes the average compression residual at iteration k, Q denotes the set of JPEG qualities, |Q| denotes the number of qualities in that set, and J_q denotes a JPEG round trip, that is encoding and decoding, at quality q.

The perturbation is then updated by adding the residual scaled by a compression weight λ and clipping to the budget, as given below. The default weight is 0.16 and the default number of iterations N is 4. Intuitively, the residual is the part of the candidate that compression threw away, so adding a fraction of it to the perturbation pre-compensates for the loss: the components that JPEG would have removed are strengthened for the next round, while the clip prevents the perturbation from ever exceeding the budget.

$$\delta_{k+1} = \text{clip}(\delta_k + \lambda f_k,\, -\varepsilon,\, +\varepsilon) \tag{4.7}$$

In the Eqn. 4.7 the variable δ_{k+1} denotes the updated perturbation, δ_k denotes the current perturbation, λ denotes the compression weight, f_k denotes the residual of Eqn. 4.6, and ε denotes the budget.

After N iterations the protected image is obtained by adding the final perturbation to the original and clipping to the valid range, as given below. Setting N to zero disables the loop and yields the initial perturbation only. The complete procedure is shown as a flowchart in Fig. 4.2.

$$x_p = \text{clip}(x + \delta_N,\, 0,\, 1) \tag{4.8}$$

In the Eqn. 4.8 the variable x_p denotes the protected image, x denotes the original image, and δ_N denotes the final perturbation after N iterations.

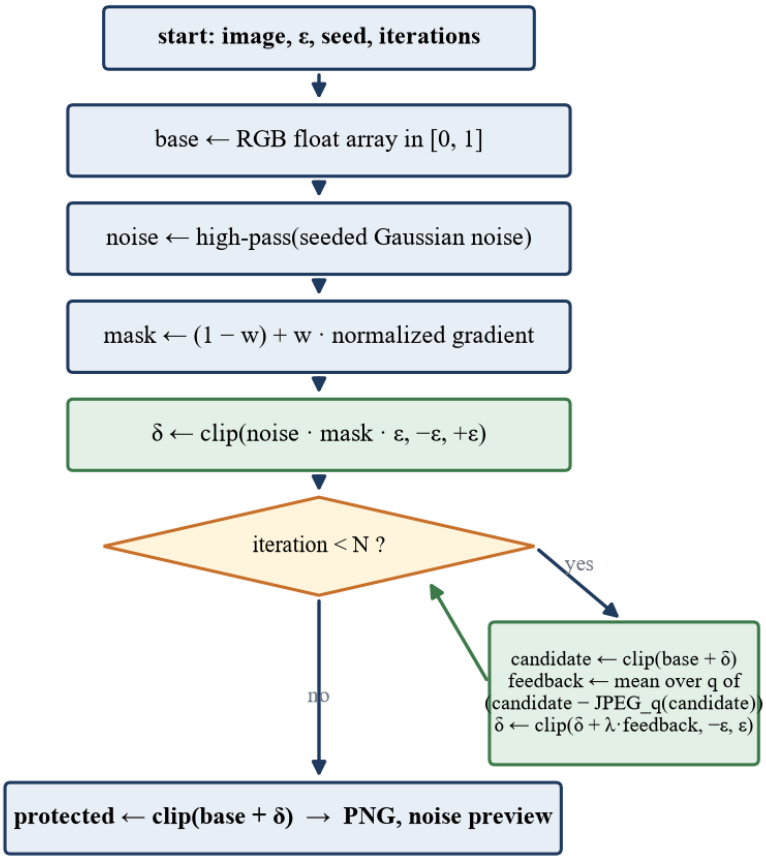


Fig. 4.2 Flowchart of the Protection Algorithm with JPEG Feedback

The feedback loop is gradient-free: it does not differentiate through JPEG, it only calls the encoder. This keeps the code short and independent of any machine-learning framework. Its cost is a few JPEG encodings per iteration, and Chapter 7 shows that the whole algorithm completes in a fraction of a second for typical images. A property that is easy to overlook is that each iteration adds a scaled residual to the perturbation, so the loop tends to raise the average magnitude of the perturbation even though the clip bounds its maximum. This is why the measured PSNR falls as iterations increase; the trade-off is quantified in Section 7.4.

The two terms of the final perturbation differ greatly in size. Because the noise is normalised by its single largest absolute value, typical initial pixel changes are a fraction of a level, about 0.49 levels on average over the 14 test images, whereas the final perturbation averages 1.62 levels. Most of the difference is contributed by the feedback term, which reflects the image's own JPEG residual and therefore depends on the image content as well as on the seed; Section 4.1.19 illustrates this on a small patch and Section

7.4.1 measures it over the whole test set.

4.1.8 Inspection outputs

Besides the protected image, the protector returns a *noise preview* in which each pixel shows the magnitude of the perturbation as a fraction of the budget, scaled to the range 0 to 255. The metrics module separately produces a *difference image*, which is the absolute difference between the protected and original images multiplied by a gain of 12 and clipped. Both images are meant for human inspection: at the default budget the true difference is nearly invisible, and the amplified view shows where the perturbation was placed and that it follows the structure of the image.

4.1.9 Quality metrics

Four numerical measures and one structural measure are reported for every comparison of two images x and y with H rows, W columns and three channels and pixel values in [0, 1]. The mean squared error is the average squared pixel difference, given by the first equation below. The peak signal-to-noise ratio expresses it on a logarithmic decibel scale relative to the peak value, which is 1 for normalised pixels, as given by the second equation below; if the images are identical the MSE is zero and the ratio is infinite. The structural similarity index of Wang et al. (2004) compares local means μ, variances σ^2 and covariance σ_{xy} with small stabilising constants C_1 and C_2, as given by the third equation below. The implementation calls the scikit-image function with the colour channel axis specified and a data range of 1.0, and falls back to reporting no SSIM if the library is unavailable. The *mean absolute delta* and the *maximum absolute delta* are the mean and maximum of the absolute pixel differences over all pixels and channels.

$$\mathrm{MSE} = \frac{1}{3HW} \sum_{i=1}^{3HW} (x_i - y_i)^2 \tag{4.9}$$

In the Eqn. 4.9 the variable MSE denotes the mean squared error, x_i and y_i denote the i-th values of the two images, and H and W denote the number of rows and columns, so that 3HW is the number of values in an image with three channels.

$$\mathrm{PSNR} = 10 \log_{10}\!\left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}}\right) = 20 \log_{10}\!\left(\frac{1}{\sqrt{\mathrm{MSE}}}\right) \tag{4.10}$$

In the Eqn. 4.10 the variable PSNR denotes the peak signal-to-noise ratio in

decibels, MAX denotes the peak pixel value, which is 1 for normalised pixels, and MSE denotes the mean squared error of Eqn. 4.9.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{4.11}$$

In the Eqn. 4.11 the variable SSIM(x, y) denotes the structural similarity index of the images x and y, μ_x and μ_y denote their local means, σ_x^2 and σ_y^2 denote their local variances, σ_{xy} denotes their covariance, and C_1 and C_2 denote the small stabilising constants.

Table 4.1 Quality Measures Reported by CogniShield

Measure	Desired direction	Interpretation
PSNR	Higher	Pixel-level similarity; values above 30 dB often indicate low visible distortion
SSIM	Closer to 1.0	Structural similarity between the original and protected images
MSE	Lower	Mean squared pixel error on the 0–1 scale
Mean absolute delta	Lower	Average absolute per-pixel change
Max absolute delta	Lower	Largest absolute change at any pixel and channel; equals the budget in the default pipeline

These measures describe similarity, not whether an image is objectively real, safe or immune. In particular, a high PSNR does not prove that a human would find the image unchanged, and a low PSNR does not prove that a model would be affected.

4.1.10 Compression robustness testing

The compression module performs a JPEG round trip by encoding the image to an in-memory buffer at the requested quality with Pillow's optimising encoder and decoding it again. For each selected quality the evaluation compresses the *protected* image and compares the result with the *original*, so the reported PSNR and SSIM include both the cost of the perturbation and the damage caused by compression itself. The same function writes the JPEG files that the interface shows for visual inspection. The web interface offers qualities 95, 85, 75, 60 and 50 and selects 95, 75 and 50 by default, and the command-line programs default to 95, 75 and 50. In the single-image command-line program only qualities of 50 or more are passed to the feedback loop, which keeps the loop to moderate compression levels, while all requested qualities are still used for evaluation.

The JPEG transform that drives these results maps each 8 × 8 block $f_{x,y}$ to frequency coefficients $F_{u,v}$ through the discrete cosine transform given below. The coefficients are divided by entries of a quantisation table scaled by the quality factor and

rounded, so high-frequency coefficients of small magnitude are driven to zero. Since the protective perturbation has a magnitude of a few levels, many of its coefficients fall below the quantisation step at low quality, which explains the retention results of Chapter 7.

$$F_{u,v} = \frac{1}{4} C_u C_v \sum_{x=0}^{7} \sum_{y=0}^{7} f_{x,y} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16} \tag{4.12}$$

In the Eqn. 4.12 the variable $F_{u,v}$ denotes the frequency coefficient for the horizontal frequency u and the vertical frequency v, $f_{x,y}$ denotes the pixel value at position x, y of the 8 × 8 block, and C_u and C_v denote normalisation factors that equal $1/\sqrt{2}$ for index zero and 1 otherwise.

4.1.11 Cryptographic provenance

The provenance module implements the sign-and-verify protocol in Fig. 4.3. To sign a protected PNG it computes the SHA-256 digest of the file bytes by reading the file in one-megabyte chunks, loads the Ed25519 private key from the key folder (generating a new key pair on first use and saving both keys in PEM format), signs the 32-byte digest and writes a JSON manifest next to the image. The relationship is summarised below.

$$h = \text{SHA-256}(B), \qquad \sigma = \text{Sign}_{sk}(h), \qquad \text{ok} = \text{Verify}_{pk}(\sigma, h) \tag{4.13}$$

In the Eqn. 4.13 the variable B denotes the bytes of the file, h denotes their SHA-256 digest, σ denotes the signature, sk and pk denote the private and the public key, and ok denotes the result of the verification.

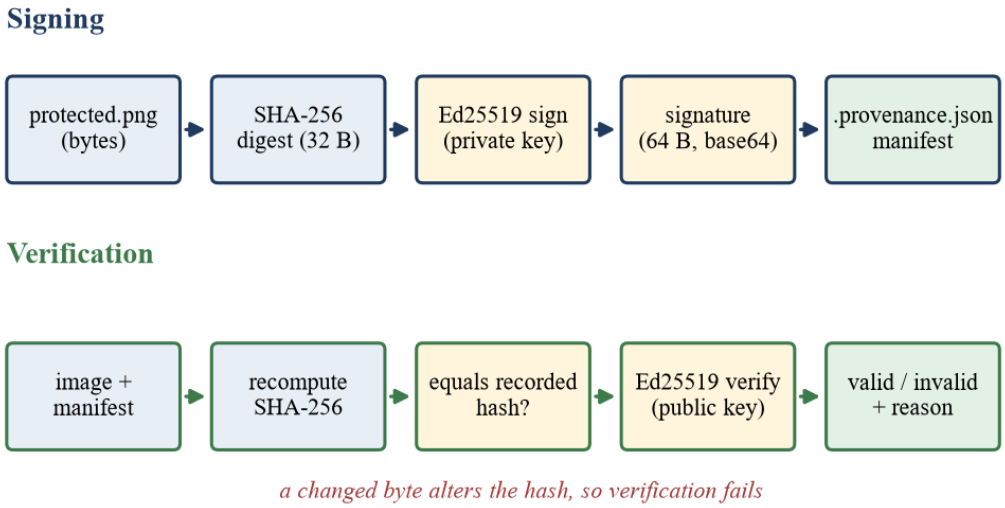


Fig. 4.3 Signing and Verification Protocol of the Provenance Module

Verification recomputes the digest of the file, compares it with the digest recorded in the manifest and then verifies the signature with the public key. The function returns a structured result that states whether verification succeeded, a human-readable reason and the current digest. The possible reasons are that the image file or manifest or public key does not exist, that the image hash changed after signing, that the signature does not match the public key, or that the signature and hash are valid. Hash comparison comes first so that the most common failure, a modified file, is reported with an informative message. Ed25519 signing is deterministic, so the same digest and key always produce the same signature, and the 64-byte signature is stored in base64.

Two design decisions are worth noting. First, the signature is computed over the digest of the *protected PNG bytes*, not over pixel values, so metadata changes are also detected, which is desirable for integrity but means that any re-encoding invalidates the signature; the manifest therefore applies to the exact file that was saved. Second, the private key is stored unencrypted in a local folder, which is acceptable for a prototype running on the user's own machine but would require protection in a deployed system. The project's ignore file excludes the key folder from version control.

4.1.12 Optional SD-Turbo target test

To observe the effect of protection on a real generative model, the SD-Turbo module loads an image-to-image pipeline from the local model folder with local files

only, uses half precision on a CUDA device and single precision otherwise, and enables attention slicing when available. The method that runs the model resizes the image so that its longer side is at most 512 pixels and both sides are multiples of eight, because the model's autoencoder downsamples by a factor of eight. It calls the pipeline with the user's prompt, a guidance scale of zero, a strength of 0.55 and a seeded generator. The number of denoising steps is chosen as the largest of two, the requested steps and the reciprocal of the strength rounded up, so that at least one denoising step remains after the strength is applied; with the default strength this is two steps.

The web application runs this test in a separate operating-system process, shown in Fig. 4.4. The parent process starts a worker, waits for at most 300 seconds, terminates the worker if it has not finished and reads the outcome from a queue. If the model directory is missing, the worker times out or exits abnormally, the page receives an explanatory message and the protection result, which was already computed and signed, is still returned. This isolation was adopted because errors in CUDA or in model loading can terminate a Python process without raising an exception that the web server could catch.

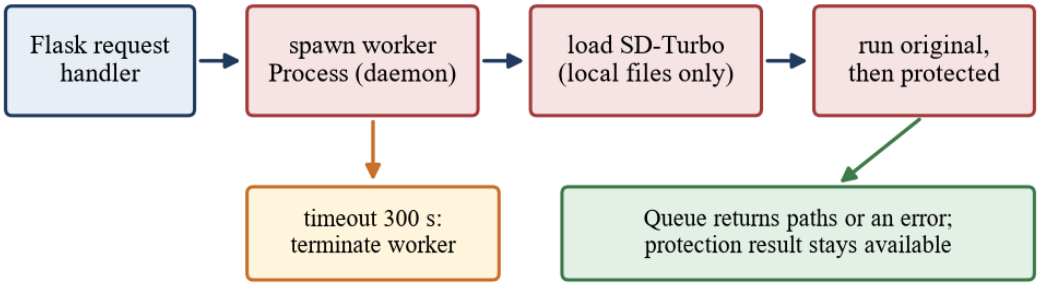


Fig. 4.4 Process Isolation of the Optional SD-Turbo Test

The original and protected images are passed through the same prompt, seed and settings so that any difference between the two outputs is attributable to the protection. Both outputs are saved in the run folder and displayed side by side in the interface. The test does not alter the protected PNG or its signature.

4.1.13 Web application

The web application is a single Flask module. On a protection request it validates

the file extension against the allowed set, clamps the numeric parameters to their ranges, reads the uploaded stream with Pillow and rejects unreadable files with a clear message. It then creates a run folder named by a random 12-character hexadecimal identifier, runs the protection, saves the images, signs and verifies the output, writes the JPEG variants and, if requested, runs the model test. Everything is returned in one JSON response. The file names inside the run folder are fixed, which, together with the use of a sanitised version of the uploaded name in the manifest, keeps user-provided strings out of file system paths. The decision flow of one request is shown in Fig. 4.5, and two further routes serve the evaluation dashboard and the files of an evaluation run (Section 7.12.5).

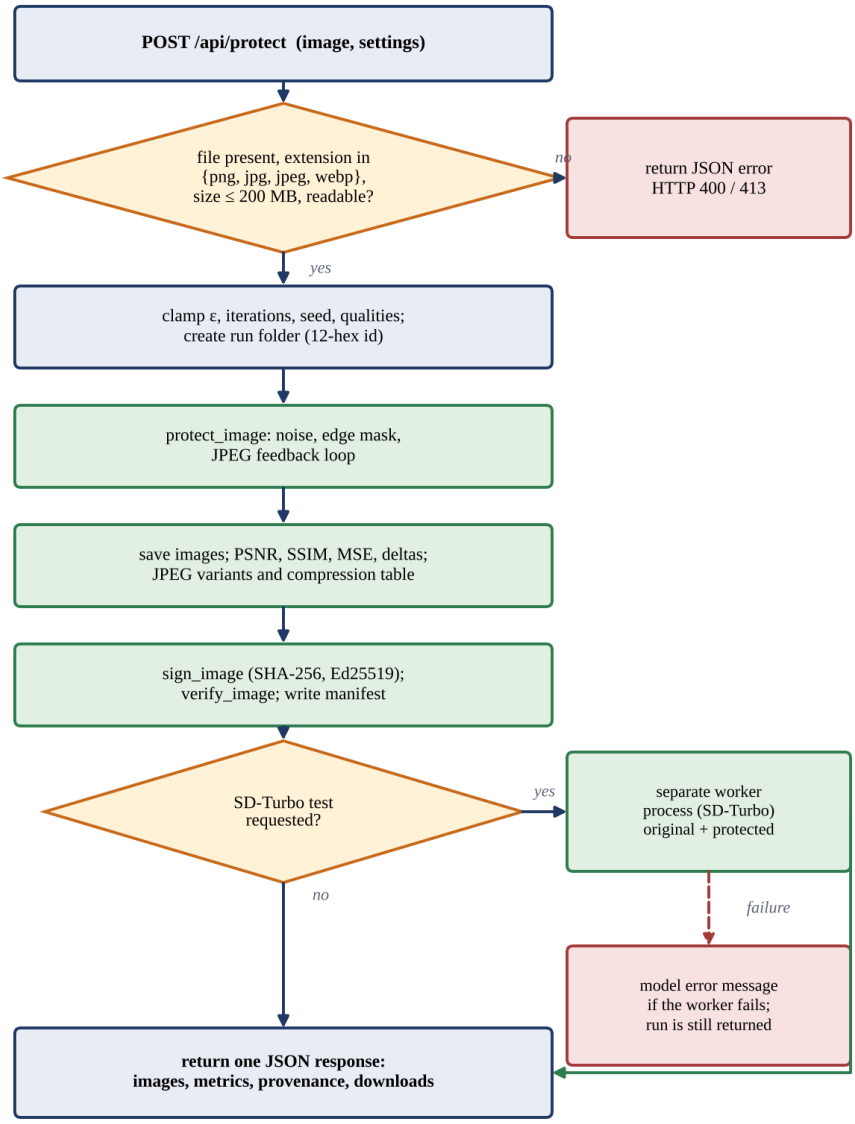


Fig. 4.5 Flowchart of a Protection Request in the Web Application

The browser interface is a single page with a settings sidebar, an upload panel

and a results panel, shown in Chapter 6. It uses drag and drop, range sliders for the budget and the number of iterations, a seed control with step buttons, checkboxes for the JPEG qualities and an optional prompt field that appears when the model test is enabled. The interface starts in dark mode, offers a light-mode toggle whose choice is remembered in the browser, and links to a *How it works* page. The JavaScript calls the protection endpoint with the Fetch API, disables the button while processing, shows a status chip, and renders metrics, the compression table, inspection images, the provenance status and the optional model comparison from the returned JSON. The interface also provides subtle audio feedback in browsers that support it, animated panels that are reduced when the operating system asks for reduced motion, and contained previews so that complete SD-Turbo images are visible instead of cropped. Next to the downloads, a Check in ChatGPT button downloads the protected PNG, opens chatgpt.com in a new tab and copies a prepared comparison prompt to the clipboard; because a local page cannot upload a file into a user's account, the user must attach the downloaded PNG manually, and this optional qualitative check is not part of the provenance guarantee. A separate Evaluation page shows the newest completed dataset-evaluation run (Section 7.12).

4.1.14 Command-line and batch tools

The single-image program *run_pipeline.py* loads an image, builds a protection configuration from its arguments, writes the protected image, noise preview, difference image and JPEG variants to a folder named after the input file, computes metrics and compression results, signs and verifies the output and writes a machine-readable *report.json* that is also printed. The batch program *run_batch.py* applies the same configuration to every image found recursively in a folder and writes one row per image to *outputs/batch/summary.csv* with the image size, the five metrics and the PSNR and SSIM at each tested JPEG quality. The batch program was designed to satisfy the review suggestion of running 10 to 20 permitted images, and the experiments of Chapter 7 use the same library calls. The evaluation program run_evaluation.py, which applies fixed criteria to a whole dataset, is described in Section 4.1.20.

4.1.15 Data flow

Fig. 4.6 presents the system as a level-one data flow diagram with four processes and three data stores. The upload and validation process receives the image and settings

from the user; the protection process produces the protected image; the measurement process computes metrics and compression results; and the signing process produces and checks the manifest. Results are written to the run folder, keys are read from and written to the key store, and the optional model reads from the model folder.

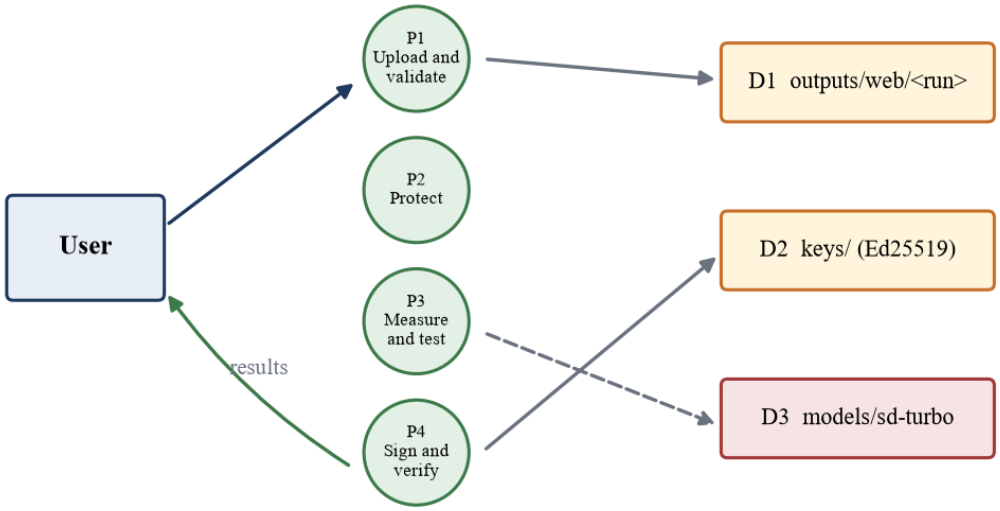


Fig. 4.6 Level-One Data Flow Diagram of CogniShield

4.1.16 Computational complexity

For an image with n = H × W pixels, noise generation, masking and clipping are linear in n. The edge mask and the Gaussian blur are also linear. Each feedback iteration encodes and decodes the candidate once per quality in Q, and the JPEG codec is linear in n, so the protection cost is of the order of n(1 + N|Q|) elementary operations, ignoring constants. Metric computation adds a linear term, with SSIM being the most expensive because of its sliding window. The measured running times in Section 7.11 confirm approximately linear growth of the protection time with the number of pixels. Memory use is a small multiple of the image size, since a handful of float arrays of the same shape are alive at once.

4.1.17 Handling of input edge cases

Because every image is converted to three-channel RGB on loading, a range of input types are handled by one code path. Grayscale images are replicated into three identical channels, palette images are expanded, and images with an alpha channel lose that channel in the conversion, so transparency is not preserved in the protected output.

Images are processed at their native resolution, with no resizing, so the protected PNG has exactly the dimensions of the input; only the optional model test resizes, and only its own copy of the image. The loader does not apply the EXIF orientation flag, so a photograph that is stored rotated and displayed rotated by a viewer is protected as stored; since the perturbation is applied pixel by pixel and the protected PNG carries no orientation flag, the protected image appears as the stored pixels, which may differ in orientation from what the user saw in a photo viewer. A user who needs the visible orientation preserved should rotate the image before protecting it. Animated WEBP files are reduced to their first frame by the decoder. These behaviours follow directly from the loading code and are noted here because they affect how the tool should be used.

4.1.18 Numerical precision and determinism

All arithmetic uses 32-bit floating point numbers. The noise is generated by the NumPy default generator, whose output for a given seed is specified by the library version, and all subsequent operations are element-wise array arithmetic, clipping and calls to the JPEG codec of the installed Pillow library. As a consequence, the same image, seed and settings produce the same protected image on the same software stack, which the experiments of Chapter 7 confirm. Across different versions of NumPy, Pillow or the underlying libjpeg implementation the outputs may differ in the last intensity level, because rounding and codec details are not guaranteed to be identical; the provenance manifest therefore records the settings but not the library versions, and the signature protects the saved bytes, not the process that created them. The conversion of the final floating-point array to 8-bit integers rounds to the nearest level, which means that the realised perturbation is an integer number of levels at every pixel, as seen in Table 4.2.

4.1.19 Worked example on a small patch

To make the algorithm concrete, Table 4.2 traces one 4 × 4 patch of the colour portrait of Chapter 7 through the pipeline at the default settings. The patch was chosen automatically as the mid-intensity region with the most varied gradient, and the red channel is shown. The columns list the grey level of each pixel, the normalised gradient, the resulting mask value, the normalised noise sample, the initial perturbation in 8-bit levels and the final perturbation measured from the protected image.

Table 4.2 Worked Example on a 4 × 4 Patch of the Colour Portrait (Red Channel, Default Settings)

Pixel (row, col)	Grey (0–255)	Norm. gradient	Mask m	Noise n	Initial δ (levels)	Final δ (levels)
(468, 230)	73.3	0.052	0.384	+0.006	+0.02	+5.00
(468, 231)	90.0	0.059	0.389	+0.398	+1.24	+8.00
(468, 232)	88.7	0.053	0.384	+0.472	+1.45	+8.00
(468, 233)	83.3	0.083	0.404	+0.080	+0.26	+8.00
(469, 230)	68.7	0.045	0.379	-0.022	-0.07	+8.00
(469, 231)	69.7	0.070	0.396	-0.582	-1.84	+8.00
(469, 232)	85.0	0.063	0.391	-0.176	-0.55	+7.00
(469, 233)	152.7	0.453	0.644	-0.119	-0.61	-8.00
(470, 230)	52.0	0.061	0.389	-0.128	-0.40	+8.00
(470, 231)	61.7	0.058	0.388	-0.073	-0.23	-2.00
(470, 232)	165.0	0.606	0.744	+0.207	+1.23	-8.00
(470, 233)	194.7	0.237	0.504	+0.105	+0.42	-8.00
(471, 230)	46.0	0.048	0.381	-0.406	-1.24	-4.00
(471, 231)	145.7	0.607	0.744	-0.032	-0.19	+3.00
(471, 232)	182.3	0.178	0.466	+0.010	+0.04	-8.00
(471, 233)	88.3	0.662	0.780	-0.330	-2.06	-8.00

Three observations follow. First, the mask behaves as designed: pixels in the smooth part of the patch have a gradient near zero and a mask value close to the floor of 0.35, while pixels next to the edge have mask values above 0.7. Second, the initial perturbation is small: because the noise is normalised by its single largest absolute value, typical noise samples are a fraction of one, and the initial changes in the table are mostly between minus two and plus two levels, far below the budget of eight. Third, the final perturbation is much larger than the initial one and in this patch reaches the budget at many pixels, with signs that often differ from the initial ones. The reason is the feedback term: the residual between the candidate image and its JPEG-compressed version is, in a detailed region of a photograph, several levels large, and four iterations with a weight of 0.16 add a substantial multiple of it to the perturbation before the clip limits it. The perturbation is therefore not simply seeded noise; it is a blend of seeded noise and a pattern derived from the image’s own compression error. Section 7.4.1 measures this blend over all test images.

4.1.20 Dataset evaluation workflow

The batch program of Section 4.1.14 reports the quality of every image but does not judge whether a result is acceptable. For a larger study the project therefore includes the evaluation runner run_evaluation.py, which applies fixed, predefined criteria to every image of a folder and writes a reproducible record of the run. The workflow is shown in Fig. 4.7.

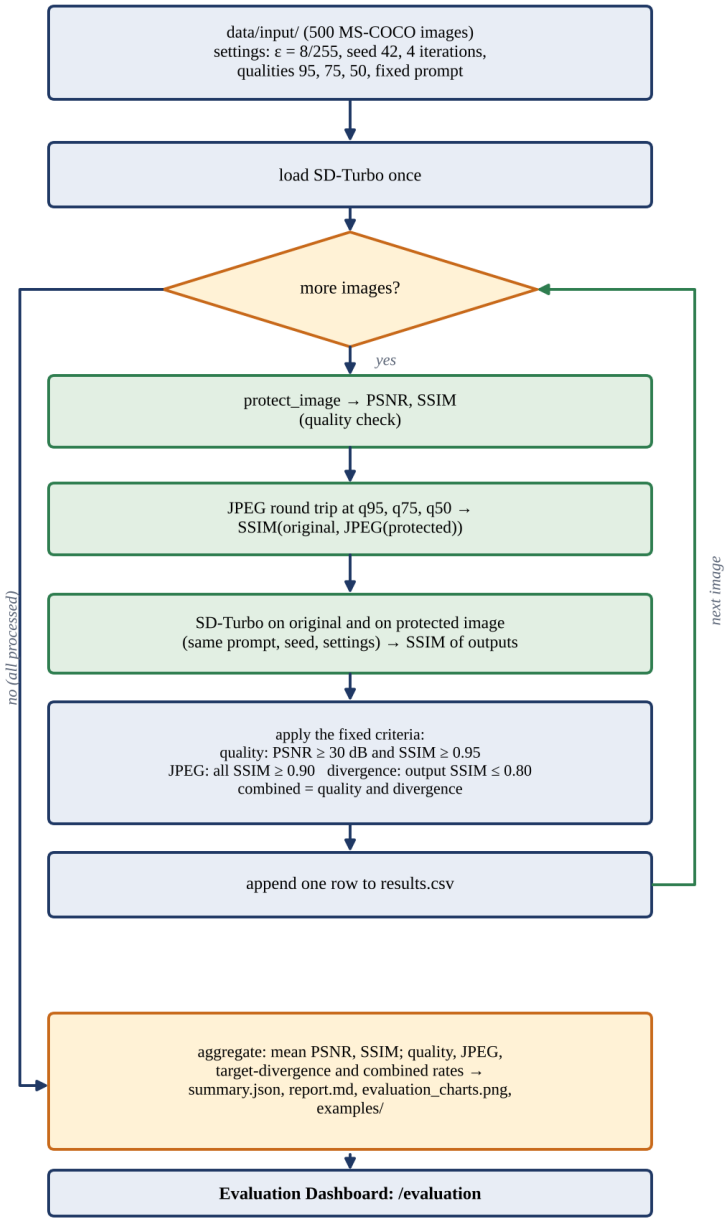


Fig. 4.7 Flowchart of the Dataset Evaluation Workflow of run_evaluation.py

The runner loads the SD-Turbo pipeline once and then processes the images in sorted order. For each image it creates the protected image with the selected configuration and computes PSNR and SSIM against the original; the image passes the quality check if PSNR is at least 30 dB and SSIM is at least 0.95. It then evaluates JPEG round trips at the selected qualities and passes the image if the SSIM between the original and every compressed protected image is at least 0.90. If the model test is enabled, the original and the protected image are passed through SD-Turbo with the same prompt and seed, and the image counts as a target divergence if the SSIM between the two outputs is at most 0.80. An image is a combined success if it passes

the quality check and also shows target divergence. An error in one image is recorded in its row and does not stop the run.

After the last image the runner computes the mean PSNR and SSIM and four rates: the quality pass rate, the JPEG pass rate, the target divergence rate and the combined success rate. Each rate is the percentage of images that satisfy the criterion; for the last two, the denominator is the number of images that have a model result. The runner writes results.csv with every per-image value and pass flag, summary.json with the configuration, criteria, averages and rates, report.md with a short summary and interpretation, evaluation_charts.png, and a set of representative four-panel example images that show the original, the protected image and the two SD-Turbo outputs.

Three rules keep the evaluation honest. All thresholds are written to summary.json, and a threshold should not be changed after the results have been seen unless the change is recorded as a new experiment. A new run name is required for every run, because the runner refuses to overwrite an existing result folder. Finally, the rates are protocol-specific: CogniShield is not a classifier, so they are not classification accuracies and they do not measure protection against every model.

4.2 Codes and Standards

The implementation relies on published standards for file formats, hashing, signatures and data interchange, listed in Table 4.3. Using standard formats makes the outputs of CogniShield readable by other tools: the protected PNG can be opened by any image viewer, the manifest is plain JSON, and the keys can be inspected with common cryptographic utilities.

Table 4.3 Standards Followed by the Implementation

Area	Standard	Use in CogniShield
Lossless image format	PNG, ISO/IEC 15948	Protected output and run images
Lossy compression	JPEG, ITU-T T.81 and ISO/IEC 10918-1	Robustness testing and JPEG variants
Hash function	SHA-256, FIPS 180-4	Digest of the protected file
Digital signature	Ed25519 as specified in RFC 8032	Signing the digest
Key serialisation	PEM text encoding (RFC 7468); PKCS #8 private keys (RFC 5958); SubjectPublicKeyInfo public keys (RFC 5280)	Key files in the key folder
Data interchange	JSON, RFC 8259	Provenance manifest, API responses, report.json
Timestamps	ISO 8601 with UTC offset	signed_at_utc field
Web transport	HTTP semantics, RFC 9110	Local web application
Coding style	PEP 8 and type hints	Python source code
Academic conduct	VIT code of academic and research ethics; APA referencing style	Use of images, citations and this report

Two points of engineering practice complement these standards. Only images

that the user owns or is authorised to process should be used, which the README states explicitly; the test images in Chapter 7 are either the team's own project images or public sample images that ship with the scikit-image library. And no claim in the user interface or documentation goes beyond what the cryptographic check proves.

4.3 Constraints, Alternatives and Tradeoffs

4.3.1 Constraints

The design operates under the constraints listed in Table 4.4. Some are technical, some come from the scale of an academic project and some are deliberate limits on the claims that can be made.

Table 4.4 Design Constraints

Constraint	Effect on the design
Perceptual budget	The per-pixel bound must stay small, so that the protected image is visually indistinguishable at normal viewing
Compression	Recompression discards high-frequency content, so the perturbation must be shaped to survive part of it
Lossless signing	The signature covers file bytes, so the protected image must be saved once as PNG and not re-encoded
No gradient access	The first version treats the target as a black box and does not backpropagate through the model
Hardware	Only the optional model test needs a GPU; the rest must run on a CPU
Time	A single academic term limits the number of target models and experiments
Ethics and legality	Only authorised images are used; protection is described as a mitigation and not a guarantee

4.3.2 Alternatives considered

Table 4.5 summarises the main alternatives that were considered and the reasons for the choices made.

Table 4.5 Design Alternatives and Reasons for the Choices Made

Decision	Alternative	Chosen	Reason
Perturbation method	Gradient-based projected gradient descent against the target model	Seeded, edge-masked noise with feedback	Model-agnostic, no GPU, short code; weaker guarantee
Compression handling	Differentiable JPEG approximation	Real JPEG encoder in a residual loop	No framework dependency and exact codec behaviour
Integrity proof	Invisible watermark	Detached signature over file bytes	Cryptographically strong and simple; does not survive re-encoding
Signature scheme	RSA or ECDSA	Ed25519	Small keys and signatures, deterministic, fast, simple API
Hash function	SHA-1 or MD5	SHA-256	Collision resistance suitable for integrity
Interface	Streamlit app (earlier prototype)	Flask with HTML, CSS and JavaScript	Full control of layout, theming, process isolation and a conventional API
Output format	JPEG	PNG	Lossless, so signature bytes stay stable
Model execution	Run the model inside the web process	Separate worker process with timeout	Isolation from CUDA and loading failures
Trust model	Certificate authority and public-key infrastructure	Local key pair	Appropriate for a prototype; no external services
Reporting results on a dataset	A single classification accuracy	Pass rates with predefined thresholds (quality, JPEG, target divergence, combined)	CogniShield is not a classifier, so accuracy has no defined meaning; each rate has an explicit, recorded criterion

The earlier proposal for the project described a Streamlit demonstration application, and an early run of that prototype with a budget of 5 out of 255 and three

iterations is still present in the output folder of the repository. The interface was later rebuilt in Flask, mainly to isolate the model test in its own process and to obtain finer control of the interface.

4.3.3 Trade-offs

The main trade-offs are summarised below, and Chapter 7 quantifies each of them.

- ✔ **Budget versus invisibility.** A larger budget gives the perturbation more influence but lowers PSNR and SSIM, and increases the chance that a viewer notices a pattern.
- ✔ **Feedback iterations versus energy.** More iterations strengthen the perturbation components that JPEG would remove, but the average perturbation grows, so image quality decreases, and beyond a few iterations the benefit saturates.
- ✔ **Edge weight versus uniformity.** A high edge weight places the budget where it is least visible but leaves flat regions nearly unprotected, whereas a low edge weight spreads the budget evenly and is more visible in smooth regions. The default of 0.65 favours edges while keeping a floor of 35 percent.
- ✔ **Locality versus power.** Running on the user's machine preserves privacy and removes dependence on a server, but limits the computation that can be spent on each image compared with a cloud optimiser.
- ✔ **Integrity versus flexibility.** Signing exact file bytes gives a strong, simple guarantee but means that any legitimate edit, including recompression by a platform, invalidates the signature of the edited copy. The original protected file remains verifiable.

4.3.4 Security considerations

Several implementation details reduce risk. The server binds only to the loopback address, so other machines on the network cannot reach it. Uploaded file names are sanitised with the werkzeug *secure_filename* function before they are stored in a manifest, and result files are stored under fixed names inside randomly named run folders. The upload size is limited to 200 MB. The private key never appears in any output or in the manifest, which stores only the name of the public key file.

Remaining risks are acknowledged: the key is unencrypted on disk, the development server is not hardened for public exposure, and the result and download routes accept the run identifier from the URL without checking that it names an existing run, which is harmless for a single-user local tool but unsuitable for public exposure. These are acceptable for a single-user local tool and are listed among the recommendations in Chapter 8.

CHAPTER 5
SCHEDULE, TASKS AND MILESTONES

The project was carried out over the Project I period from 06 July 2026 to 03 November 2026, a span of about seventeen weeks. This chapter breaks the work into tasks, places the tasks on a timeline, defines the milestones that were used to check progress with the project guide and records the main risks that were identified together with the measures taken against them. The schedule presented here is the plan that structures the work; its phases overlap, as is usual in iterative software projects, because the interface, the evaluation and the report were refined while the core algorithm was still being tuned.

5.1 Work Breakdown

The work was divided into nine tasks, shown in Table 5.1. Each task has a concrete deliverable that can be inspected in the repository or in the run outputs, which made it possible to decide objectively whether a task was finished.

Table 5.1 Work Breakdown Structure

ID	Task	Deliverable	Approx. period
T1	Problem study and literature review	Notes on generative editing, adversarial examples, immunization tools and provenance	Weeks 1–3
T2	Requirement analysis and threat model	Scope, threat model, goals and the report outline in the docs folder	Weeks 3–5
T3	Core algorithm: noise and edge mask	protector module with seeded high-frequency noise, edge mask and bounded perturbation	Weeks 4–8
T4	JPEG feedback and metrics	compression and metrics modules; feedback loop; difference and noise previews	Weeks 6–10
T5	Provenance	provenance module: SHA-256, Ed25519 keys, manifest, verification	Weeks 8–11
T6	Web interface	Flask application, templates, stylesheet, JavaScript, How it works page	Weeks 10–13
T7	Command-line and batch tools	run_pipeline.py, run_batch.py and PowerShell scripts	Weeks 11–14
T8	SD-Turbo target test	sd_turbo module, isolated worker process, model download instructions	Weeks 12–15
T9	Experiments, evaluation and report	Experiments of Chapter 7, figures, tables and this report	Weeks 13–17

5.2 Timeline

Fig. 5.1 shows the tasks on a calendar scale. The first five weeks were devoted to understanding the problem and fixing the scope, which paid off later because the library interface did not change substantially after the core modules were written. The algorithmic work in tasks T3 to T5 occupied the middle of the term, the interface and tools in T6 to T8 followed as soon as the library produced stable outputs, and the evaluation began as soon as batch processing was available.



Fig. 5.1 Project Timeline from 06 July to 03 November 2026

5.3 Milestones

Milestones were chosen so that each represents a state of the system that can be demonstrated. Table 5.2 lists them with the evidence that they were reached.

Table 5.2 Project Milestones and Evidence of Completion

ID	Milestone	Evidence	Target
M1	Scope and threat model agreed	Report outline with problem statement, objectives, threat model and metrics	Week 5
M2	First protected image	Protected PNG with bounded perturbation and noise preview from the protector	Week 8
M3	Metrics and compression tests	PSNR, SSIM, MSE, deltas and JPEG results for any image	Week 10
M4	Signed provenance	Manifest written and verified; tampering detected	Week 11
M5	Working web application	Upload, protect, inspect and download in the browser	Week 13
M6	Batch experiments	summary.csv produced for a folder of images	Week 14
M7	Target-model comparison	SD-Turbo outputs for original and protected images	Week 15
M8	Final report and demonstration	This report and the demonstration described in Chapter 6	Week 17

The output folder of the repository holds evidence for the later milestones: eighteen web-interface runs recorded on 8 October 2026, all with a verified provenance manifest, and two of them with SD-Turbo outputs.

5.4 Risks and Mitigation

Seven risks were identified at the start and monitored during the project. They are listed with the mitigation that was applied in Table 5.3.

Table 5.3 Risk Register

ID	Risk	Impact	Mitigation	Status
R1	Protection too visible	High	Bounded budget, edge-aware mask, PSNR and SSIM reported for every run	Mitigated
R2	Perturbation destroyed by JPEG	High	JPEG feedback loop and robustness table at several qualities	Partly mitigated
R3	SD-Turbo needs a GPU and a large download	Medium	Model test is optional and isolated; rest of system runs without it	Mitigated
R4	Model failure crashes the web server	High	Separate worker process with timeout and error message	Mitigated
R5	Over-claiming protection	High	Wording in the interface and report limited to what is measured	Mitigated

ID	Risk	Impact	Mitigation	Status
R6	Private key exposure	Medium	Key folder excluded from version control; local-only server	Accepted for prototype
R7	Scope growth (more models, video)	Medium	Scope fixed to images and one target model; others listed as future work	Mitigated

Risk R2 is only partly mitigated because compression at low quality removes most of the perturbation, as Chapter 7 shows. Risk R6 is accepted because encrypting the key would complicate the demonstration, and it is noted among the recommendations of Chapter 8.

5.5 Quality Assurance and Review Practice

Quality assurance was planned around repeatable runs and regular review of outputs rather than a heavy process. A change to the algorithm can be judged by running the pipeline on the same test image with a fixed seed and comparing the metric table, the difference image and the JPEG results with the previous run. Because the pipeline is deterministic for a given seed, differences between runs are attributable to the change and not to chance, and the web application stores every run in its own folder, so earlier results remain available for comparison. Reviews with the project guide were planned around the milestones in Table 5.2, with the working system demonstrated at each review. The same discipline was applied to the report: every number in Chapter 7 is generated by a script from the library code and stored in a data file, so the report can be regenerated if the algorithm changes.

CHAPTER 6
PROJECT DEMONSTRATION

This chapter demonstrates the working system. It follows the path that a user takes through the web interface, shows the outputs of real runs that were recorded in the output folder of the project during development, and then shows the command-line workflow and the behaviour of the system in error situations. All figures in this chapter are produced by the project code from the images named in the text; except Fig. 6.1, which is a screenshot of the running web application.

6.1 Demonstration Setup

The demonstration environment is a Windows computer with Python 3.12 and the project's virtual environment. The web server is started with the launch script or with the virtual environment's interpreter running *app.py*, after which the interface is opened in a browser at the address http://127.0.0.1:5000. For the optional model test an NVIDIA GPU, the CUDA build of PyTorch and the SD-Turbo snapshot in *models/sd-turbo/* are required. All runs shown below used the default controls unless stated otherwise: a perturbation budget of 8, four feedback iterations, seed 42 and JPEG tests at qualities 95, 75 and 50.

6.2 Walkthrough of the Web Interface

Fig. 6.1 shows the layout of the workspace. The sidebar on the left holds the protection settings. The central area contains an upload panel with a drag-and-drop zone and the *Protect image* button, and a result panel that is empty until a run completes. Four detail panels, for quality metrics, the compression table, inspection images and provenance, appear below the result after a run, followed by the SD-Turbo panel if the model test was requested.

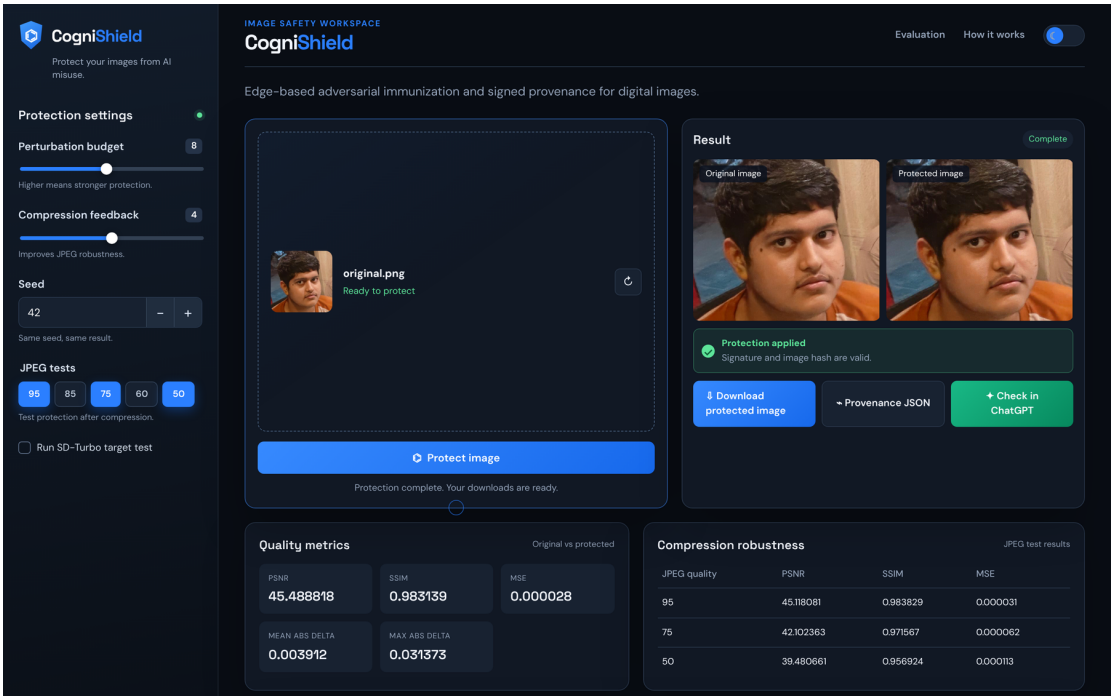


Fig. 6.1 Screenshot of the CogniShield Web Interface in Dark Mode After a Protection Run

A typical session consists of the steps in Table 6.1.

Table 6.1 Steps of a Demonstration Session

Step	User action	System response
1	Open http://127.0.0.1:5000	The workspace loads in dark mode; the Protect button is disabled
2	Drag an image into the upload zone, or click Choose image	A preview and the file name appear; the button is enabled and a status line asks the user to review the settings
3	Adjust budget, feedback, seed and JPEG tests if desired	Sliders update their fill and value labels; the seed can be stepped with the plus and minus buttons
4	Click Protect image	The button shows Protecting, the status chip changes to Processing
5	Wait for the response	Original and protected images appear side by side with a success banner; details panels are shown
6	Inspect metrics, compression table, difference image, noise preview and JPEG previews	Values from the JSON response are rendered
7	Click Download protected image and Provenance JSON	The protected PNG and the signed manifest are saved
8	Optionally tick Run SD-Turbo target test and enter a prompt	A second run shows the model outputs for the original and protected images

The top-right switch changes between dark and light themes and remembers the choice. The *How it works* page opens in the same tab and explains the flow in six cards: input, protection, quality, JPEG tests, provenance and the optional model test. The page also states the local-by-design property: the application runs at 127.0.0.1 and does not send the image to an external upload service. The Evaluation link in the header opens the evaluation dashboard of Section 7.12.5.

6.3 Demonstration 1: Portrait Protection

The first demonstration uses a portrait photograph of 801 × 775 pixels, taken from the team's own project runs. Fig. 6.2 shows the original, the protected image, the

difference image magnified twelve times and the noise preview. At normal size the original and protected images cannot be told apart by eye. The magnified difference shows a fine, speckled pattern that is strongest along the hair, the eyes and the clothing edges and weaker on the skin, which is the behaviour that the edge mask is designed to produce. The noise preview shows the same pattern as a fraction of the budget.

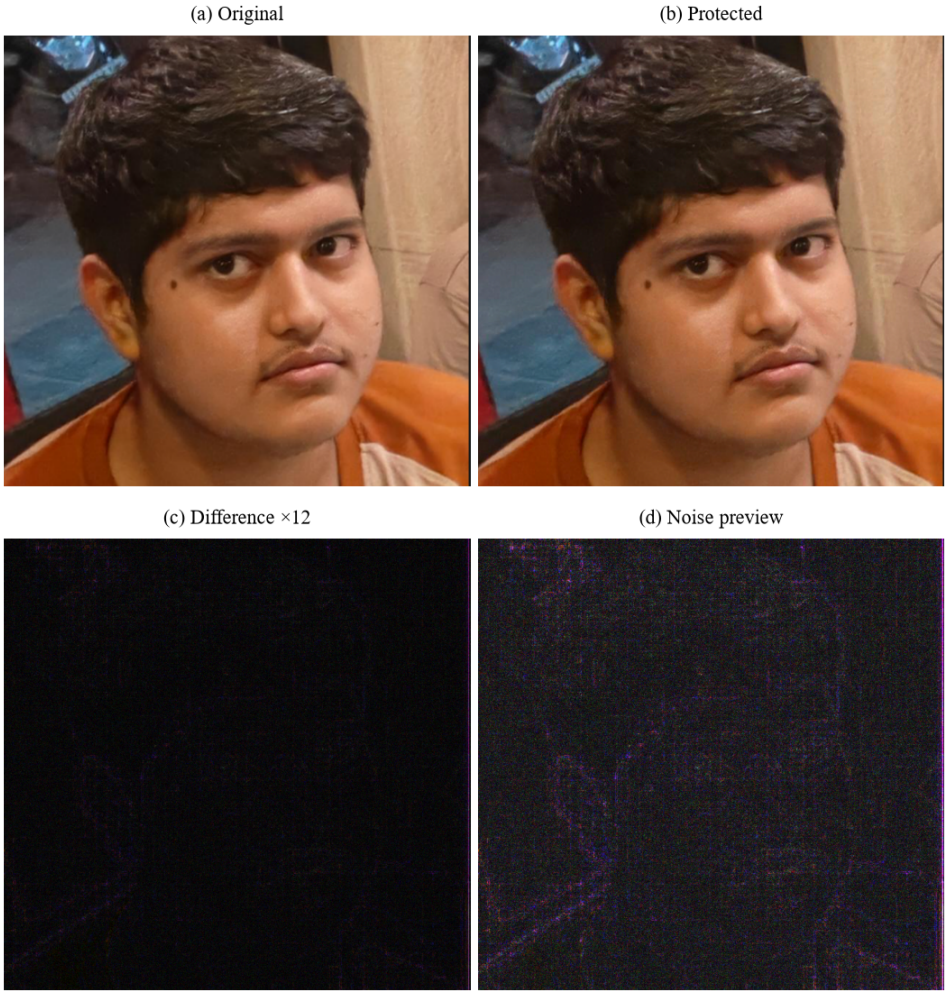


Fig. 6.2 Portrait Run at the Default Settings: Original, Protected, Difference ×12 and Noise Preview

The metrics for this run are listed in Table 6.2. The PSNR of 45.49 dB is well above the 30 dB guideline, the SSIM of 0.9831 is close to 1.0, the average pixel changed by about 1.00 level out of 255 and the largest change was exactly 8 levels, the selected budget. The run took about 0.52 seconds on the machine used for the experiments, including noise generation, the feedback loop and the metric computation for a single image.

Table 6.2 Metrics of the Portrait Run (Budget 8/255, Four Iterations, Seed 42)

Metric	Value
PSNR	45.49 dB
SSIM	0.9831
MSE	0.28×10^{-4}
Mean absolute delta	1.00 levels
Maximum absolute delta	8.00 levels
PSNR after JPEG q=95 / 75 / 50	45.12 / 42.10 / 39.48 dB
SSIM after JPEG q=95 / 75 / 50	0.9838 / 0.9716 / 0.9569

6.4 Demonstration 2: JPEG Variants

The interface writes one JPEG file for every selected quality and shows them next to the metric table. Fig. 6.3 shows the three variants of the protected portrait. The PSNR values in Table 6.2 fall from 45.12 dB at quality 95 to 39.48 dB at quality 50, and most of this decrease is caused by compression itself, not by the perturbation: Section 7.5 shows that for the average image the protected and unprotected JPEG results are within 0.16 dB of one another at quality 50. At all three qualities the images look alike at normal size, with block artefacts becoming noticeable at quality 50 on smooth regions.

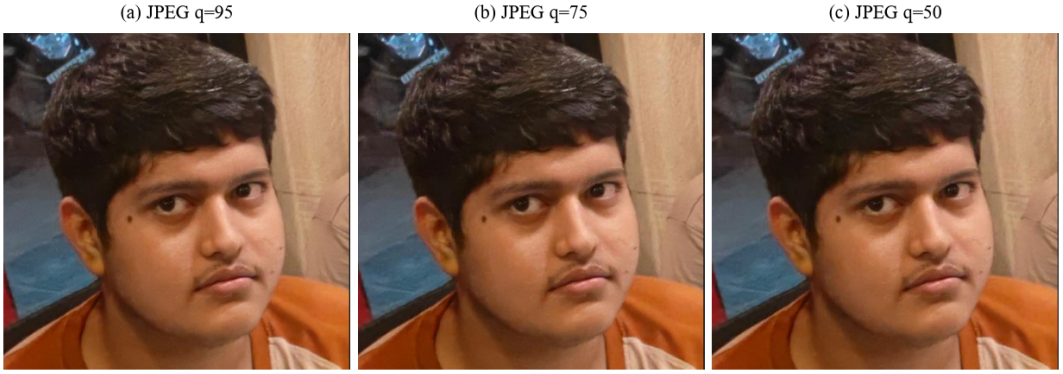


Fig. 6.3 Protected Portrait After JPEG Compression at Qualities 95, 75 and 50

6.5 Demonstration 3: Signed Provenance

After the protected PNG is written, the application signs it and immediately verifies the signature. The manifest of the portrait run is reproduced in the listing below. The digest is the SHA-256 of the protected PNG, the signature is the 64-byte Ed25519 signature encoded in base64, and the metadata record the settings used.

```
{
  "project": "CogniShield",
  "algorithm": "Ed25519 over SHA-256(image_bytes)",
  "image_file": "protected.png",
  "image_hash_sha256":
  "72f0d265dd32131476100d7847719ce044fae02c3c2396baba88f6d8ac8ae954",
```

```
  "signature_base64":
  "fYUaYdHDCzgSWL5Rlv02j+OJF46tuxPzAPj1ciZLShFnSlyZS7aVDWx+7x2ruVXYHZSlnmdEwzSK1pcBhWw9CQ
  "public_key_file": "cognishield_public.pem",
  "signed_at_utc": "2026-10-08T19:41:40.309030+00:00",
  "metadata": {
    "source_filename": "Screenshot_2026-10-08_233221.png",
    "epsilon": 0.03137254901960784,
    "epsilon_pixels": 8, "iterations": 4, "seed": 42,
    "jpeg_qualities": [95, 75, 50]
  }
}
```


Two checks were performed on the recorded runs. First, the SHA-256 digest of every one of the 18 protected images in the web output folder was recomputed and compared with the digest stored in its manifest; all 18 matched. Second, to emulate tampering, one bit in the middle of each protected file was flipped in memory and the digest recomputed; in 0 of the 18 cases did the altered file’s digest still equal the recorded one. Because the signature covers the digest, a changed digest cannot verify. The signature length was 64 bytes in every manifest, as expected for Ed25519. The signature check itself is performed by the application at the end of every protection request, and the interface reports the result in the provenance panel together with the digest.

6.6 Demonstration 4: SD-Turbo Target Test

To observe the effect on a generative model, the portrait was passed through the local SD-Turbo image-to-image pipeline with the default prompt, a realistic portrait photograph with natural lighting, the same seed for both inputs and the strength and step settings of Section 4.1.12. The input was resized to 512 × 488 pixels, as the model requires multiples of eight. Fig. 6.4 shows the output for the original image and for the protected image.

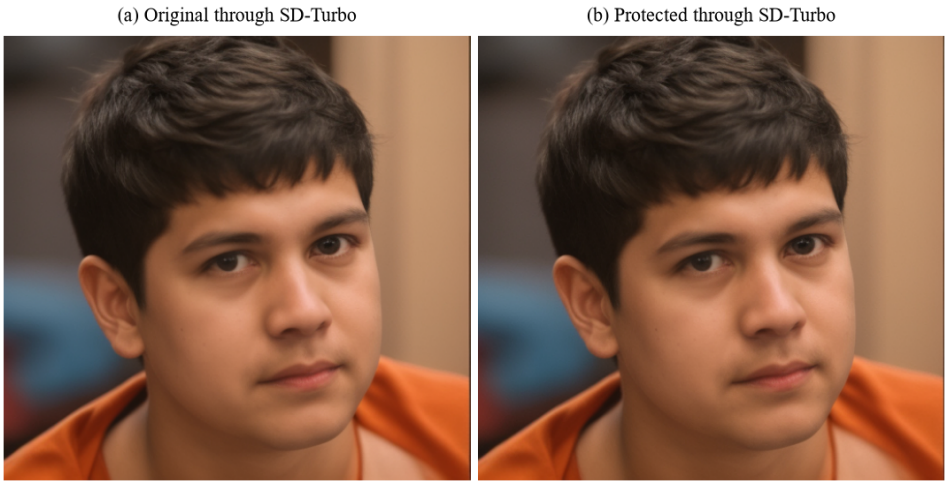


Fig. 6.4 SD-Turbo Image-to-Image Outputs for the Original and the Protected Portrait

Both outputs show a regenerated, smoothed version of the person with the same pose, hair and background colours, which is what image-to-image generation with moderate strength produces. The two outputs are visually almost identical. Fig. 6.5 quantifies this by showing the absolute difference between them, magnified twelve times, and its histogram. The PSNR between the two outputs is 41.06 dB and their SSIM is 0.9863. Section 7.10 interprets this result: at the default settings the perturbation is far too small to make this particular model visibly fail, which is a limitation of the gradient-free design and not a defect in the test. Two separate runs of the model test on the same input gave outputs with identical metrics, confirming that the seeded test is repeatable.

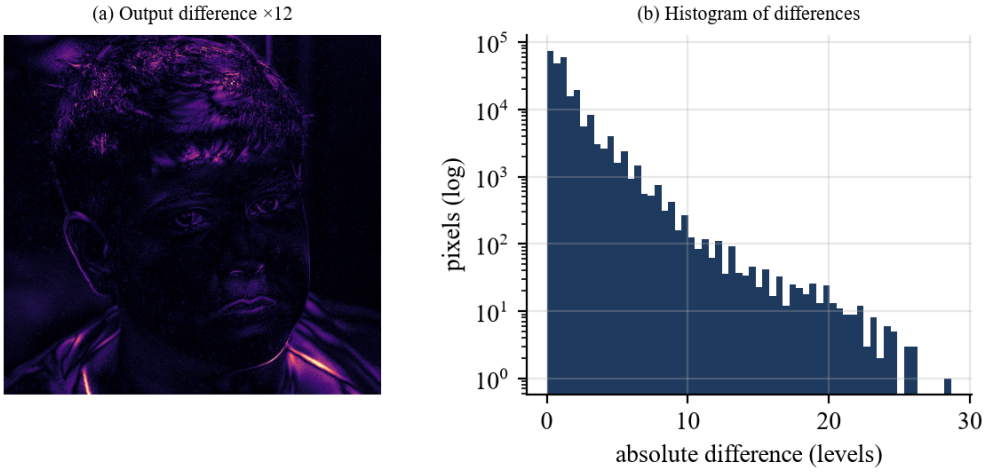


Fig. 6.5 Difference Between the Two SD-Turbo Outputs, Magnified Twelve Times, and Its Histogram

6.7 Demonstration 5: Other Images

The system was also run on images with very different content. Fig. 6.6 shows

a colour portrait with texture from the scikit-image sample data, and Fig. 6.7 shows a photograph of a handwritten notebook page from the team’s own runs, which has large flat regions of paper and fine pen strokes. In both cases the protected images are indistinguishable from the originals at normal size, and the difference images concentrate on the strokes, the textures and the borders. The notebook page is also a useful stress case for the edge mask because most of its area is nearly uniform: the initial perturbation there is limited to the 35 percent floor of the budget, although the feedback term can raise it, as the noise preview in the figure shows.

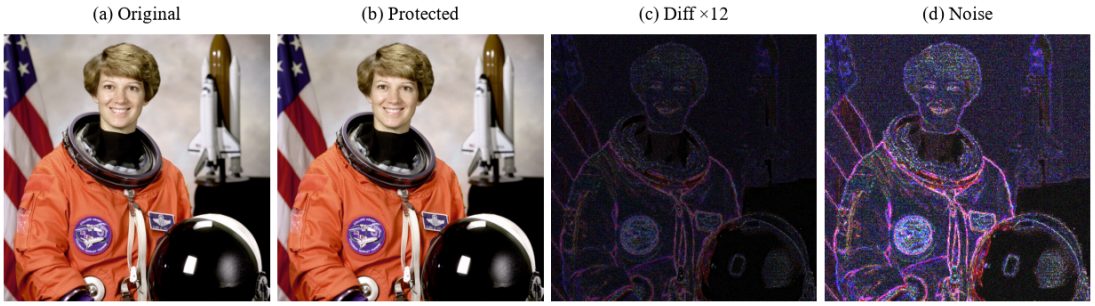


Fig. 6.6 Protection of a Colour Portrait From the Sample Data

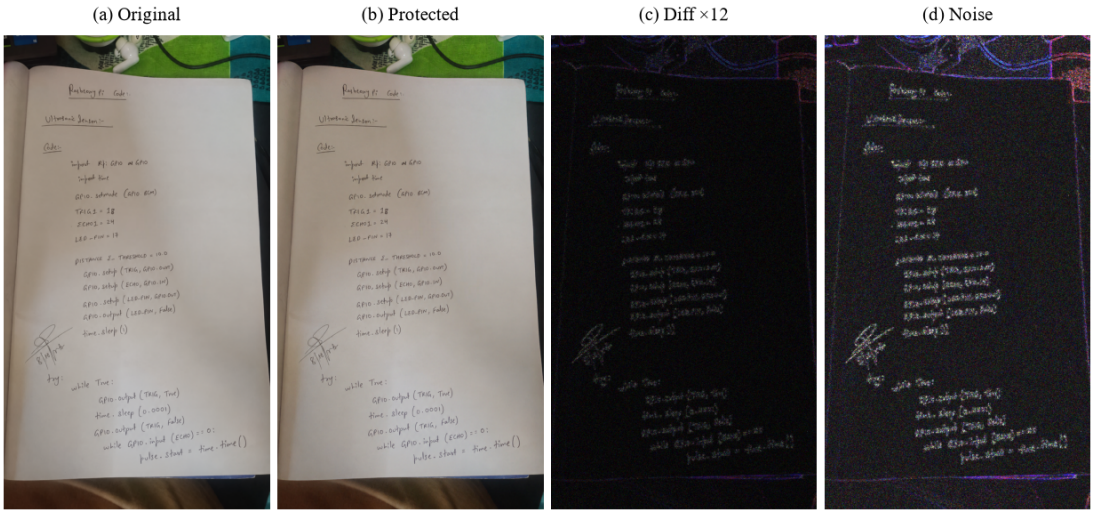


Fig. 6.7 Protection of a Handwritten Notebook Page

6.8 Demonstration 6: Command-Line and Batch Workflow

For automation, the same pipeline can be driven from the command line. Running the single-image program on a file writes a folder named after the file, which contains the protected PNG, the noise preview, the difference image, a JPEG file for each quality, the manifest and a *report.json* that combines the input and output paths, the metrics, the compression results and the verification result. The same report is

printed to the terminal. The batch program applies the protection to every image in the input folder and writes *summary.csv*, which has one row per image with its width, height, PSNR, SSIM, MSE, mean absolute delta, maximum absolute delta and the PSNR and SSIM at each JPEG quality. The experiments reported in Chapter 7 were produced by calling the same library functions that these programs call, on 14 images, and so the data in the tables correspond to what the batch program would produce for the same inputs.

6.9 Behaviour in Error Situations

A usable tool must fail clearly. Table 6.3 lists test cases for the behaviour of the web application in normal and abnormal situations, with the expected response according to the implementation and how it was checked.

Table 6.3 Functional Test Cases and Expected Behaviour

ID	Case	Expected behaviour	Checked by
TC1	Valid PNG with default settings	Success response with images, metrics, compression table and a valid provenance result	Recorded web runs
TC2	Protect pressed with no file	HTTP 400, message to choose an image first	Code review
TC3	File with an unsupported extension	HTTP 400, message to use PNG, JPG, JPEG or WEBP	Code review
TC4	Unreadable file with a valid extension	HTTP 400, message that the file could not be read as an image	Code review
TC5	Budget outside 1–16 or iterations outside 0–8	Value clamped to the allowed range	Code review
TC6	No JPEG quality ticked	Default qualities 95 and 75 are used	Code review
TC7	Upload larger than 200 MB	HTTP 413, message about the 200 MB limit	Code review
TC8	SD-Turbo requested without the model folder	Protection succeeds; the model panel reports that the directory was not found	Code review
TC9	Same image, seed and settings twice	Bit-identical protected images	Experiment, Section 7.8
TC10	One bit of the protected file changed	Digest differs from the manifest; verification fails	Experiment, Section 7.9

The web application also tolerates operating-system level failure of the model test: if the worker process exits abnormally or exceeds the five-minute limit, the application reports this and still returns the signed protection result.

CHAPTER 7
COST ANALYSIS / RESULT & DISCUSSION

This chapter reports the experimental evaluation of CogniShield and then discusses what the results mean, what they do not show and what the system costs to build and to run. All numbers in this chapter were produced by running the project's own library functions (*protect_image*, *calculate_metrics*, *jpeg_roundtrip* and the provenance hashing) and were written to a data file from which the tables and figures were generated, so each value can be reproduced. Section 7.12 reports a second experiment on 500 images of the MS-COCO dataset, whose numbers were produced by the evaluation runner of Section 4.1.20.

7.1 Experimental Setup

7.1.1 Test images

The evaluation uses 14 images, listed in Table 7.1 and shown in Fig. 7.1. Two are images from the team's own recorded runs, a portrait photograph and a photograph of a handwritten notebook page. The remaining twelve are sample images that ship with the scikit-image library and were chosen to cover different content: photographs of people, animals and objects, three textures (brick, grass and gravel), two grayscale images, a microscopy image, an astronomical image and a retinal image. The two largest images were reduced to at most 800 pixels on the longer side and the notebook photograph to at most 1000 pixels to keep experiments quick. Grayscale images are converted to three-channel RGB by the same loading rule that the application uses.

Table 7.1 Test Images Used in the Experiments

No.	Image	Content	Size (px)	Source
1	portrait	Portrait photograph	801 × 775	Project run (web app)
2	notebook	Handwritten notebook page	562 × 1000	Project run (web app)
3	astronaut	Portrait with texture	512 × 512	scikit-image data
4	coffee	Still life	600 × 400	scikit-image data
5	chelsea	Cat photograph	451 × 300	scikit-image data
6	rocket	Outdoor object	640 × 427	scikit-image data
7	immunohistochemistry	Microscopy stain	512 × 512	scikit-image data
8	brick	Texture	512 × 512	scikit-image data
9	grass	Texture	512 × 512	scikit-image data
10	gravel	Texture	512 × 512	scikit-image data
11	camera	Photographer, grayscale	512 × 512	scikit-image data (grayscale)
12	moon	Lunar surface, grayscale	512 × 512	scikit-image data (grayscale)
13	hubble	Astronomical field	800 × 697	scikit-image data (resized)
14	retina	Retinal fundus	800 × 800	scikit-image data (resized)



Fig. 7.1 Contact Sheet of the 14 Test Images

7.1.2 Protocol

Unless a section states otherwise, every experiment uses the default configuration of the web interface: a budget of 8/255, four feedback iterations, seed 42 and feedback qualities 95, 75 and 50. For each image the original is compared with the protected image, and with the protected image after JPEG round trips at qualities 95, 75 and 50. As a baseline, the original is also compared with its own JPEG round trip at the same qualities, so that the cost of compression alone can be separated from the cost of the perturbation. Sensitivity studies vary one parameter at a time on a subset of four images, the portrait, the colour portrait, the still life and the notebook page, and average the results. All experiments ran on CPU on a Windows 11 computer with Python 3.12.

7.1.3 Measures

The measures are those defined in Section 4.1.9: PSNR, SSIM, MSE, mean and maximum absolute delta. For the study of how much of the perturbation survives a transformation T, a further measure is introduced for this report. Let δ be the perturbation added to an image x, so that the protected image is $x + \delta$. The *retention* ρ_T is the coefficient obtained by projecting the change that T leaves between the transformed protected and transformed original images onto δ, as given below. A value of 1 means that the perturbation passes through the transformation unchanged along its own direction, a value near 0 means that it has been removed, and values above 1 can occur when the transformation interacts with the image content. The measure is a linear

proxy for how much of the protective signal remains; it does not measure the effect on a generative model.

$$\rho_T = \frac{\langle \delta,\ T(x+\delta) - T(x) \rangle}{\|\delta\|^2} \tag{7.1}$$

In the Eqn. 7.1 the variable ρ_T denotes the retention under the transformation T, δ denotes the perturbation added to the image, x denotes the original image, T denotes the transformation applied to an image, and the angle brackets denote the inner product.

7.2 Imperceptibility of the Protection

Table 7.2 reports the metrics for each image at the default settings, and Fig. 7.2 plots PSNR and SSIM per image. All 14 images exceed the 30 dB guideline: the mean PSNR is 41.56 dB with a standard deviation of 2.72 dB, the lowest value is 38.07 dB for the *grass* texture and the highest is 45.78 dB for the *moon* image. All images also have an SSIM above 0.95, with a mean of 0.9800; the lowest value is 0.9618 for *hubble* and the highest is 0.9938 for *grass*. The mean squared error is of the order of 10^{-4}, the average pixel changes by 1.62 levels out of 255 (between 0.97 and 2.49 across images) and the maximum absolute change is exactly 8.00 levels in every case, which confirms that the clip enforces the budget.

Table 7.2 Imperceptibility Metrics at the Default Settings (ε = 8/255, 4 Iterations, Seed 42)

Image	PSNR (dB)	SSIM	MSE ($\times 10^{-4}$)	Mean abs. Δ (levels)	Max abs. Δ (levels)	Time (s)
portrait	45.49	0.9831	0.28	1.00	8.00	0.52
notebook	44.02	0.9788	0.40	1.12	8.00	0.35
astronaut	39.79	0.9763	1.05	1.82	8.00	0.17
coffee	38.66	0.9703	1.36	2.16	8.00	0.17
chelsea	41.04	0.9813	0.79	1.66	8.00	0.10
rocket	39.11	0.9703	1.23	1.82	8.00	0.21
immunohistochemistry	40.82	0.9880	0.83	1.78	8.00	0.20
brick	44.41	0.9860	0.36	1.13	8.00	0.18
grass	38.07	0.9938	1.56	2.49	8.00	0.21
gravel	39.86	0.9935	1.03	2.01	8.00	0.22
camera	41.79	0.9820	0.66	1.46	8.00	0.19
moon	45.78	0.9798	0.26	0.97	8.00	0.20
hubble	38.71	0.9618	1.35	2.24	8.00	0.50
retina	44.24	0.9747	0.38	1.11	8.00	0.51
Mean	**41.56**	**0.9800**	**0.82**	**1.62**	**8.00**	**0.27**

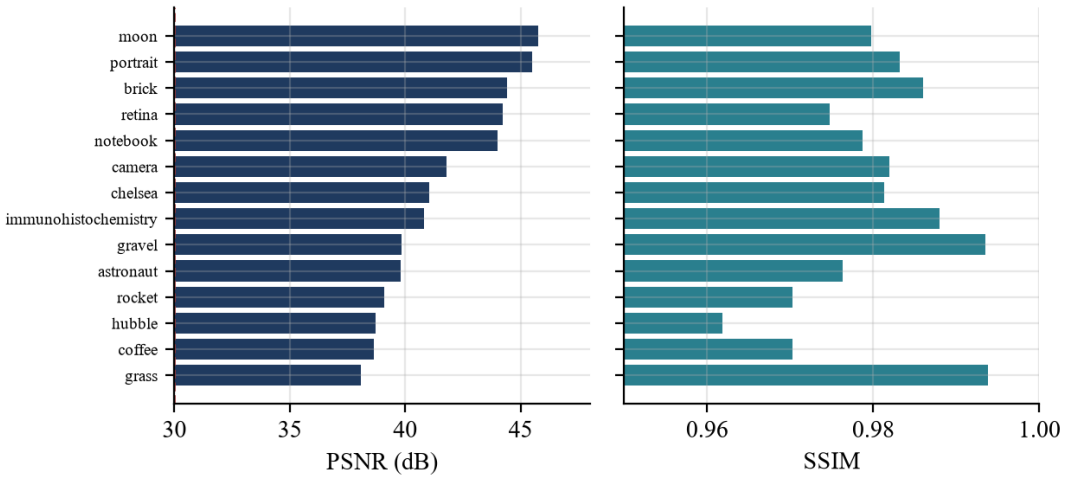


Fig. 7.2 PSNR and SSIM of the Protected Images at the Default Settings

The ordering of the images shows how content influences the cost of protection. Images with a lot of fine detail, such as the grass texture, the astronomical field and the still life, have lower PSNR because the edge mask directs more of the budget to them, so their average pixel change is larger. Smooth images, such as the lunar surface and the portrait, have the highest PSNR. PSNR and SSIM do not rank the images identically: the grass texture has the lowest PSNR of all images but the highest SSIM, and the gravel texture has a below-median PSNR and the second-highest SSIM. This is expected, because SSIM is partly insensitive to small changes in regions of high local variance, where texture masks the perturbation, which is also the property that the edge mask exploits.

Fig. 7.3 shows the distribution of the integer pixel changes for the colour portrait. The histogram is concentrated near zero, falls off rapidly towards the budget limits at plus and minus 8 levels, and is symmetric, which reflects the zero-mean noise. Only a small fraction of the pixels reaches the extreme values, because the product of the normalised noise and the mask is rarely at its maximum.

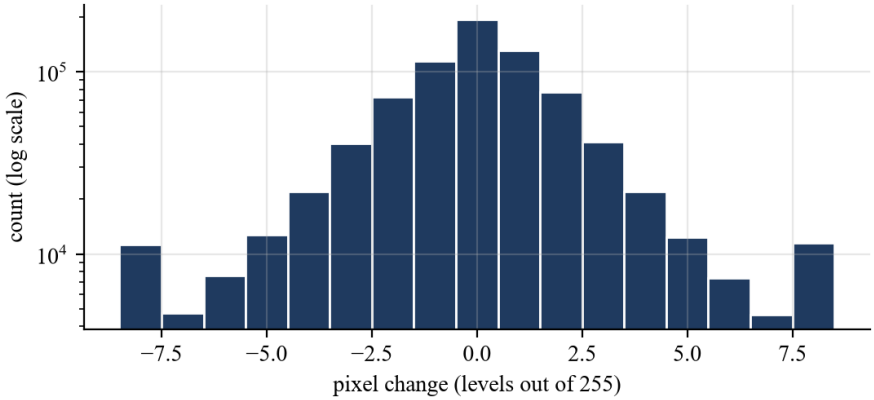


Fig. 7.3 Distribution of Pixel Changes for the Colour Portrait (Budget 8/255)

7.3 Effect of the Perturbation Budget

The budget is the main control, and Table 7.3 and Fig. 7.4 show its effect. PSNR falls from 50.25 dB at a budget of 1 to 39.06 dB at a budget of 16, a decrease of 11.18 dB over a sixteen-fold increase in the allowed change, and SSIM falls from 0.9953 to 0.9508. The maximum absolute change follows the budget exactly. Even at the largest budget the mean PSNR is well above 30 dB. A pure doubling of the perturbation amplitude would lower PSNR by about 6 dB, but the measured decrease per doubling is smaller, between 2.5 and 3.2 dB; a likely reason is that the final conversion to 8-bit integers and the feedback loop make the realised perturbation grow more slowly than the budget.

Table 7.3 Effect of the Perturbation Budget ε (Mean of 4 Images, 4 Iterations)

ε (of 255)	PSNR (dB)	SSIM	Mean abs. Δ (levels)	Max abs. Δ (levels)	ρ at q=75	ρ at q=50
1	50.25	0.9953	0.63	1.00	0.43	0.28
2	47.07	0.9915	0.89	2.00	0.50	0.29
4	44.46	0.9863	1.18	4.00	0.53	0.25
6	43.03	0.9819	1.36	6.00	0.52	0.22
8	41.99	0.9771	1.52	8.00	0.48	0.19
10	41.13	0.9717	1.67	10.00	0.44	0.16
12	40.37	0.9655	1.82	12.00	0.39	0.14
16	39.06	0.9508	2.13	16.00	0.32	0.11

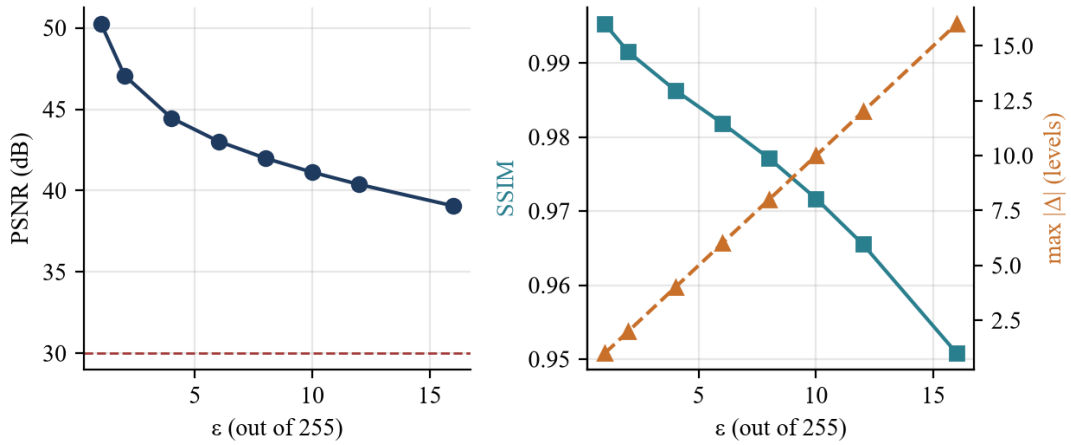


Fig. 7.4 Effect of the Perturbation Budget on PSNR, SSIM and Maximum Change

The retention columns of Table 7.3 show that a larger budget does not translate into a larger surviving fraction. For JPEG at quality 75 the retention is 0.43 at a budget of 1, rises to 0.53 at a budget of 4, is 0.48 at the default of 8 and falls to 0.32 at 16; for quality 50 it falls monotonically from 0.29 at a budget of 2 to 0.11 at 16. Because the retention is normalised by the energy of the perturbation, this means that the compression removes proportionally more of a large perturbation, which suggests that raising the budget buys less additional robustness than it costs in quality, and that moderate budgets between 4 and 8 give the best balance in these measurements.

7.4 Effect of the Compression Feedback

Table 7.4 and Fig. 7.5 show how the number of feedback iterations changes quality and retention. Without feedback the perturbation is only the masked noise; its PSNR is 51.90 dB and the retention is 0.21, 0.10 and 0.07 at qualities 95, 75 and 50. With the default four iterations the PSNR is 41.99 dB and the retention is 0.55, 0.48 and 0.19. Feedback therefore multiplies the surviving fraction at quality 95 by about 2.7 and at quality 75 by about 5.0, which is the intended effect of the loop.

Table 7.4 Effect of the Number of Compression-Feedback Iterations (Mean of 4 Images, ε = 8/255)

Iterations	PSNR (dB)	SSIM	ρ at q=95	ρ at q=75	ρ at q=50	PSNR of JPEG q=50 (dB)
0	51.90	0.9960	0.21	0.10	0.07	34.62
1	49.27	0.9942	0.37	0.33	0.20	34.62
2	46.23	0.9906	0.48	0.46	0.22	34.59
3	43.84	0.9849	0.53	0.50	0.21	34.54
4	41.99	0.9771	0.55	0.48	0.19	34.47
5	40.49	0.9674	0.56	0.46	0.17	34.40
6	39.23	0.9558	0.56	0.43	0.15	34.34
7	38.14	0.9423	0.55	0.39	0.13	34.28
8	37.17	0.9271	0.54	0.36	0.11	34.22

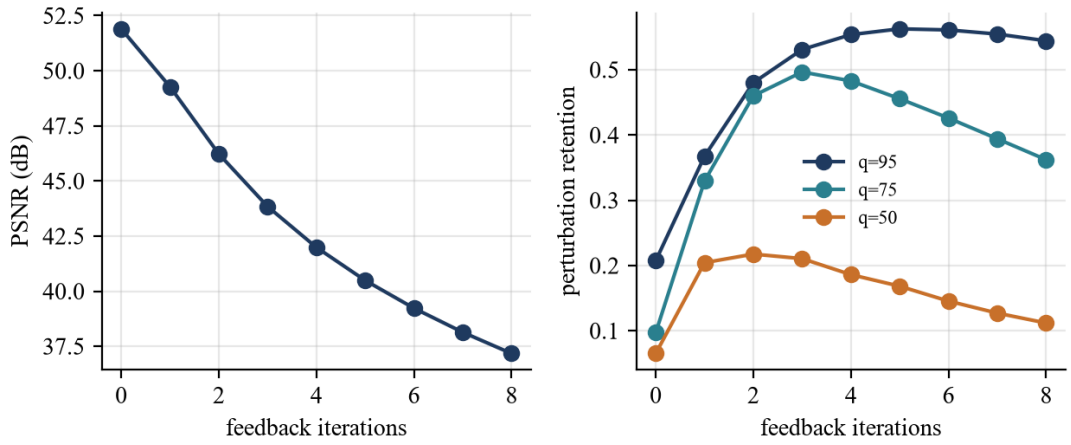


Fig. 7.5 Effect of the Number of Feedback Iterations on PSNR and Perturbation Retention

Two cautions apply. First, the improvement is bought with a larger perturbation: the PSNR decreases by about 9.9 dB between zero and four iterations, and by a further several decibels up to eight iterations, where it reaches 37.17 dB with an SSIM of 0.9271. Second, the benefit saturates and then reverses. The retention at quality 95 peaks at 0.56 after 5 iterations, and at quality 75 it peaks at 0.50 after 3 iterations before decreasing to 0.36 after eight, so more than three to four iterations are counter-productive for the lower qualities. The default of four is close to the plateau for quality 95, but two or three iterations give nearly the same retention at quality 75 (0.46 and 0.50) with a PSNR that is 4.2 and 1.9 dB higher than the default. This suggests that three iterations may be a better default; the choice was not changed in the evaluated version. Finally, the PSNR of the JPEG-compressed protected image at quality 50 barely changes with the number of iterations, from 34.62 dB to 34.22 dB, because at that quality the compression error dominates everything else.

7.4.1 Composition of the final perturbation

The worked example of Section 4.1.19 suggested that the final perturbation is not mainly the seeded noise. To measure this for all images, three quantities were computed at the default settings: the mean absolute size of the initial perturbation δ_0 and of the final perturbation, the fraction of pixel-channel values that sit at the budget limit, and the correlation of the final perturbation with the initial noise perturbation and with the image's own JPEG residual, defined as the mean over qualities 95, 75 and 50 of the difference between the image and its compressed version. Table 7.5 gives the results and Fig. 7.6 plots the two correlations.

Table 7.5 Composition of the Final Perturbation at the Default Settings

Image	Pixel-channels at the budget (%)	Mean abs. initial δ0 (levels)	Mean abs. final δ (levels)	Correlation with δ0	Correlation with JPEG residual
portrait	0.1	0.45	1.00	0.72	0.58
notebook	0.4	0.46	1.12	0.62	0.68
astronaut	2.9	0.47	1.82	0.34	0.85
coffee	4.2	0.48	2.16	0.31	0.86
chelsea	0.7	0.51	1.66	0.47	0.81
rocket	6.5	0.46	1.82	0.33	0.82
immunohistochemistry	0.4	0.54	1.78	0.48	0.80
brick	0.0	0.52	1.13	0.75	0.57
grass	2.9	0.53	2.49	0.32	0.90
gravel	0.8	0.55	2.01	0.43	0.82
camera	0.7	0.48	1.46	0.48	0.79
moon	0.0	0.45	0.97	0.75	0.56
hubble	3.7	0.47	2.24	0.30	0.81
retina	0.2	0.45	1.11	0.59	0.71
Mean	**1.7**	**0.49**	**1.62**	**0.49**	**0.75**

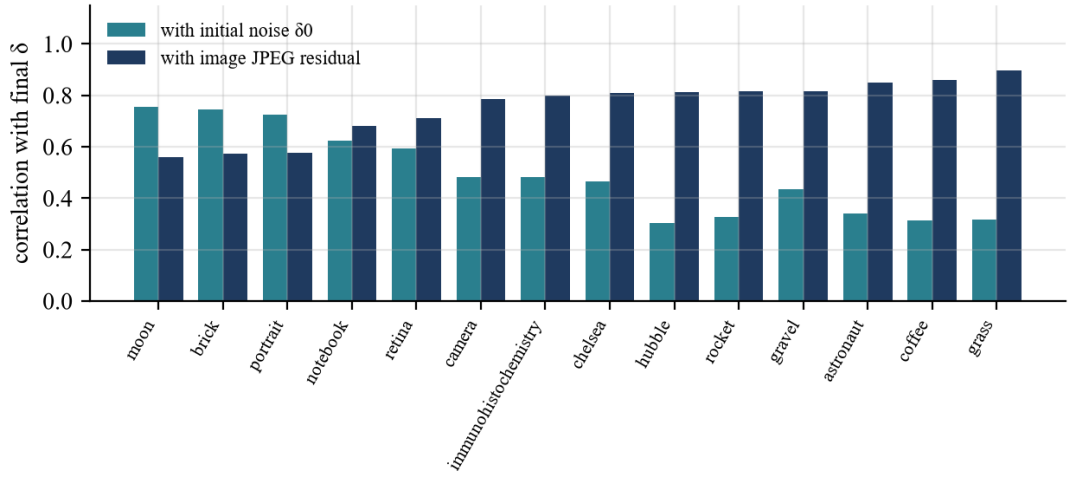


Fig. 7.6 Correlation of the Final Perturbation With the Initial Noise and With the Image's Own JPEG Residual

The initial perturbation is small, with a mean absolute size of 0.49 levels, and the final perturbation is about 3.3 times larger, at 1.62 levels. On average only 1.7 percent of the pixel-channel values reach the budget limit of 8 levels, at most 6.5 percent for *rocket*, so the budget is a ceiling that most pixels never touch, and the typical change is a small fraction of it. The correlation of the final perturbation with the image's own JPEG residual averages 0.75 and ranges from 0.56 for *moon* to 0.90 for *grass*, while its correlation with the initial noise averages 0.49 and is highest for *moon* (0.75). The initial noise and the JPEG residual are essentially uncorrelated with each other (the largest absolute correlation between them is 0.003), so the two correlations describe two distinct contributions. For smooth images such as the lunar surface, the portrait and the brick texture, the seeded noise still accounts for a visible share of the final perturbation; for textured images such as the grass, the still life and the astronomical field, the image's

own residual dominates.

Two consequences follow. The protection is **image-adaptive**: its structure depends on the content as much as on the seed, which explains why different seeds change the quality figures so little (Section 7.8) and helps explain why the ratio of edge to flat perturbation in Section 7.7 exceeds what the mask alone can produce. And the effective strength of the protection is governed by how detailed the image is, not by the budget alone: for a smooth image the final perturbation is only about one level on average, which is why a large budget has a modest effect on PSNR in Section 7.3. These findings also qualify the notion of a ”budget of 8”: it bounds the worst case, and the typical change is much smaller.

7.5 Robustness to JPEG Compression

Table 7.6 compares, on average over the 14 images, the quality of the protected image after JPEG compression with the quality of the original after the same compression, and Table 7.7 gives the per-image values of the protected image. Fig. 7.7 shows the averages graphically. At quality 95 the protected image reaches 39.63 dB, which is 3.37 dB lower than the compressed original (43.00 dB), so the perturbation still makes a measurable contribution to the error. At quality 75 the difference shrinks to 0.94 dB and at quality 50 to 0.16 dB. This means that as compression becomes harsher, the compression error swamps the perturbation, and the protected and unprotected images become equally distant from the original.

Table 7.6 Mean Effect of JPEG Compression With and Without Protection (14 Images)

JPEG quality	PSNR: JPEG of original	PSNR: JPEG of protected	Gap (dB)	SSIM: JPEG of original	SSIM: JPEG of protected
95	43.00	39.63	3.37	0.9792	0.9687
75	36.22	35.29	0.94	0.9454	0.9363
50	33.96	33.80	0.16	0.9182	0.9158

Table 7.7 Quality of the Protected Image After JPEG Compression, Compared With the Original

Image	PSNR q=95	PSNR q=75	PSNR q=50	SSIM q=95	SSIM q=75	SSIM q=50
portrait	45.12	42.10	39.48	0.9838	0.9716	0.9569
notebook	42.88	38.10	36.02	0.9774	0.9612	0.9505
astronaut	37.47	33.66	32.00	0.9582	0.9334	0.9178
coffee	36.40	31.87	30.40	0.9488	0.8975	0.8673
chelsea	38.95	35.06	33.70	0.9710	0.9400	0.9170
rocket	34.70	31.76	30.44	0.9482	0.9069	0.8833
immunohistochemistry	38.64	34.54	32.78	0.9793	0.9509	0.9282
brick	43.62	40.27	38.74	0.9860	0.9757	0.9698
grass	35.99	26.72	26.89	0.9903	0.9278	0.9191
gravel	37.61	31.82	30.35	0.9887	0.9593	0.9437
camera	39.99	33.78	32.39	0.9769	0.9405	0.9115

Image	PSNR q=95	PSNR q=75	PSNR q=50	SSIM q=95	SSIM q=75	SSIM q=50
moon	45.11	42.23	40.87	0.9781	0.9639	0.9527
hubble	34.63	31.79	30.78	0.9042	0.8303	0.7943
retina	43.71	40.30	38.36	0.9716	0.9491	0.9091
Mean	**39.63**	**35.29**	**33.80**	**0.9687**	**0.9363**	**0.9158**

Table 7.8 Perturbation Retention ρ After JPEG Compression for Each Image

Image	ρ at q=95	ρ at q=75	ρ at q=50
portrait	0.38	0.34	0.17
notebook	0.58	0.57	0.24
astronaut	0.61	0.44	0.16
coffee	0.65	0.57	0.17
chelsea	0.81	0.55	0.17
rocket	0.30	0.33	0.10
immunohistochemistry	0.85	0.65	0.24
brick	0.55	0.41	0.13
grass	1.09	4.58	0.36
gravel	0.97	1.04	0.29
camera	0.88	1.08	0.26
moon	0.52	0.33	0.10
hubble	0.60	0.29	0.10
retina	0.52	0.36	0.20
Median	**0.60**	**0.50**	**0.17**

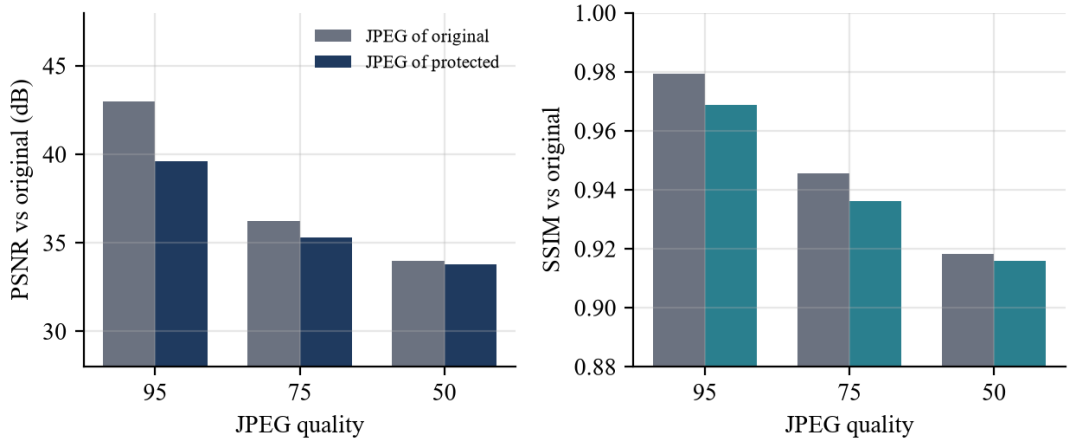


Fig. 7.7 Mean PSNR and SSIM After JPEG Compression, With and Without Protection

The retention measure gives a more direct view of how much of the perturbation survives. The median retention over the 14 images is 0.60 at quality 95, 0.50 at quality 75 and 0.17 at quality 50; the means are 0.66, 0.82 and 0.19. The mean at quality 75 is inflated by a single outlier, the grass texture, whose retention of 4.58 exceeds 1: for this image the quantisation at quality 75 presumably interacts strongly with the fine texture, and since the measure is not bounded by 1, the median is the more representative statistic. In summary, roughly half of the perturbation along its own direction survives at qualities 95 and 75 and less than a fifth survives at quality 50. These values are consistent with the discussion in Section 4.1.10: JPEG discards high-frequency coefficients, and a

perturbation of a few levels does not survive coarse quantisation.

Compression also reveals which images are fragile. The astronomical image has the lowest SSIM after compression, 0.8303 at quality 75, not because of the perturbation but because its many small, bright objects on a dark background are damaged by the codec itself; its baseline compressed-original SSIM is similar. This is a reminder that quality figures after compression depend strongly on image content.

7.6 Robustness to Other Transformations

JPEG is not the only operation that images undergo. Table 7.9 reports the retention of the perturbation under five further transformations applied to six images, and Fig. 7.8 plots the means with their standard deviations. Adding independent Gaussian noise leaves the retention at 1.00 by construction, because noise independent of the perturbation does not remove it along its direction, although it may still disturb a model. Resizing the image to half its size and back retains only 0.18 of the perturbation, Gaussian blur with a radius of one pixel retains 0.28 and a 3 × 3 median filter retains 0.30. These operations are low-pass filters and remove high-frequency content, which is exactly where the perturbation lives.

Table 7.9 Perturbation Retention ρ Under Common Transformations (6 Images)

Transformation T	Mean ρ	Std. dev.
None (identity)	1.00	0.00
JPEG, quality 95	0.55	0.18
JPEG, quality 75	0.47	0.11
JPEG, quality 50	0.17	0.04
Down-scale to 50% and back (bilinear)	0.18	0.03
Gaussian blur, σ = 1 px	0.28	0.04
Median filter, 3 × 3	0.30	0.07
Added Gaussian noise, σ = 4/255	1.00	0.00

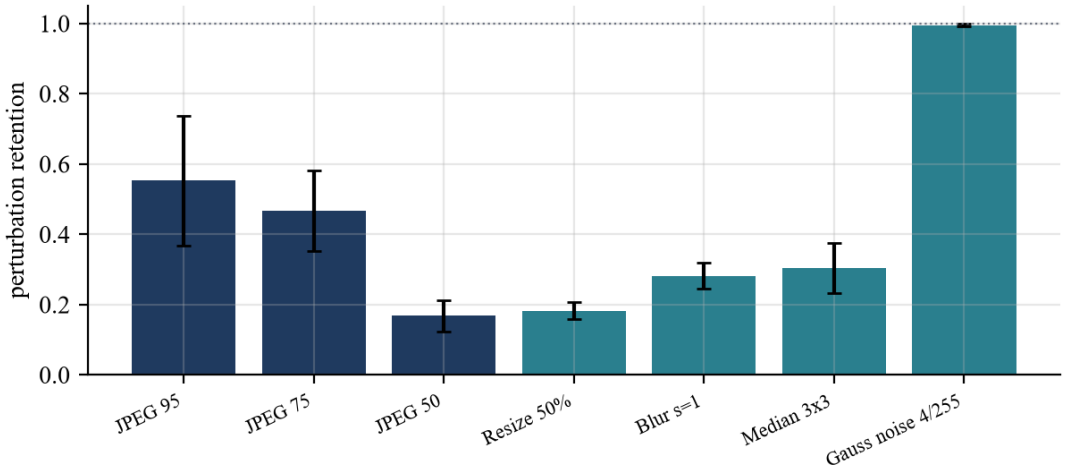


Fig. 7.8 Mean Perturbation Retention Under Compression and Filtering (Error Bars Show One Standard Deviation)

The consequence is a clear and honest limitation. Any sharing path that rescales, blurs or denoises the image removes most of the perturbation, and so a protection that relies on a high-frequency signal cannot be expected to survive such paths. This agrees with the limitations stated at the outset, namely that strong cropping, resizing, denoising or screenshots may weaken the perturbations. Making the perturbation robust to these operations would require lower-frequency or optimisation-based designs, which are listed as future work.

7.7 Edge-Aware Allocation of the Perturbation

The edge mask is meant to concentrate the perturbation on detailed regions. To verify this, the mean absolute perturbation was computed over the 10 percent of pixels with the largest gradient and over the 50 percent with the smallest gradient, and their ratio r, defined below, was computed for every image. Table 7.10 and Fig. 7.9 show the results.

$$r = \frac{\text{mean } |\delta| \text{ over top-10\% gradient pixels}}{\text{mean } |\delta| \text{ over bottom-50\% gradient pixels}} \tag{7.2}$$

In the Eqn. 7.2 the variable r denotes the ratio of the edge perturbation to the flat-area perturbation, and the numerator and the denominator denote the mean absolute perturbation δ over the 10 percent of pixels with the largest gradient and over the 50 percent with the smallest gradient.

Table 7.10 Edge-Aware Allocation of the Perturbation (Values on the 0–1 Pixel Scale)

Image	Mean abs. δ at edges	Mean abs. δ in flat areas	Ratio r
portrait	0.0050	0.0036	1.40
notebook	0.0095	0.0036	2.65
astronaut	0.0136	0.0047	2.89
coffee	0.0163	0.0056	2.89
chelsea	0.0107	0.0050	2.16
rocket	0.0180	0.0036	5.04
immunohistochemistry	0.0089	0.0062	1.44
brick	0.0073	0.0034	2.12
grass	0.0143	0.0083	1.72
gravel	0.0109	0.0068	1.62
camera	0.0113	0.0036	3.17
moon	0.0056	0.0035	1.59
hubble	0.0174	0.0070	2.48
retina	0.0070	0.0035	2.01

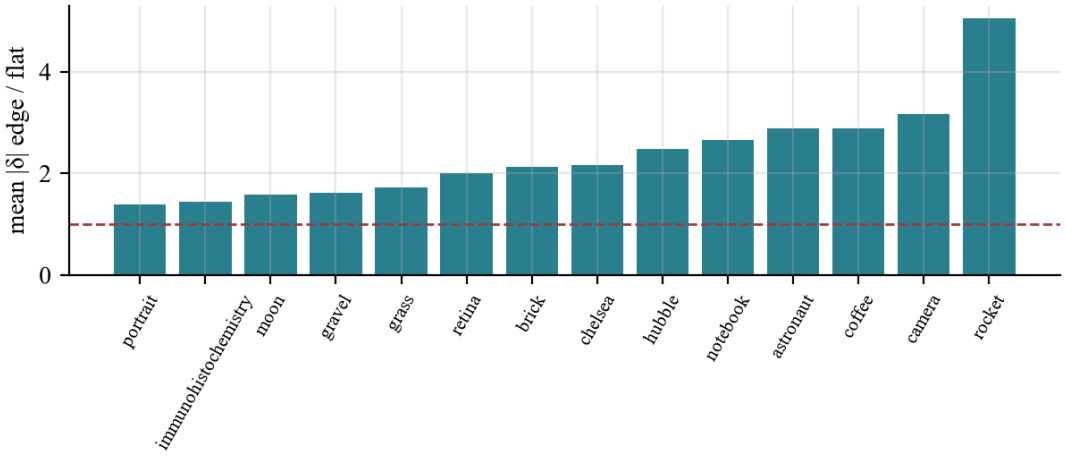


Fig. 7.9 Ratio of Mean Perturbation at Edges to Mean Perturbation in Flat Areas

The ratio exceeds 1 for all 14 images, with a mean of 2.37 and a median of 2.14, so the perturbation is indeed larger on average on detailed pixels than on flat ones. The ratio is largest for *rocket* (5.04) and smallest for *portrait* (1.40). The mask alone cannot explain the largest ratios. Its values span 0.35 to 1, so for noise of equal amplitude the ratio could not exceed about 2.9, yet *rocket* and *camera* exceed that value. The most likely additional cause is the feedback loop, because the JPEG residual that it adds back is itself largest at edges and textures and close to zero in flat areas, so the loop further concentrates the perturbation where the image has detail. Clipping at the limits of the intensity range in saturated regions may also play a role.

7.8 Reproducibility and Seed Sensitivity

Two runs with the same image, seed and settings produced identical protected images, bit for bit, which satisfies goal G4. Different seeds give different perturbations: the ten seeds of Table 7.11 gave 10 distinct output digests. The visual quality is stable

across seeds. The PSNR ranges from 45.11 to 45.90 dB, with a standard deviation of 0.262 dB, and the SSIM from 0.9813 to 0.9849, so the choice of seed has a much smaller effect on quality than the choice of image or budget. Fig. 7.10 shows the PSNR per seed.

Table 7.11 Seed Sensitivity on the Portrait Image (ε = 8/255, 4 Iterations)

Seed	PSNR (dB)	SSIM	Output digest (first 12 hex)
0	45.65	0.9839	0a872d67319d
1	45.46	0.9830	3df51b11e313
2	45.89	0.9849	1cdcfae8416d
3	45.90	0.9849	513dcadae6a1
4	45.52	0.9833	5acb00455fd3
5	45.11	0.9813	abe2958dc577
6	45.28	0.9822	b0c411dbb296
7	45.83	0.9846	f3d87dbb024a
8	45.75	0.9843	280bdbb81da9
9	45.57	0.9835	c1a1914b75ca

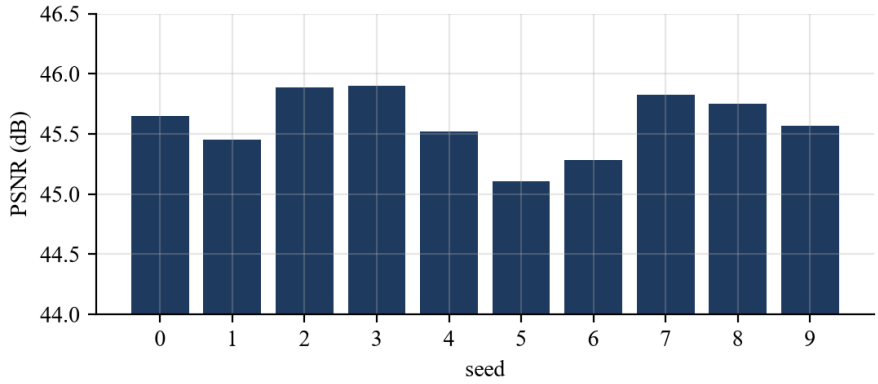


Fig. 7.10 PSNR of the Portrait Image for Ten Different Seeds

7.9 Provenance Verification

The provenance experiments, whose method is described in Section 6.5, used the 18 protected images recorded by the web application. Table 7.12 summarises the outcome. For every image the SHA-256 digest recomputed from the file equals the digest stored in the manifest, and flipping a single bit of the file makes the digest differ in every case. Since the signature is computed over the digest, a modified file cannot present a valid signature for the stored digest. The signature length was 64 bytes in every manifest. The signature verification itself is executed by the application on every request and its result is displayed to the user, so goal G5 is met at the level of the integrity of the exact file bytes.

Table 7.12 Provenance Integrity Results

Check	Result	Cases
Recomputed SHA-256 equals manifest digest	All match	18 of 18
Digest equals manifest digest after one flipped bit	None match	0 of 18
Signature length (Ed25519)	64 bytes	18 of 18
Hash-comparison failure reported before signature check	By design	Code review

It is important to restate what these results do and do not prove. They show that the manifest describes the exact bytes of the protected file and that tampering with the file is detectable. They do not show that the image depicts a real scene, who created it, or that it has not been copied elsewhere, and a user who can run the application can create valid signatures with their own key. The scheme therefore provides integrity and origin-from-this-installation, not authenticity of content.

7.10 Effect on the SD-Turbo Target Model

The target-model test was run on the portrait with the default prompt, strength and seed, as described in Section 6.6. Table 7.13 lists the measurements and Fig. 6.4 and Fig. 6.5 show the outputs and their difference. The PSNR between the SD-Turbo output for the original image and the output for the protected image is 41.06 dB and the SSIM is 0.9863. For comparison, the PSNR between the original and protected *inputs* is 45.49 dB with an SSIM of 0.9831. The mean absolute difference between the outputs is 1.35 levels, which is about 1.36 times the 1.00 level of the inputs.

Table 7.13 SD-Turbo Test on the Portrait Image

Measurement	Value
PSNR between inputs (original, protected)	45.49 dB
SSIM between inputs	0.9831
PSNR between model outputs	41.06 dB
SSIM between model outputs	0.9863
Mean absolute output difference	1.35 levels
Output resolution	512 × 488 pixels

The interpretation must be cautious. The model output changes by slightly more than the input does, which shows that the model does respond to the perturbation, but the amplification is only about 1.36 times and the outputs remain visually equivalent. In other words, at the default settings the perturbation does **not** disrupt this model in a way that a viewer would notice, and the result does not support a claim that the protected image resists SD-Turbo editing. This outcome is consistent with the design: the perturbation is model-agnostic noise, not an optimisation against the model’s

encoder or denoiser, and published immunization methods rely on such optimisation. The experiment is also limited to one image, one prompt and one seed. What the experiment does demonstrate is that the test harness works, is repeatable and exposes the effect, positive or negative, for any image and prompt. Section 8.4 lists target-model optimisation as the first item of future work. Section 7.12 repeats the target-model measurement on 500 images.

7.11 Running Time and Scalability

Table 7.14 and Fig. 7.11 show the time of the protection step as the image side grows from 128 to 1024 pixels, using the colour portrait resized to square images. The time grows approximately linearly with the number of pixels, with a slope of about 0.75 seconds per megapixel, from 0.19 seconds for a 512 × 512 image to 0.75 seconds for 1024 × 1024. Over the 14 test images, the protection step took 0.27 seconds on average and at most 0.52 seconds. At this speed 1000 images of typical size would be protected in a few minutes of CPU time, so the batch workflow is not limited by the algorithm. The measured times exclude the metric computation, in which SSIM dominates, and the signing step, which is negligible, and they are specific to the machine used.

Table 7.14 Protection Time Versus Image Size (Three Runs, Median)

Side (px)	Pixels (×10^6)	Median time (s)
128	0.016	0.001
256	0.066	0.027
384	0.147	0.097
512	0.262	0.188
768	0.590	0.466
1024	1.049	0.754

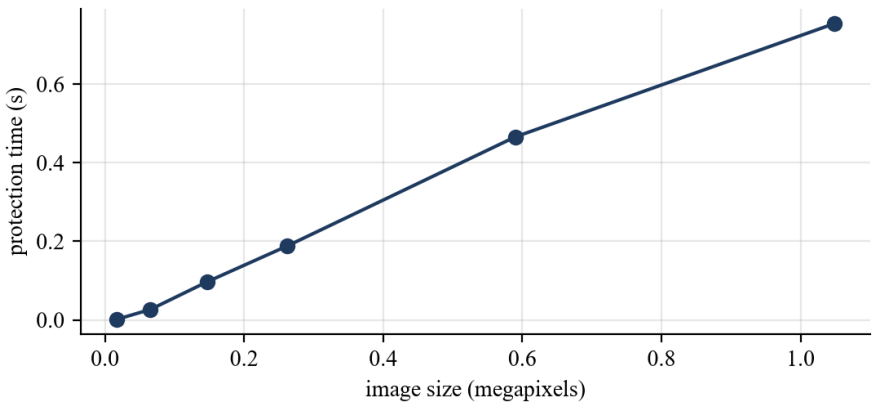


Fig. 7.11 Protection Time Versus Image Size

7.12 Evaluation on the MS-COCO Dataset

The 14-image study of Sections 7.1 to 7.11 allows a detailed analysis, but it is small. To check whether its main observations hold on a larger and more varied collection, a second experiment was run on 500 photographs from the MS-COCO dataset (Lin et al., 2014), which contains everyday scenes with people, animals, vehicles, food and indoor and outdoor settings. The images were placed in the data/input folder and processed with the evaluation runner run_evaluation.py, whose workflow is described in Section 4.1.20. The run was named coco500, and all 500 images were processed without error and all 500 were also passed through SD-Turbo.

7.12.1 Dataset and protocol

The file names of the images follow the twelve-digit identifier scheme of MS-COCO. The method used to choose the 500 images and any random-selection seed were not recorded when the run was made, so they are not stated here; the dataset source and the fixed protocol in Table 7.15 are the information that is available for reproduction. Every image was processed with the default settings of the web interface, and the SD-Turbo test used the same prompt and seed for the original and the protected image. The thresholds were fixed before the run and are stored in the file summary.json.

Table 7.15 Configuration and Criteria of the 500-Image Evaluation

Setting	Value
Dataset	MS-COCO, 500 images
Images processed	500 of 500 (no failures); SD-Turbo comparison for all 500
Perturbation budget	8/255
Seed	42
Compression-feedback iterations	4
JPEG qualities tested	95, 75, 50
Target model and prompt	SD-Turbo image-to-image; “a realistic photograph, natural lighting”
Quality pass	PSNR $\geq$ 30 dB and SSIM $\geq$ 0.95 between original and protected image
JPEG pass	SSIM between the original and the compressed protected image $\geq$ 0.90 at every tested quality
Target divergence	SSIM between the SD-Turbo outputs for the original and for the protected image $\leq$ 0.80
Combined success	Quality pass and target divergence on the same image

7.12.2 Why pass rates are reported instead of accuracy

CogniShield is not a classifier, so a classification accuracy would have no defined meaning. The evaluation therefore reports four pass rates, each defined as the percentage of images that satisfy a predefined criterion from Table 7.15. They describe the behaviour of the prototype under one protocol and must not be read as the

probability that an image is protected against editing by any model.

7.12.3 Aggregate results

Table 7.16 gives the aggregate results of the run, and Table 7.17 gives the distribution of the per-image values. Fig. 7.12 shows the charts that the runner generated.

Table 7.16 Aggregate Results on 500 MS-COCO Images

Measure	Result
Mean protected-image PSNR	39.06 dB
Mean protected-image SSIM	0.9774
Mean protected-image MSE	0.000141
Quality pass rate	99.6% (498 of 500 images)
JPEG pass rate	26.6% (133 of 500 images)
Target divergence rate	10.2% (51 of 500 images)
Combined success rate	10.2% (51 of 500 images)
Mean SD-Turbo output SSIM (original versus protected)	0.8834
Mean SD-Turbo output PSNR (original versus protected)	28.92 dB

Table 7.17 Distribution of the Per-Image Values (500 Images)

Measure	Mean	Median	Std. dev.	Minimum	Maximum
Protected-image PSNR (dB)	39.06	38.87	2.27	33.96	45.72
Protected-image SSIM	0.9774	0.9780	0.0068	0.9397	0.9961
SSIM after JPEG quality 95	0.9690	0.9718	0.0166	0.8734	0.9964
SSIM after JPEG quality 75	0.9032	0.9064	0.0347	0.7228	0.9850
SSIM after JPEG quality 50	0.8714	0.8746	0.0444	0.6548	0.9704
SD-Turbo output SSIM	0.8834	0.8952	0.0593	0.6398	0.9867
SD-Turbo output PSNR (dB)	28.92	28.46	3.56	19.54	42.15

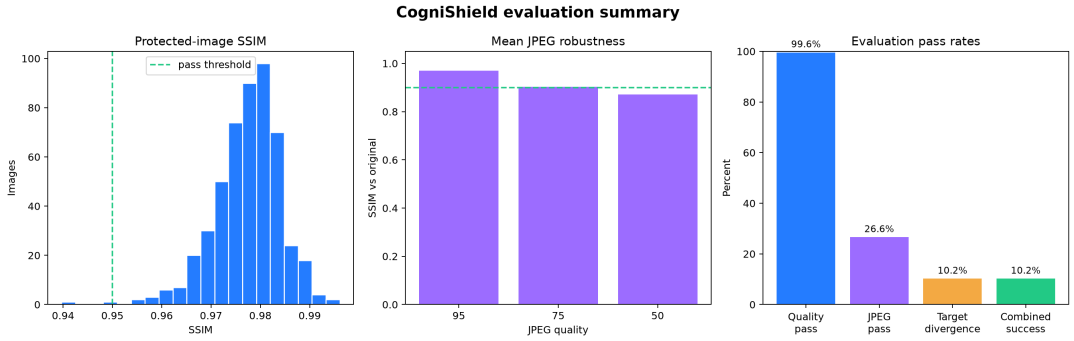


Fig. 7.12 Charts Generated by the Evaluation Runner for the 500 MS-COCO Images

The protected images meet the quality standard on almost the whole set. The mean PSNR is 39.06 dB and the mean SSIM is 0.9774, and 498 of the 500 images pass both thresholds. The two exceptions have PSNR values of 36.83 dB and 36.13 dB, well above 30 dB, but SSIM values of 0.9494 and 0.9397, just below the 0.95 threshold. The maximum absolute pixel change was 0.0314 on the 0–1 scale, which is exactly 8

levels, in all 500 images, so the budget was respected everywhere. The mean PSNR is lower than the 41.56 dB of the 14-image study, and the spread, from 33.96 dB to 45.72 dB, confirms the observation of Section 7.2 that the cost of protection depends on the content of the image.

7.12.4 Interpretation of the JPEG and target-model rates

The JPEG pass rate of 26.6 percent looks low, but it has to be read with care. The criterion compares the compressed protected image with the original, so it includes the loss that compression itself causes and not only the effect of the perturbation. By quality, the share of images with SSIM of at least 0.90 was 99.0 percent at quality 95, 57.4 percent at quality 75 and 26.6 percent at quality 50. The overall pass rate equals the rate at quality 50 because every image that passes at quality 50 also passes at the higher qualities. The 500-image run did not include an unprotected baseline that was compressed in the same way; that comparison was made only for the 14 images, in Table 7.6.

The target divergence rate is 10.2 percent: for 51 of the 500 images the SD-Turbo outputs for the original and for the protected image had an SSIM of at most 0.80. For the other 449 images the outputs stayed more similar than that. The median output SSIM was 0.8952, 234 images had an output SSIM above 0.90 and 49 had one above 0.95. The combined success rate equals the target divergence rate because all 51 images that diverged also passed the quality check, and only 6 of the 500 images satisfied the JPEG criterion and the divergence criterion together.

These results change the picture given by the single portrait of Section 7.10. For that image the SD-Turbo outputs differed by 41.06 dB in PSNR, which is close to the top of the range observed here (maximum 42.15 dB), so the portrait was one of the cases in which the model output stayed close. Over 500 images the mean output PSNR is 28.92 dB, and a minority of images produce a clearly different output. The result is nevertheless not a demonstration of protection. Image-to-image diffusion is sensitive to small changes of its input, so some divergence is expected even from noise that has no protective design, and the protocol contains no control run with unstructured noise of the same size. The divergence rate therefore shows that the perturbation can change the model output but not that the change is caused by the protective structure.

7.12.5 Evaluation dashboard

The results of the run can be inspected in the web application on the Evaluation page, which is opened from the link in the header or at the address http://127.0.0.1:5000/evaluation. The page shows the newest completed run: its name and image count, the four headline rates, the fixed protocol settings and the charts, and it offers downloads of results.csv and report.md. Fig. 7.13 shows the page for the coco500 run.

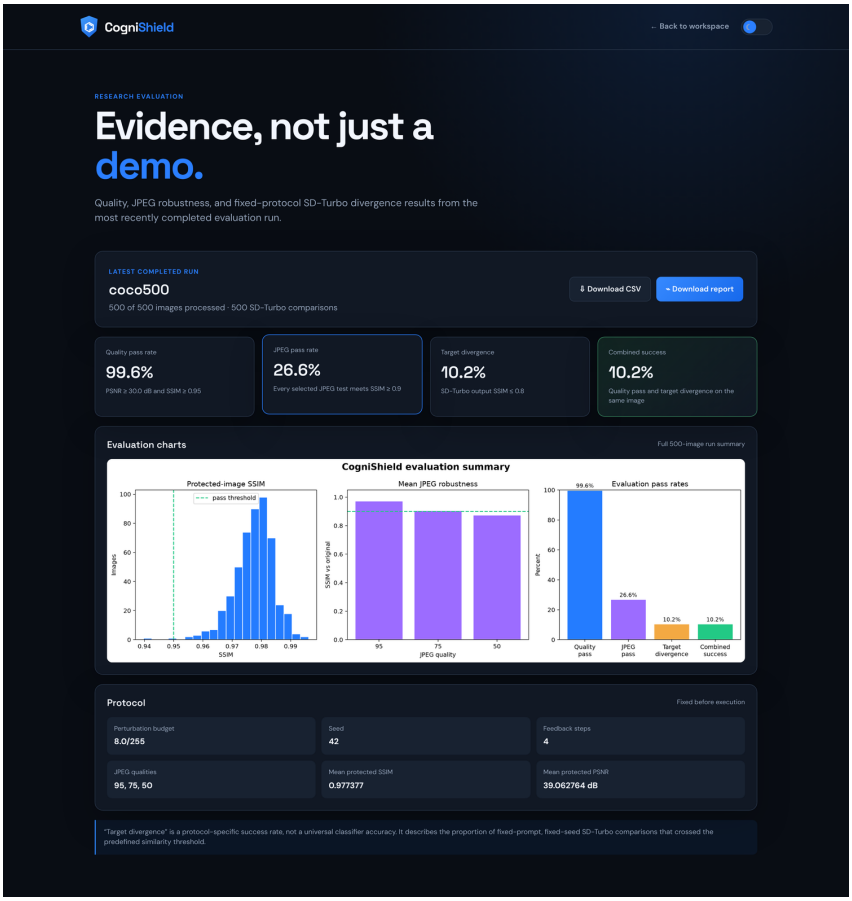


Fig. 7.13 Evaluation Dashboard of the Web Application for the coco500 Run

In summary, the larger experiment supports the earlier findings on imperceptibility and boundedness, shows that moderate JPEG compression keeps most protected images close to their originals while strong compression does not, and gives a first, protocol-specific measure of how often SD-Turbo outputs diverge. It also shows what remains undemonstrated: a controlled comparison that ties the divergence to the protective design.

7.13 Discussion

7.13.1 Assessment against the project goals

Table 7.18 summarises the extent to which each goal of Section 2.4 was met.

Table 7.18 Assessment Against the Project Goals

ID	Goal	Status	Evidence
G1	Imperceptible protection	Met	14 of 14 images above 30 dB; mean PSNR 41.56 dB; mean SSIM 0.9800; 500 MS-COCO images: mean PSNR 39.06 dB, mean SSIM 0.9774, 99.6% pass the quality thresholds
G2	Bounded change	Met	Maximum absolute change exactly 8.00 levels at budget 8 in all 14 images and in all 500 MS-COCO images
G3	Compression awareness	Met, partly effective	Feedback raises retention at q=95 and 75; little survives at q=50; JPEG pass rate 26.6% on 500 images
G4	Reproducibility	Met	Identical outputs for identical settings
G5	Verifiable provenance	Met	18 of 18 digests match; no tampered file matches
G6	Target-model comparison	Implemented	Side-by-side outputs and metrics; no visible disruption for the test portrait; 10.2% of 500 images crossed the divergence threshold
G7	Usability	Met by design	Single-page workflow with drag and drop and downloads
G8	Local operation	Met by design	Loopback server; local storage of images and keys

7.13.2 Strengths

The prototype delivers what it promises on imperceptibility, reproducibility and integrity. The perturbation is small, strictly bounded, and measured at every run. The pipeline is fast enough for interactive use and for batch experiments. The provenance mechanism is simple, uses well-established primitives and fails safely. The architecture isolates the heavy optional component, and every stage of the process can be inspected through the images and numbers that the interface displays.

7.13.3 Limitations

The results also expose limitations that must be stated plainly. First, the protective effect on a generative model was demonstrated only weakly: for the test portrait the SD-Turbo output was nearly unchanged, and in the 500-image evaluation only 10.2 percent of the images produced outputs that crossed the divergence threshold, without a control experiment that would show that the divergence is caused by the protective design. Second, the perturbation is removed to a large degree by low-pass operations such as rescaling, blurring and median filtering and by coarse JPEG compression. Third, the signed provenance is invalidated by any re-encoding, so a platform that recompresses an upload produces a copy that no longer verifies. Fourth, the local key proves only that a file came from this installation, not who the person is. These limitations do not invalidate the prototype as a measurement and demonstration

platform, but they bound the claims that may be made.

7.13.4 Threats to validity

The evaluation has several limits. The set of 14 images used for the detailed analysis is small and contains no large collection of faces, and the 500 MS-COCO images come from a single dataset with an unrecorded selection method and fixed pass-fail thresholds; the retention measure is a linear proxy and not a model-based effectiveness metric; the SD-Turbo experiment of Section 7.10 is limited to one image, one prompt and one seed, and the 500-image run has no control experiment with unstructured noise; timings depend on the machine; and no study with human viewers was carried out, so imperceptibility is judged by PSNR and SSIM and by the authors' inspection of the images. The conclusions about parameters, such as the suggestion of fewer iterations, are therefore indicative.

7.14 Cost Analysis

7.14.1 Software and licensing cost

All software components used in CogniShield are available without licence fees. Table 7.19 lists the main components with their cost for this project. The model weights for SD-Turbo are distributed under the licence of their publisher, whose terms should be reviewed before any commercial use.

Table 7.19 Software and Resource Cost for the Prototype

Item	Source	Cost for this project
Python, NumPy, Pillow, scikit-image, Flask, cryptography	Open-source packages	No licence fee
PyTorch, diffusers, transformers, accelerate	Open-source packages	No licence fee
SD-Turbo model weights	Publisher's model hub	No fee; publisher's licence terms apply
Development tools and editor	Free tools	No licence fee
Test images	Team's own images, scikit-image sample data and MS-COCO images	No fee; dataset terms of use apply
Hosting and servers	Not used; runs locally	None

7.14.2 Hardware and running cost

The core pipeline needs no special hardware and ran on an ordinary Windows computer. A GPU is needed only for the optional model test, and the team used hardware it already had, so no hardware was purchased. The running cost of the core pipeline is the processor time of the protection step, about 0.27 seconds per image on average in the test set, which is about 4.4 minutes of CPU time per 1000 images, and the storage of the

outputs, about 4.2 MB per run for an 801 × 775 image, which is about 4.2 GB per 1000 such images if all inspection images and JPEG variants are kept. The batch program stores the same kinds of files. Disk use can be reduced by deleting JPEG variants and inspection images, which are not part of the signed output.

7.14.3 Effort estimate

The project plan in Chapter 5 spans seventeen weeks for three students. As an illustration, if each team member contributed about eight hours per week, the total effort would be about 400 person-hours: this figure is an assumption for planning and costing purposes and not a record of hours worked. At typical junior-developer rates this would correspond to a development cost that is far larger than the running cost, which is the usual pattern for software of this kind. A deployed version would add costs for security review, key management, user support and compute for model-based optimisation.

7.15 Ethical and Societal Considerations

A tool that changes images and signs them touches several ethical questions, and the evaluation would be incomplete without them. The first is **false confidence**. The results of this chapter show that the evaluated perturbation does not visibly disrupt SD-Turbo and is largely removed by common processing, so presenting the tool as a shield could lead a user to believe that a photograph is safe when it is not. For this reason the interface and the report describe the output as a protected copy with measured properties and not as an immune image. The second is **dual use**. The same techniques can in principle be used to degrade other people’s images or to make a manipulated image look signed; the design limits this by running locally on images that the user supplies and by making the provenance claim narrow, but the code could be misused like any other.

The third consideration is **consent and data protection**. The project uses the team’s own photographs and public sample images, and the README directs users to process only images they own or are authorised to use. The portrait used in the demonstrations comes from the team’s own project runs. The fourth is **fairness of access**: protection that requires a GPU or an account on a cloud service favours those who can afford it, whereas a tool that runs on an ordinary computer in a fraction of a second is more broadly available. The fifth is **environmental cost**: the core

pipeline uses a negligible amount of computation per image, in contrast with iterative optimisation against large models, which is one argument for combining cheap, local protection with provenance before investing in heavier methods. Finally, any protection and provenance system should complement, and not replace, legal, platform and educational measures against image-based abuse.

CHAPTER 8
SUMMARY

This report presented CogniShield, a local prototype for edge-based adversarial immunization of digital images with cryptographic provenance. The project started from the observation that detection of manipulated images is reactive and that a complementary, proactive defence is worth exploring: an image owner could alter a photograph imperceptibly before publication so that it is harder to misuse, and sign the published file so that its integrity can be checked. The prototype turns this idea into a working pipeline that runs entirely on the user's own computer.

8.1 Summary of the Work

The system takes an input image, converts it to a floating-point RGB array and builds a seeded, high-frequency noise field. An edge mask derived from brightness gradients allocates more of the allowed change to detailed regions, and every pixel change is clipped to a budget of at most 16 and by default 8 levels out of 255. A feedback loop that uses the real JPEG encoder adds back a fraction of the residual that compression removes, so that more of the perturbation survives recompression. The protected PNG is accompanied by a noise preview and a difference image magnified twelve times. The tool then reports PSNR, SSIM, MSE, mean absolute delta and maximum absolute delta, evaluates the protected image after JPEG compression at several qualities, hashes the protected file with SHA-256 and signs the digest with Ed25519. A manifest records the digest, the signature, the settings and the signing time, and the signature is verified immediately. An optional module sends the original and protected images through a local SD-Turbo image-to-image pipeline in a separate process so that the model's behaviour can be compared without risking the main workflow.

The software consists of six small library modules, a Flask web application with a dark-by-default responsive interface and a light-mode toggle, and command-line programs for single images and for batches, and an evaluation runner with a dashboard that reports pass rates over a folder of images. The chapters of this report documented the objective and background, the problem and threat model, the technical specification, the design and algorithms with their mathematical definitions, the schedule, a demonstration

on real runs and an experimental evaluation on 14 images and on 500 images of the MS-COCO dataset.

8.2 Key Findings

The evaluation led to the following findings, each of which is supported by the measurements in Chapter 7.

- ✔ **The protection is imperceptible by the measures used.** On 14 varied images at the default settings the mean PSNR was 41.56 dB and the mean SSIM was 0.9800; every image stayed above 30 dB and above 0.95, and the maximum pixel change was exactly 8 levels.
- ✔ **The perturbation budget controls the quality cost predictably.** PSNR fell by about 11.18 dB when the budget was increased from 1 to 16, and each doubling of the budget cost between 2.5 and 3.2 dB.
- ✔ **Compression feedback helps, with diminishing returns.** Feedback multiplied the surviving fraction of the perturbation by about 2.7 at JPEG quality 95 and 5.0 at quality 75, at the price of about 9.9 dB of PSNR over four iterations. The benefit peaked after three to five iterations and declined beyond that.
- ✔ **Compression and filtering remove most of the perturbation at low quality.** About half of the perturbation along its own direction survived at qualities 95 and 75, less than a fifth survived at quality 50, and rescaling, blurring and median filtering retained between 0.18 and 0.30.
- ✔ **The edge mask places the perturbation where intended.** The mean perturbation on the most detailed pixels was larger than on the flattest pixels in all 14 images, by a mean factor of 2.37.
- ✔ **The pipeline is reproducible and fast.** Identical settings gave bit-identical outputs, the visual quality varied little between seeds, and a 512 × 512 image was protected in about 0.19 seconds.
- ✔ **Provenance detects tampering.** All 18 recorded manifests matched their files, and a single flipped bit changed the digest in every case.
- ✔ **The final perturbation is image-adaptive.** The initial noise averages only 0.49 levels while the final perturbation averages 1.62 levels, and the final perturbation

correlates with the image's own JPEG residual (0.75 on average) more than with the seeded noise (0.49); only 1.7 percent of the pixel values reach the budget.

- ✔ **No protective effect on SD-Turbo was demonstrated for the test portrait.** The model outputs for the original and the protected image were visually almost identical, with a PSNR of 41.06 dB between them.
- ✔ The protection stays imperceptible at scale. On 500 MS-COCO images the mean PSNR was 39.06 dB and the mean SSIM was 0.9774; 99.6 percent of the images met both quality thresholds, and the maximum pixel change was exactly 8 levels in every image.
- ✔ Moderate JPEG compression keeps protected images close to their originals, strong compression does not. The share of the 500 images whose SSIM against the original stayed at or above 0.90 was 99.0 percent at quality 95, 57.4 percent at quality 75 and 26.6 percent at quality 50.
- ✔ The effect on SD-Turbo is measurable but limited. For 10.2 percent of the 500 images the SD-Turbo outputs for the original and the protected image had an SSIM of at most 0.80, and the mean output SSIM was 0.8834. A control experiment with unstructured noise is still needed to attribute the divergence to the protective design.

8.3 Limitations

CogniShield is a prototype and its limits should be kept in view. The perturbation is generated without access to the internals of any target model, and the single-portrait SD-Turbo experiment showed no visible disruption, while the 500-image evaluation has no control run. The perturbation lives in the high-frequency band and is therefore largely removed by rescaling, blurring, median filtering and strong JPEG compression. Signing the exact bytes of the PNG means that any re-encoding invalidates the signature of the re-encoded copy. The local key proves only that a file came from this installation, and the key is stored unencrypted. The detailed evaluation uses a small image set and a linear retention proxy, the 500-image evaluation comes from one dataset with fixed pass-fail thresholds, both use a single target model, and no human perceptual study was carried out. The tool does not guarantee resistance to every model, image editor, transformation, crop or re-encoding pipeline, and it should be used only with images that the user owns

or is authorised to process.

8.4 Recommendations and Future Work

The results suggest the following recommendations and directions, in order of expected value.

- ✔ **Target-model optimisation.** Add gradient-based optimisation of the perturbation against the encoder or denoiser of the target generative model, in the manner of published immunization methods, and use the existing SD-Turbo harness to measure the effect on outputs. This is the most important step towards a demonstrable protective effect.
- ✔ **Face-swap and further model evaluation.** Extend the target-model tests to face-swap systems, inpainting and other image-to-image pipelines, and run them over a larger set of faces, prompts and seeds.
- ✔ **Better default parameters.** The sweep suggests that two or three feedback iterations and a budget between 4 and 8 offer a better quality and retention trade-off than the current defaults; a larger study could confirm and refine this.
- ✔ **Robustness to resizing and filtering.** Explore perturbations with lower-frequency components, differentiable approximations of resizing and JPEG, or expectation-over-transformation training so that the signal survives common platform processing.
- ✔ **Watermarking in addition to sidecar signatures.** Embed a robust watermark that survives recompression and combine it with the signed manifest so that provenance is not lost when a file is re-encoded.
- ✔ **Interoperable provenance.** Align the manifest with an open provenance standard and support certificate-based trust and key protection with a passphrase or a hardware-backed store.
- ✔ **Social-media transformation tests.** Measure retention after real upload and download cycles on common platforms.
- ✔ **Mobile and on-device deployment.** Convert the protection pipeline to a portable runtime such as ONNX or TFLite for use on phones, once the local prototype is validated.

- ✔ **Perceptual validation.** Run a small study with human viewers to confirm that the protected images are indistinguishable at normal viewing sizes.
- ✔ **Hardening.** Validate run identifiers on the result routes, serve the application through a production-grade server if it is ever exposed beyond the local machine and add automated tests for the library functions.
- ✔ Control experiments and dataset documentation. Repeat the 500-image protocol with random noise of the same magnitude, with further prompts and seeds, and record the image-selection method and the random seed, so that the target-divergence rate can be attributed to the protective design.
- ✔ Broader evaluation protocol. Extend the evaluation runner to further datasets, target models and transformations such as rescaling, blurring and denoising, so that the pass rates cover the sharing paths that matter in practice.

8.5 Conclusion

CogniShield shows that a complete, inspectable pipeline for image immunization and provenance can be built with modest resources and run locally in a fraction of a second per image. It meets its goals for imperceptibility, boundedness, reproducibility, measurement and verifiable integrity (on 500 MS-COCO images, 99.6 percent of the protected images met the quality thresholds), and it is honest about what it does not yet achieve: the evaluated perturbation is not a proven defence against a diffusion editor, and much of it is removed by common image processing. The value of the project therefore lies in the platform it provides, a transparent baseline against which stronger, model-aware protection methods can be implemented, measured and compared, and in the demonstration that protection, measurement and cryptographic provenance can be combined in a single tool that respects the privacy of the person it is meant to protect.

CHAPTER 9
REFERENCES

[1.] Bala, A., Chowdhury, R., Jaiswal, R., & Roheda, S. (2025). DCT-Shield: A robust frequency domain defense against malicious image editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 18876–18884).

[2.] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering, 2*(2), 77–89. https://doi.org/10.1007/s13389-012-0027-1

[3.] Carlini, N., & Wagner, D. (2017). Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)* (pp. 39–57). IEEE. https://doi.org/10.1109/SP.2017.49

[4.] Cherepanova, V., Goldblum, M., Foley, H., Duan, S., Dickerson, J., Taylor, G., & Goldstein, T. (2021). LowKey: Leveraging adversarial attacks to protect social media users from facial recognition. In *International Conference on Learning Representations (ICLR 2021)*.

[5.] Choi, J. S., Lee, K., Jeong, J., Xie, S., Shin, J., & Lee, K. (2025). DiffusionGuard: A robust defense against malicious diffusion-based image editing. In *International Conference on Learning Representations (ICLR 2025)*.

[6.] Dziugaite, G. K., Ghahramani, Z., & Roy, D. M. (2016). *A study of the effect of JPG compression on adversarial images*. arXiv:1608.00853.

[7.] Golaszewski, E., Krawetz, N., Sherman, A. T., Zieglar, E., Matukumalli, S. K., Yus, R., Kegley, C. L., Barthel, M., Bowman, W., Barot, B., & Kullman, K. (2026). *Verifying provenance of digital media: Security analysis of C2PA and its implementation* (Paper 2026/804). Cryptology ePrint Archive. https://eprint.iacr.org/2026/804

[8.] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR 2015)*. arXiv:1412.6572.

[9.] Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.

[10.] Guan, J., Zhao, Y., Xu, Z., Meng, C., Xu, K., & Zhao, Y. (2024). Adversarial robust safeguard for evading deep facial manipulation. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(1), 118–126. https://doi.org/10.1609/aaai.v38i1.27762

[11.] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

[12.] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models.

Advances in Neural Information Processing Systems, 33, 6840–6851.

[13.] Huynh-Thu, Q., & Ghanbari, M. (2008). Scope of validity of PSNR in image/video quality assessment. *Electronics Letters*, *44*(13), 800–801. https://doi.org/10.1049/el:20080522

[14.] Josefsson, S., & Liusvaara, I. (2017). *Edwards-curve digital signature algorithm (EdDSA)* (RFC 8032). Internet Engineering Task Force. https://doi.org/10.17487/RFC8032

[15.] Kim, J., Nam, Y., Kim, M., Kim, S., & Jeong, J. (2025). BlurGuard: A simple approach for robustifying image protection against AI-powered editing. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.

[16.] Kurakin, A., Goodfellow, I. J., & Bengio, S. (2017). Adversarial examples in the physical world. In *ICLR 2017 Workshop Track*. arXiv:1607.02533.

[17.] Li, B., Sun, J., Poskitt, C. M., & Wang, X. (2023). *How generalizable are deepfake image detectors? An empirical study*. arXiv:2308.04177.

[18.] Liang, C., Wu, X., Hua, Y., Zhang, J., Xue, Y., Song, T., Xue, Z., Ma, R., & Guan, H. (2023). Adversarial example does good: Preventing painting imitation from diffusion models via adversarial examples. In *Proceedings of the 40th International Conference on Machine Learning* (PMLR 202).

[19.] Liu, X., Peng, H., Zheng, N., Yang, Y., Hu, H., & Yuan, Y. (2023). EfficientViT: Memory efficient vision transformer with cascaded group attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 14420–14430).

[20.] Lo, L., Yeo, C. Y., Shuai, H.-H., & Cheng, W.-H. (2024). Distraction is all you need: Memory-efficient image immunization against diffusion-based image editing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 24462–24471).

[21.] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR 2018)*.

[22.] Meng, C., He, Y., Song, Y., Song, J., Wu, J., Zhu, J.-Y., & Ermon, S. (2022). SDEdit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations (ICLR 2022)*.

[23.] Mirsky, Y., & Lee, W. (2021). The creation and detection of deepfakes: A survey. *ACM Computing Surveys*, *54*(1), Article 7. https://doi.org/10.1145/3425780

[24.] National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

[25.] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style,

high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024–8035.

[26.] Qin, D., Leichner, C. H., Delakis, M., Fornoni, M., Luo, S., Yang, F., Wang, W., Banbury, C., Ye, C., Akin, B., Aggarwal, V., Zhu, T., Moro, D., & Howard, A. (2024). MobileNetV4: Universal models for the mobile ecosystem. In *Computer Vision – ECCV 2024* (pp. 78–96). Springer.

[27.] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10684–10695).

[28.] Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to detect manipulated facial images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 1–11).

[29.] Salman, H., Khaddaj, A., Leclerc, G., Ilyas, A., & Madry, A. (2023). Raising the cost of malicious AI-powered image editing. In *Proceedings of the 40th International Conference on Machine Learning* (PMLR 202).

[30.] Sandoval-Segura, P., Geiping, J., & Goldstein, T. (2023). *JPEG compressed images can bypass protections against AI editing*. arXiv:2304.02234.

[31.] Sauer, A., Lorenz, D., Blattmann, A., & Rombach, R. (2023). *Adversarial diffusion distillation*. arXiv:2311.17042.

[32.] Shan, S., Cryan, J., Wenger, E., Zheng, H., Hanocka, R., & Zhao, B. Y. (2023). Glaze: Protecting artists from style mimicry by text-to-image models. In *32nd USENIX Security Symposium*.

[33.] Shan, S., Wenger, E., Zhang, J., Li, H., Zheng, H., & Zhao, B. Y. (2020). Fawkes: Protecting privacy against unauthorized deep learning models. In *29th USENIX Security Symposium* (pp. 1589–1604).

[34.] Shin, R., & Song, D. (2017). JPEG-resistant adversarial images. In *NIPS 2017 Workshop on Machine Learning and Computer Security*.

[35.] Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., & Fergus, R. (2014). Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR 2014)*.

[36.] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). Deepfakes and beyond: A survey of face manipulation and fake detection. *Information Fusion, 64*, 131–148. https://doi.org/10.1016/j.inffus.2020.06.014

[37.] van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & the scikit-image contributors. (2014). scikit-image: Image processing in Python. *PeerJ, 2*, e453. https://doi.org/10.771

7/peerj.453

[38.] Van Le, T., Phung, H., Nguyen, T. H., Dao, Q., Tran, N. N., & Tran, A. (2023). Anti-DreamBooth: Protecting users from personalized text-to-image synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.

[39.] von Platen, P., Patil, S., Lozhkov, A., et al. (2022). *Diffusers: State-of-the-art diffusion models* [Computer software]. Hugging Face. https://github.com/huggingface/diffusers

[40.] Wallace, G. K. (1991). The JPEG still picture compression standard. *Communications of the ACM, 34*(4), 30–44. https://doi.org/10.1145/103085.103089

[41.] Wang, Z., & Bovik, A. C. (2009). Mean squared error: Love it or leave it? A new look at signal fidelity measures. *IEEE Signal Processing Magazine, 26*(1), 98–117.

[42.] Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing, 13*(4), 600–612. https://doi.org/10.1109/TIP.2003.819861

[43.] Xu, C., & Singh, G. (2024). Robust universal adversarial perturbations. In *Proceedings of the 41st International Conference on Machine Learning* (PMLR 235, pp. 55241–55266).

[44.] Xu, X., Butt, M. A., Kamath, S., & Raducanu, B. (2024). Privacy protection in personalized diffusion models via targeted cross-attention adversarial attack. In *NeurIPS 2024 Workshops: SafeGenAi*.

[45.] Yan, Z., Zhang, Y., Fan, Y., & Wu, B. (2023). UCF: Uncovering common features for generalizable deepfake detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 22412–22423).

[46.] Zhao, Z., Duan, J., Xu, K., Wang, C., Zhang, R., Du, Z., Guo, Q., & Hu, X. (2024). Can protective perturbation safeguard personal data from being exploited by Stable Diffusion? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 24398–24407).

[47.] Zhu, P., Takahashi, T., & Kataoka, H. (2024). Watermark-embedded adversarial examples for copyright protection against diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 24420–24430).

Websites

[48.] Coalition for Content Provenance and Authenticity. (n.d.). *C2PA technical specification*. https://c2pa.org/specifications/

[49.] Pallets Projects. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

[50.] Python Software Foundation. (n.d.). *Python 3.12 documentation*. https://docs.pyt

hon.org/3.12/

[51.] Stability AI. (n.d.). *SD-Turbo* [Model card]. Hugging Face. https://huggingface.co/stabilityai/sd-turbo

Data Sets

[52.] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

[53.] scikit-image contributors. (n.d.). *skimage.data: Example images bundled with scikit-image*. https://scikit-image.org/docs/stable/api/skimage.data.html

APPENDIX A

This appendix collects sample code, sample outputs and additional figures from the project. Only representative extracts are included: the algorithmic core, the metric and provenance functions, the optional model runner, the experiment helper used for this report, the dataset-evaluation runner, the structure of the output files and the commands needed to reproduce the results. Page numbers are not shown in this appendix.

A.1 Quick-Start Commands

The following commands, run in PowerShell from the project folder, install the dependencies and start the application. The first block performs the full setup, including the optional model dependencies; the second shows the core-only alternative and the command-line programs.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\setup.ps1
.\.venv\Scripts\python.exe app.py          # then open http://127.0.0.1:5000
py -3.12 -m venv .venv                      # core-only alternative
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\pip.exe install -r requirements-core.txt

.\.venv\Scripts\python.exe run_pipeline.py data\input\example.jpg --epsilon
0.03137 --seed 42 --iterations 4 --qualities 95 75 50
.\.venv\Scripts\python.exe run_batch.py --input-dir data\input
```

A.2 Protection Module (protector.py)

The first listing shows the configuration object with its default values, the second the edge mask, the third the high-frequency noise generator and the fourth the core function with the feedback loop. They implement the formulas of Section 4.1.

```
@dataclass(frozen=True)
class ProtectionConfig:
    epsilon: float = 8.0 / 255.0
    seed: int = 42
    iterations: int = 4
    compression_weight: float = 0.16
    jpeg_qualities: tuple[int, ...] = (95, 75)
    edge_weight: float = 0.65
def _edge_mask(image_arr: np.ndarray, edge_weight: float) -> np.ndarray:
    gray = image_arr.mean(axis=2)
    grad_y = np.zeros_like(gray)
    grad_x = np.zeros_like(gray)
    grad_y[1:, :] = np.abs(gray[1:, :] - gray[:-1, :])
    grad_x[:, 1:] = np.abs(gray[:, 1:] - gray[:, :-1])
```

```
    edges = grad_x + grad_y
    if edges.max() > 0:
        edges = edges / edges.max()
    mask = (1.0 - edge_weight) + edge_weight * edges
    return mask[..., None].astype(np.float32)
def _high_frequency_noise(shape: tuple[int, int, int], seed: int) ->
np.ndarray:
    rng = np.random.default_rng(seed)
    raw = rng.normal(0.0, 1.0, size=shape).astype(np.float32)
    raw_image = Image.fromarray(((raw - raw.min()) / (raw.max() - raw.min()
    + 1e-8) * 255).astype(np.uint8))
    blurred =
    np.asarray(raw_image.filter(ImageFilter.GaussianBlur(radius=1.2)),
    dtype=np.float32)
    blurred = (blurred / 255.0) * 2.0 - 1.0
    high = raw - blurred
    high = high / (np.max(np.abs(high)) + 1e-8)
    return high.astype(np.float32)
def protect_image(image: Image.Image, config: ProtectionConfig) ->
tuple[Image.Image, Image.Image]:
    """Create an imperceptible, compression-aware perturbation.

    This MVP uses high-frequency and edge-masked perturbations plus JPEG
    feedback. The
    target-model evaluation module measures whether the protected image
    disrupts SD-Turbo.
    """
    base = pil_to_float_array(image)
    noise = _high_frequency_noise(base.shape, config.seed)
    mask = _edge_mask(base, config.edge_weight)
    perturbation = np.clip(noise * mask * config.epsilon, -config.epsilon,
    config.epsilon)

    for _ in range(max(0, config.iterations)):
        candidate = np.clip(base + perturbation, 0.0, 1.0)
        feedback = np.zeros_like(candidate)
        for quality in config.jpeg_qualities:
           feedback += candidate - _jpeg_array_roundtrip(candidate, quality)
        feedback /= max(1, len(config.jpeg_qualities))
        perturbation = np.clip(
            perturbation + config.compression_weight * feedback,
            -config.epsilon,
            config.epsilon,
        )

    protected_arr = np.clip(base + perturbation, 0.0, 1.0)
    protected = float_array_to_pil(protected_arr)
    noise_preview = Image.fromarray(
        ((np.abs(perturbation) / max(config.epsilon, 1e-8)) * 255.0).clip(0,
        255).astype(np.uint8),
        mode="RGB",
    )
    return protected, noise_preview
```

A.3 Quality Metrics (metrics.py)

```
def calculate_metrics(original: Image.Image, protected: Image.Image) ->
ImageQualityMetrics:
    if original.size != protected.size:
        protected = protected.resize(original.size,
        Image.Resampling.LANCZOS)

    original_arr = pil_to_float_array(original)
```

```
    protected_arr = pil_to_float_array(protected)
    diff = original_arr - protected_arr
    mse = float(np.mean(diff**2))
    psnr = float("inf") if mse == 0 else float(20.0 * math.log10(1.0 /
    math.sqrt(mse)))

    ssim_value: float | None
    try:
        from skimage.metrics import structural_similarity

        ssim_value = float(
            structural_similarity(original_arr, protected_arr,
            channel_axis=2, data_range=1.0)
        )
    except Exception:
        ssim_value = None

    return ImageQualityMetrics(
        psnr=psnr,
        ssim=ssim_value,
        mse=mse,
        mean_abs_delta=float(np.mean(np.abs(diff))),
        max_abs_delta=float(np.max(np.abs(diff))),
    )
```

A.4 Provenance Module (provenance.py)

```
def sign_image(
    image_path: str | Path,
    key_dir: Path,
    metadata: dict[str, Any] | None = None,
) -> Path:
    image_path = Path(image_path)
    private_path, public_path = ensure_keys(key_dir)
    image_hash = sha256_file(image_path)
    private_key = _load_private_key(private_path)
    signature = private_key.sign(bytes.fromhex(image_hash))

    manifest = {
        "project": "CogniShield",
        "algorithm": "Ed25519 over SHA-256(image_bytes)",
        "image_file": image_path.name,
        "image_hash_sha256": image_hash,
        "signature_base64": base64.b64encode(signature).decode("ascii"),
        "public_key_file": public_path.name,
        "signed_at_utc": datetime.now(timezone.utc).isoformat(),
        "metadata": metadata or {},
    }
    manifest_path = image_path.with_suffix(image_path.suffix +
    ".provenance.json")
    manifest_path.write_text(json.dumps(manifest, indent=2),
    encoding="utf-8")
    return manifest_path


def verify_image(image_path: str | Path, manifest_path: str | Path,
key_dir: Path) -> VerificationResult:
    image_path = Path(image_path)
    manifest_path = Path(manifest_path)
    public_path = key_dir / PUBLIC_KEY_FILE

    if not image_path.exists():
        return VerificationResult(False, "Image file does not exist.")
```

```
    if not manifest_path.exists():
        return VerificationResult(False, "Provenance manifest does not
        exist.")
    if not public_path.exists():
        return VerificationResult(False, "Public key does not exist.")

    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    current_hash = sha256_file(image_path)
    recorded_hash = manifest.get("image_hash_sha256")
    if current_hash != recorded_hash:
        return VerificationResult(False, "Image hash changed after
        signing.", current_hash)

    signature = base64.b64decode(manifest["signature_base64"])
    public_key = _load_public_key(public_path)
    try:
        public_key.verify(signature, bytes.fromhex(current_hash))
    except InvalidSignature:
        return VerificationResult(False, "Signature does not match the
        public key.", current_hash)

    return VerificationResult(True, "Signature and image hash are valid.",
    current_hash)
```

A.5 Optional SD-Turbo Runner (sd_turbo.py)

```
    def run(
        self,
        image: Image.Image,
        prompt: str,
        strength: float = 0.55,
        steps: int = 2,
        seed: int = 42,
    ) -> SDTurboResult:
        self.load()

        import torch

        safe_strength = min(max(strength, 0.05), 1.0)
        effective_steps = max(2, steps, math.ceil(1.0 / safe_strength))
        prepared = resize_for_model(image, self.max_side)
        generator = torch.Generator(device=self._device).manual_seed(seed)
        result = self._pipe(
            prompt=prompt,
            image=prepared,
            num_inference_steps=effective_steps,
            strength=safe_strength,
            guidance_scale=0.0,
            generator=generator,
        ).images[0]
        return SDTurboResult(
            image=result,
            device=self._device,
            prompt=prompt,
            strength=safe_strength,
            steps=effective_steps,
        )
```

A.6 Experiment Helper Used for This Report

The retention measure of Section 7.1.3 was computed with the short function below, which compares the change left by a transformation with the original perturbation. The same script ran the budget sweep, the iteration sweep, the seed study and the transformation study using the library functions.

```
def delta(o, p):
    return pil_to_float_array(p) - pil_to_float_array(o)

def retention(o, p, tf):
    d = delta(o, p)
    a = pil_to_float_array(tf(p)) - pil_to_float_array(tf(o))
    return float((d * a).sum() / ((d * d).sum() + 1e-12))
```

A.7 Structure of the Command-Line Report

The single-image program writes *report.json* with the keys listed below, and the batch program writes *summary.csv* whose columns are sorted alphabetically.

```
report.json
  input, protected, noise_preview, difference
  metrics:       psnr, ssim, mse, mean_abs_delta, max_abs_delta
  compression:   {"95": {...metrics...}, "75": {...}, "50": {...}}
  provenance_manifest
  verification:  {valid, reason, image_hash}

summary.csv columns (alphabetical)
  height, image, jpeg_50_psnr, jpeg_50_ssim, jpeg_75_psnr, jpeg_75_ssim,
  jpeg_95_psnr, jpeg_95_ssim, max_abs_delta, mean_abs_delta, mse, psnr,
  ssim, width
```

A.8 Requirement Files

```
requirements-core.txt          cryptography, numpy, opencv-python, pandas,
Pillow, scikit-image, Flask
requirements-ai.txt            accelerate, diffusers, hf_xet,
huggingface_hub, matplotlib, safetensors, transformers
requirements-torch-cu128.txt   --index-url
https://download.pytorch.org/whl/cu128 ; torch ; torchvision
requirements.txt               -r requirements-core.txt ; -r
requirements-ai.txt
```

A.9 Additional Sample Screens and Outputs

The figures below show further protected images from the experiments. In each

figure the panels are the original, the protected image, the difference image magnified twelve times and the noise preview.

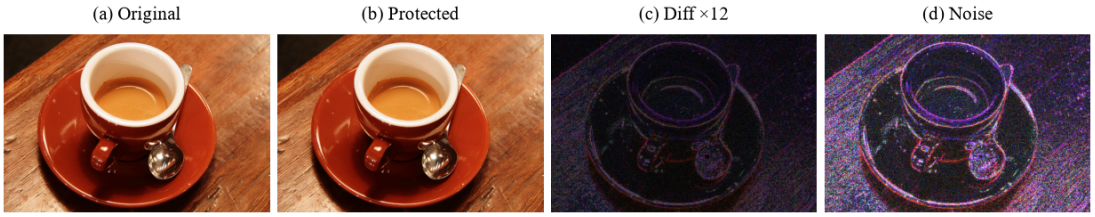


Fig. A.1 Protection of the Still-Life Image

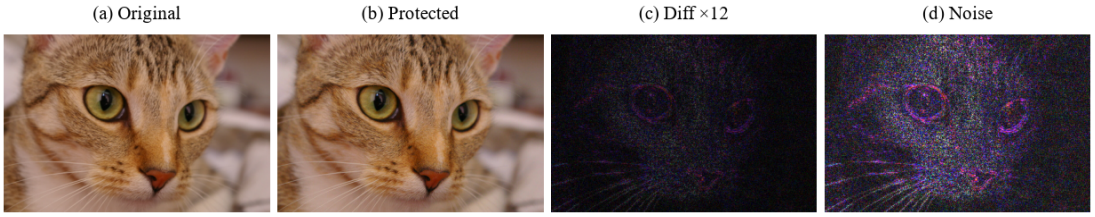


Fig. A.2 Protection of the Cat Photograph

A.10 Dataset Evaluation Runner (run_evaluation.py)

The dataset evaluation of Section 7.12 was produced by run_evaluation.py. The first listing shows how a single image is judged against the four criteria; the second shows the commands used to run the 500-image evaluation and to open the dashboard; the third shows the summary.json written for the coco500 run.

```
row = {"image": str(image_path), "filename": image_path.name, "status":
"ok"}
original = load_rgb_image(image_path)
protected, _noise = protect_image(original, config)
quality = calculate_metrics(original, protected)
quality_pass = quality.psnr >= args.quality_psnr_threshold and quality_ssim
>= args.quality_ssim_threshold

compression = evaluate_compression(original, protected, args.qualities)
jpeg_pass = all(m.ssim >= args.jpeg_ssim_threshold for m in
compression.values())

original_target = tester.run(original, prompt=args.prompt,
seed=args.seed).image
protected_target = tester.run(protected, prompt=args.prompt,
seed=args.seed).image
target_metrics = calculate_metrics(original_target, protected_target)
target_success = target_metrics.ssim <= args.target_ssim_threshold
row["combined_success"] = bool(quality_pass and target_success)
```

```
.\.venv\Scripts\python.exe run_evaluation.py --input-dir data\input
--run-name coco500
.\.venv\Scripts\python.exe run_evaluation.py --input-dir data\input
--max-images 10 --skip-model --run-name quality_smoke_test
.\.venv\Scripts\python.exe app.py
http://127.0.0.1:5000/evaluation
```


```
{
  "run_name": "coco500",
  "images_requested": 500,
  "images_processed": 500,
  "images_failed": 0,
  "target_images_evaluated": 500,
  "config": {"epsilon_pixels": 8.0, "seed": 42, "iterations": 4,
  "qualities": [95, 75, 50],
             "prompt": "a realistic photograph, natural lighting"},
  "criteria": {"quality_psnr_min": 30.0, "quality_ssim_min": 0.95,
               "jpeg_ssim_min": 0.9, "target_output_ssim_max": 0.8},
  "averages": {"quality_psnr": 39.062764, "quality_ssim": 0.977377,
  "quality_mse": 0.000141,
               "target_output_ssim": 0.883378, "target_output_psnr":
               28.920906},
  "rates": {"quality_pass_rate": 99.6, "jpeg_pass_rate": 26.6,
            "target_divergence_rate": 10.2, "combined_success_rate": 10.2}
}
```